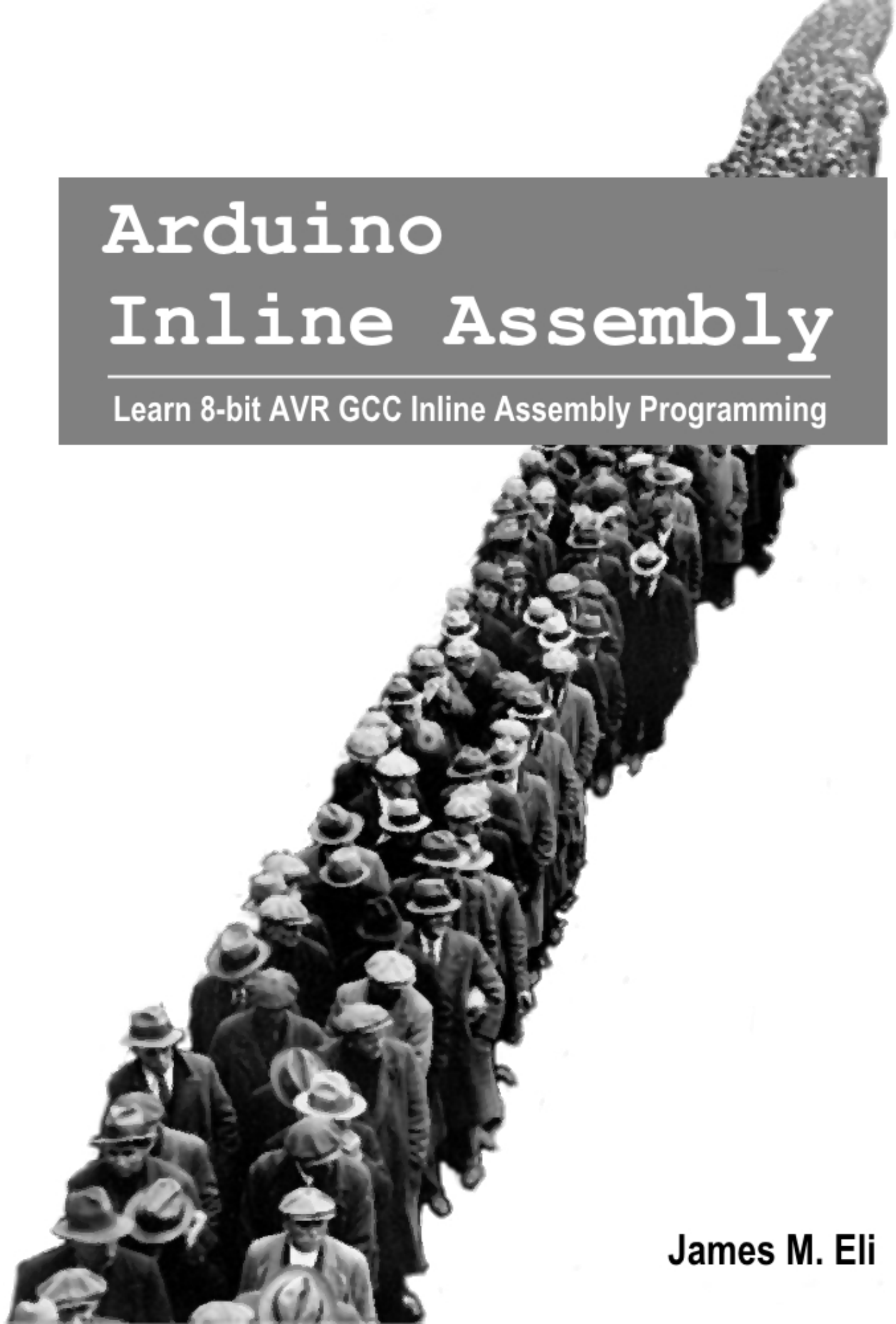
# Arduino Inline Assembly

## Learn 8-bit AVR GCC Inline Assembly Programming



James M. Eli

# Table of Contents

# Preface

## Motivation

Learning inline assembly language on the Arduino AVR 8-bit platform is a daunting task for many (at least it was for me). Besides the cryptic syntax and the high level of understanding the semi-official documentation assumes, there exists very little information about GCC inline assembler coding. The main focus of existing documentation is the "Cookbook", which dates from 2002. Trust me when I state, any neophyte needs to spend hours searching and studying piece-meal examples while possessing overwhelming patience in order to crack the code.

At least I did.

Hopefully this series of tutorials will help alleviate many of the discouraging troubles I encountered while teaching myself inline assembly coding. An Arduino Inline Assembly Tutorial was long overdue!

## Who This Book is for

This book has been written primarily for individual with one of the following qualifications:

- Knowledge of the Arduino and/or programmed Arduino inside the Arduino IDE
- Experienced C language embedded programmer

It is important to realize this book assumes at least a basic understanding of the underlying Arduino hardware (ports, timers, interrupts, USART, I2C, SPI, etc.).

Hopefully the reader has already written programs for the Arduino utilizing the functionality provided by the IDE, or has a general understanding of microcontroller programming. A wealth of information on the Arduino boards and software is available at the website:

`http://www.arduino.cc/`

No attempt is made in this book to teach how to interface to the underlying hardware of the Arduino AVR microcontroller. If a reader needs any specifics or details about the hardware, they should be comfortable with consulting the ATMEL datasheet. Furthermore, familiarity with the GCC compiler, assembler and/or general compiler functionality is helpful but not required.

## Why Put Yourself Through this?

Why learn inline assembly language programming? Many experts loudly proclaim that current optimizing compilers are capable of producing code that is as good, if not better than hand-coded assembly language. However, keep in mind a large percentage of the AVR Libc library is hand-coded assembly. That should be something that at least makes you go, “hmmm?”

Any serious development on the Arduino platform, or an ATMEL based 8-bit AVR microcontroller requires an examination of the underlying assembly code produced by the compiler. This code should be checked for correctness, bugs, and inefficiencies. If nothing else, learning AVR assembly language will give you the knowledge to determine what is going on deep inside of your program.

There are times when using assembly is needed, and however, many cases when it is not. Here are some advantages to using inline assembly:

- Easy access to machine-dependent registers, I/O and features.
- Control exact code behavior in critical sections preventing conflicts between software or hardware devices.
- Override standard conventions of your compiler, which might allow optimization (in memory allocation, calling conventions, etc.).
- Build interfaces between code fragments using non-standard conventions (i.e. produce a separate low-level interface).
- Ease access to non-typical programming modes of your processor.
- Produce reasonably fast code for tight loops, etc.
- Produce hand-optimized code perfectly tuned for your particular hardware setup.
- Gain complete control of your code.

However, assembly is a very low-level language (the lowest above hand-coding the binary instructions). This means,

- It is long and tedious to write initially.
- It is bug-prone, and bugs can be very difficult to find.
- Assembly code can be fairly difficult to understand, modify, and maintain.
- Assembly code is not portable to other architectures.
- It causes the programmer to spend more time on a few details.
- Small changes in design may completely invalidate existing assembly code.
- It is very difficult to use complex data structures in assembly.
- Many times an optimizing compiler outperforms hand-coded assembly.

Here are some general guidelines for including inline assembly code:

- Minimize the use and amount of assembly code.
- Encapsulate the code in well-defined functions or blocks.
- Incorporate higher-level language MACROs to automatically generate assembly code.
- Thoroughly debug your code.

Even when assembly is absolutely required, you'll probably find that you can get away with far less than you initially think need.

## Legal Information

# Chapter 1

# ATMEL Studio

## Advantages of Using ATMEL Studio

While installing ATMEL Studio 7 (AS7) is not required in order to learn inline assembly language programming, it has worthwhile advantages. The ability to compile code for the Arduino, run it inside the included Simulator and immediately debug it will greatly speed your learning process. This seamless and iterative process would take several minutes to complete if using the Arduino IDE. In fact, the Arduino IDE alone cannot perform the disassembly and debug functions.

AS7 now features seamless, one-click importation of projects created in the Arduino development environment. Your sketch, including any libraries it references, can be imported into AS7 as a C++ project. Once imported, you can leverage many additional capabilities of AS7 to fine-tune and debug your design. For most Arduino boards, shield-adapters that expose debug connectors are available, or one could use an ATMEL-ICE debug wire interface with a standard Arduino (slight modification of the Arduino is required).

The help system in AS7 allows on-line as well as off-line access. Device aware context sensitivity and an IO-view allow you to look up register-specific information from the datasheet of the part you are using without leaving the editor. The context sensitive AVR-Libc documentation allows easy access to function definitions too.

It gives you a seamless and easy-to-use environment to write, build and debug your applications written in C/C++ or assembly code. AS7 incorporates a start

page that contains relevant and up-to-date information on the devices you are using. An updated Visual Assist plug-in is included free in Atmel Studio, and provides several productivity enhancements to the editor.

If you are concerned about low-power performance, the Atmel Data Visualizer plug-in can capture and display run-time power data from your application when used with the Atmel Power debugger. You can profile the power usage of your application as part of a standard debug session. Sampling the program counter during power measurements makes it possible to correlate power spikes with the code that causes them.

AS7 is integrated with the Atmel Software Framework (ASF), which is a collection of production-ready source code, such as drivers, communication stacks, graphic services and touch functionality. It includes 1,600 project examples with source code to accelerate development of new applications. However, ASF has almost no support for the standard Arduino, and I would advise not to install ASF unless you have a specific need for it with other Arduinos or ATMEL MCUs.

Best of all, AS7 is free of charge. It is also important to note, that most of the functionality of AS7 is also available in the older Atmel Studio 6 IDE (AS6). AS6 has a similar installation process to what is outlined here for AS7.

## Installing ATMEL Studio 7

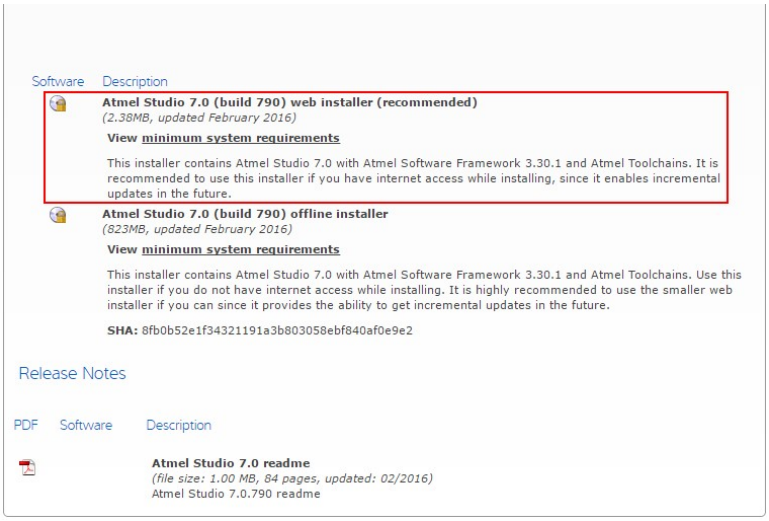
The first step is to acquire the program. On the internet, navigate to the ATMEL Studio 7 website download page, and click the DOWNLOAD NOW link:

```
http://www.atmel.com/Microsite/atmel-studio/
```

Direct link to the download page:

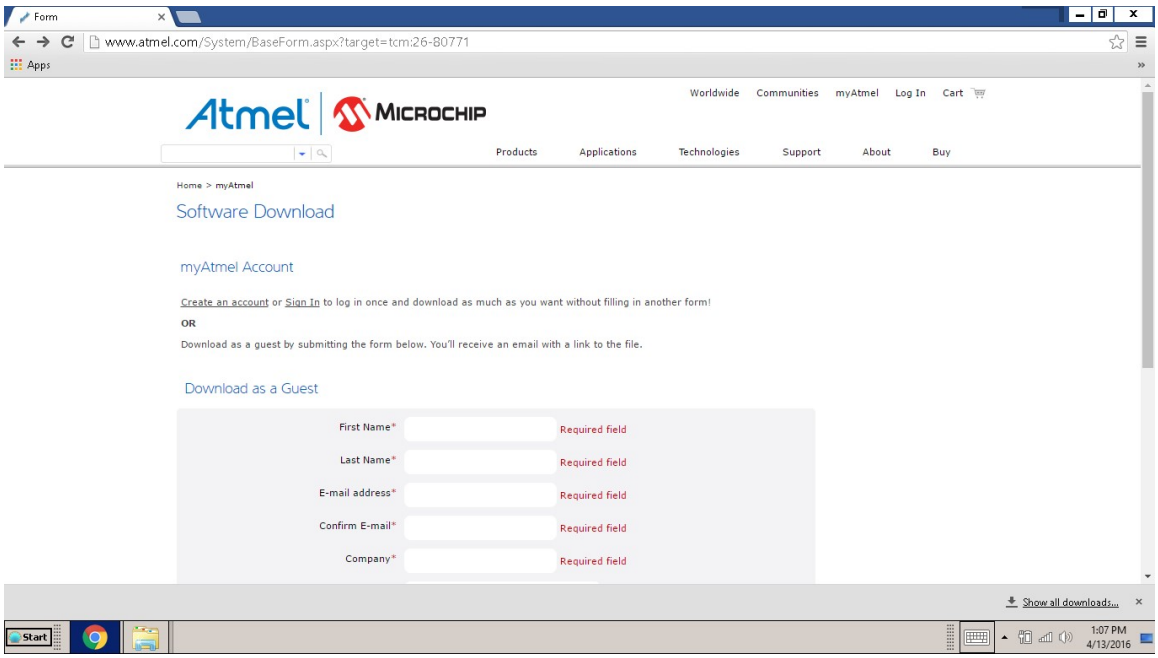
```
http://www.atmel.com/tools/atmelstudio.aspx#download
```

Select the web installer unless you have a specific requirement to install off-line. Take the note that the off-line installation requires a download of 823MB:

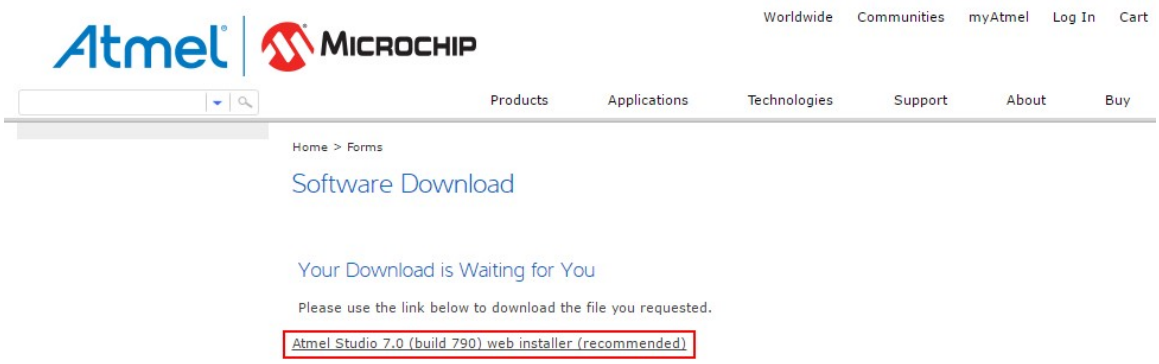



You will be prompted to answer a few questions prior to obtaining the actual download link. It is not required to initiate an “myATMEL” account to download AS7. However, there are a few advantages to having the account.

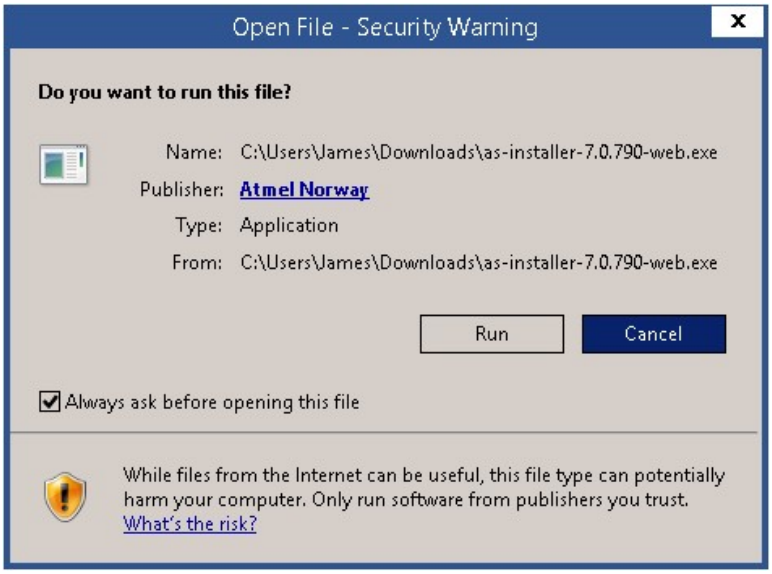
The data entry is required even if you choose to download as a “Guest”.




After supplying the necessary information, you will be directed to a link for downloading the installer. If you opened a “myATMEL” account, you may receive the download link via email. Please note these links will expire after a short period of time, so be sure to download and complete the installation process as without delay.




After downloading the installer, run the program. Acknowledge the security warning, and select RUN.




Next you will need to progress through a series of dialogs, making selections as the process continues.

1. The first item to negotiate will be a standard End User License Agreement, checking the appropriate box about your concurrence with the terms and conditions.

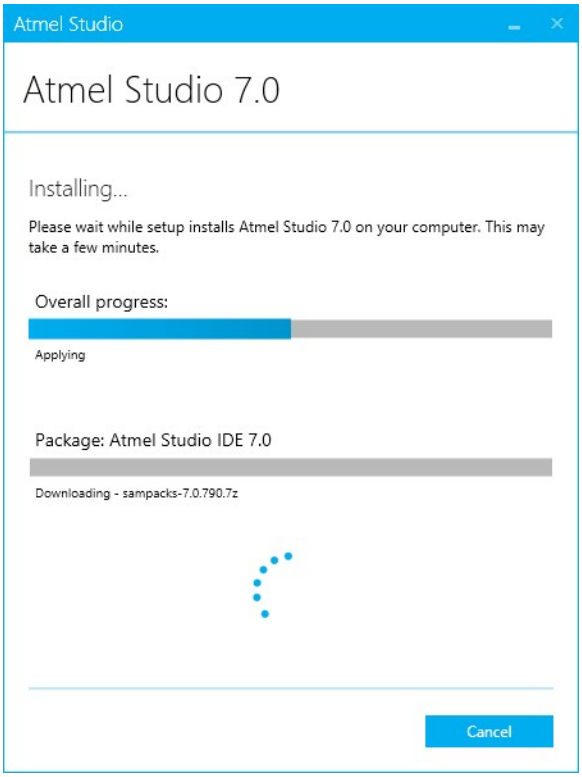
2. Next, select the AVR 8-bit MCU architecture unless you desire to use AS7 with other ATMEL MCUs (32-it MCUs primarily encompass ATMEL's Xmega series, and SMART ARM is applicable to the Arduino Zero and Due).

3. As previously discussed, you will next need to decide if you want the ASF.

4. The installer will perform a system validation check, in which you may need to perform a reboot, or shutdown an noncompatible program before proceeding.

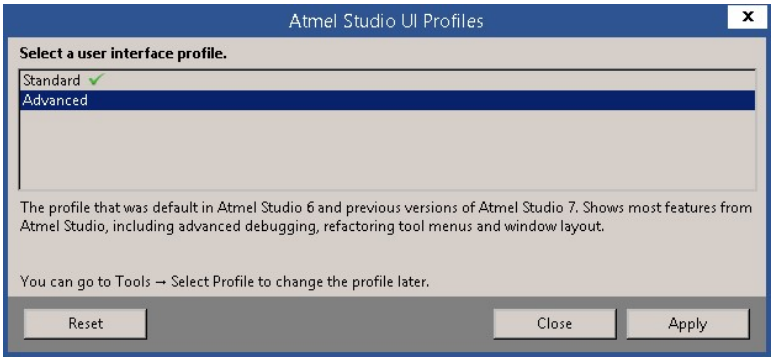
5. Finally, select NEXT in order to complete the installation. You obviously will need an internet connection, unless you are performing an off-line install.

At this point, sit back and relax, expect to wait several minutes for the installation process to complete.

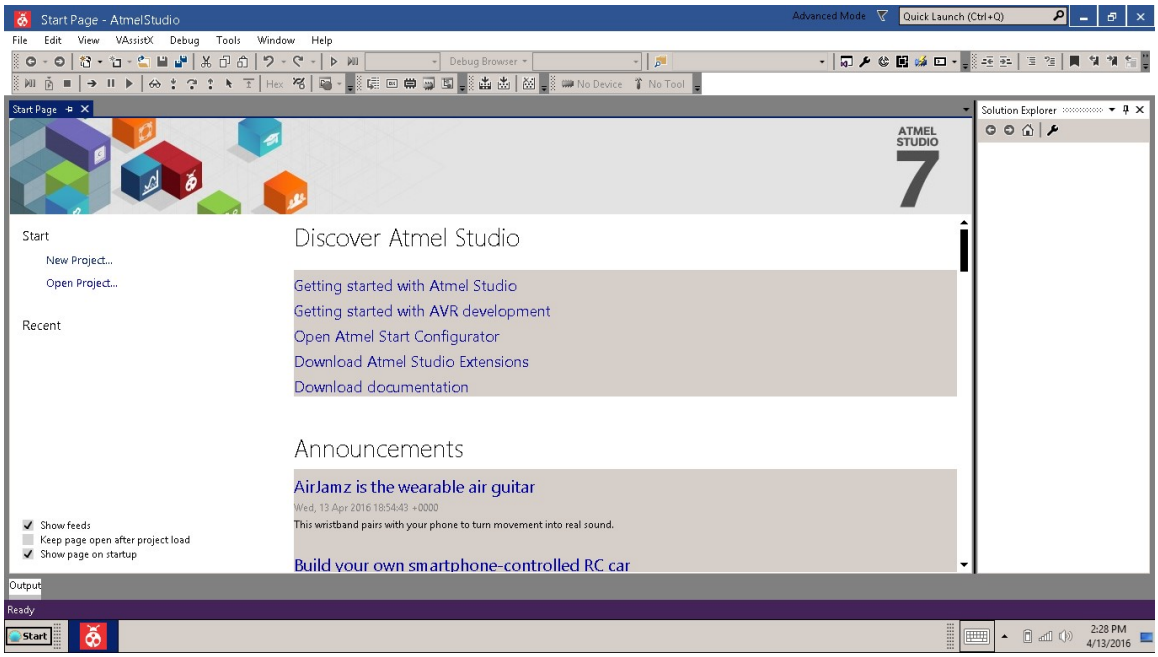



## ATMEL Studio 7 Runs Arduino Blink

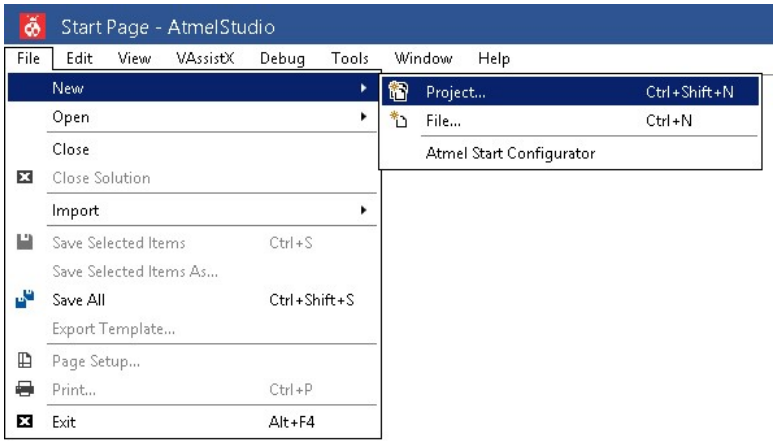
Congratulations, you have completed the installation. Let's run the Studio for the first time. On initial start, AS7 will ask you to select a user interface profile. I suggest the "Advanced" version, however either option is satisfactory. If desired, the profile can always be changed later.




Be patient, it will seem like a long period of time for the program to load, but eventually you will be greeted with the IDE and the startup page populated with an ATMEL announcement internet feed. It is possible to disable this if it is unwanted (checkbox on the bottom left).

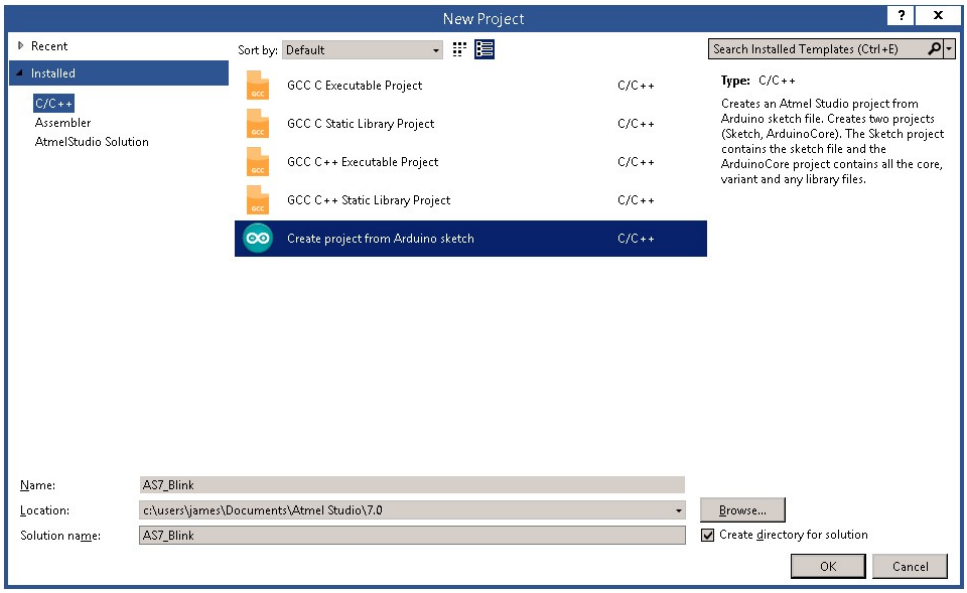



From the file menu select NEW->Project.




The New Project dialog will open and make the following selections:

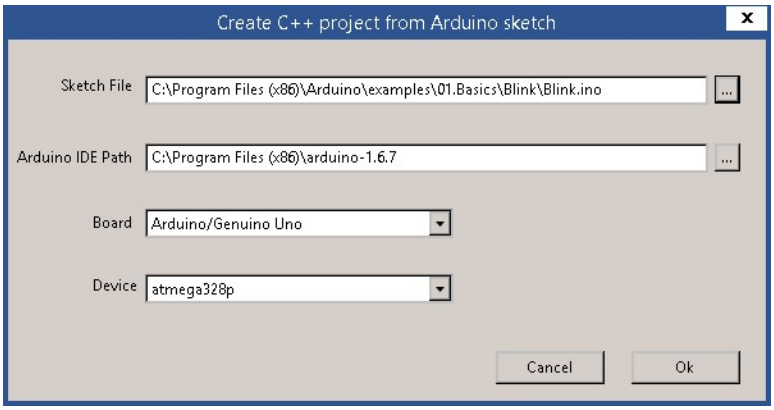
- Insure “Installed” and “C/C++” is highlighted on the far left.
- Highlight “Create project from Arduino sketch”.
- Give the project an appropriate name like, “AS7_Blink”, noting, spaces are not allowed in the project name.
- The default location should fill automatically.
- The solution name defaults to the project name, and that is acceptable.



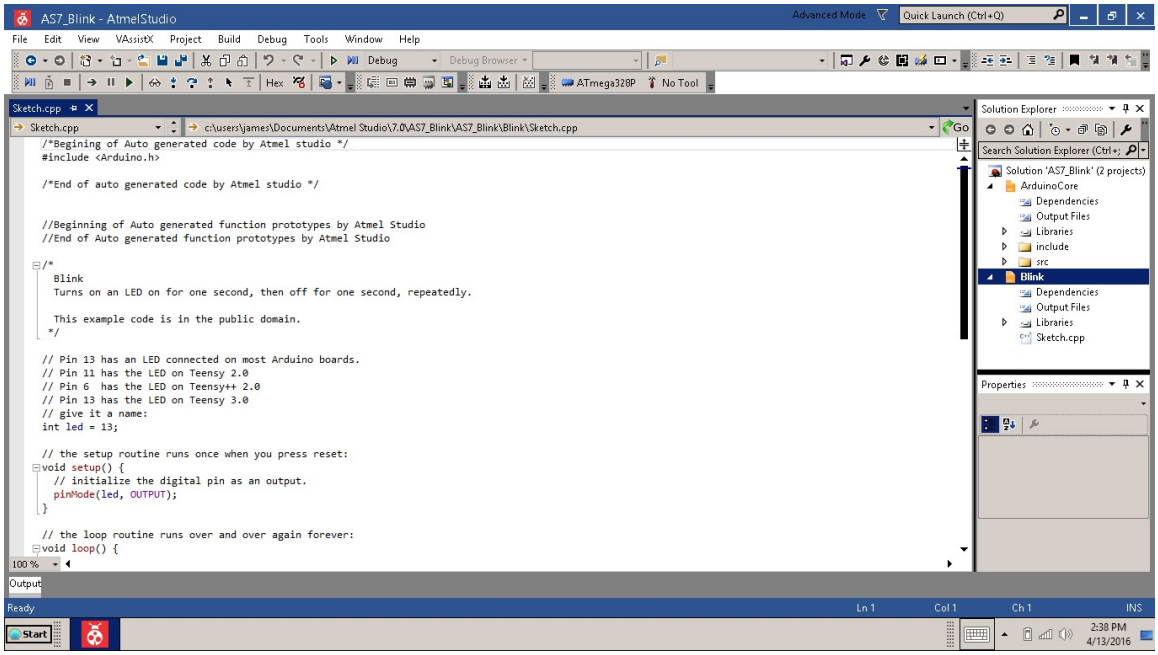

The next dialog will ask for the existing Arduino sketch location. Click on the button containing the ellipsis to the right of the Sketch File. Navigate to the Arduino Blink example sketch, which should be located inside your Arduino installation folder (similar to my location):

```
C:\Program Files (x86)\Arduino\examples\01.Basics\Blink\Blink.ino
```

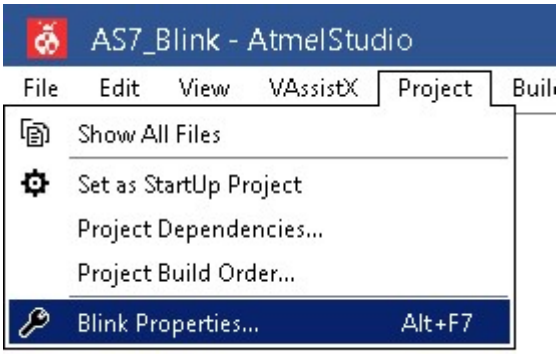
Likewise, insure your Arduino IDE path is properly filled in. Make the appropriate selections for your board (Arduino/Genuino Uno) and device (atmega328p), and click “Ok”. AS7 will take some time creating the folder(s) for the the project and pulling in the Blink sketch with all of its dependencies (all core and variant include files, Arduino core source files and libraries). It will finish by creating and opening an editable “sketch.cpp” file inside the IDE.



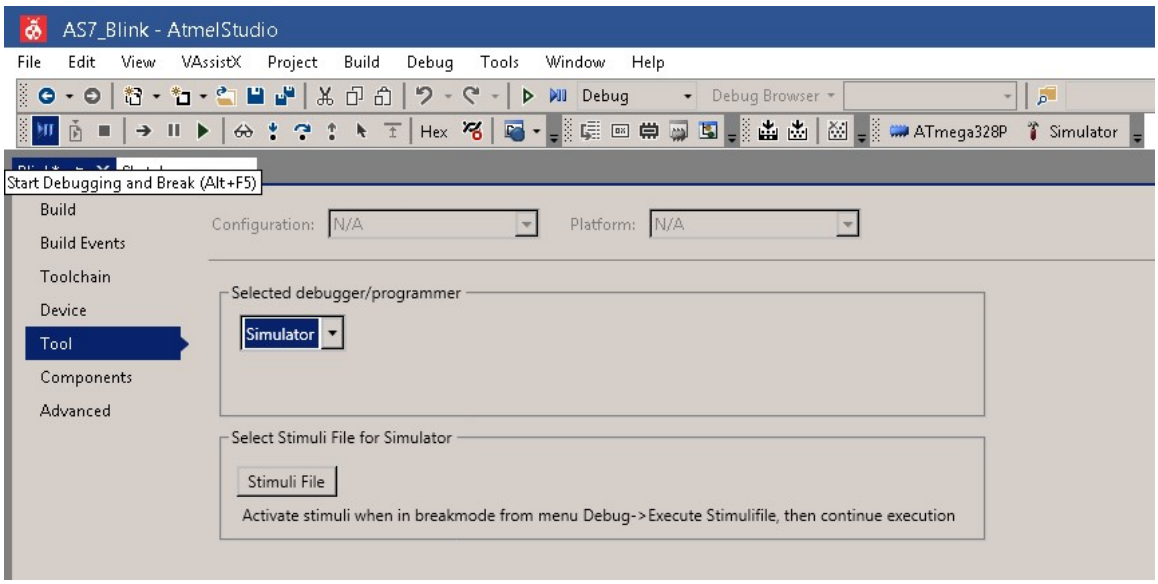

The sketch.cpp file is primarily the code from the example Arduino blink program including some additional automatically generated code by the ATMEL studio. Your solution should look like this:




We can now run the Blink program without an actual Arduino board using the simulator included with AS7. But first we need to inform AS7 to use the Simulator. Under the Project menu, select “Blink Properties…”.

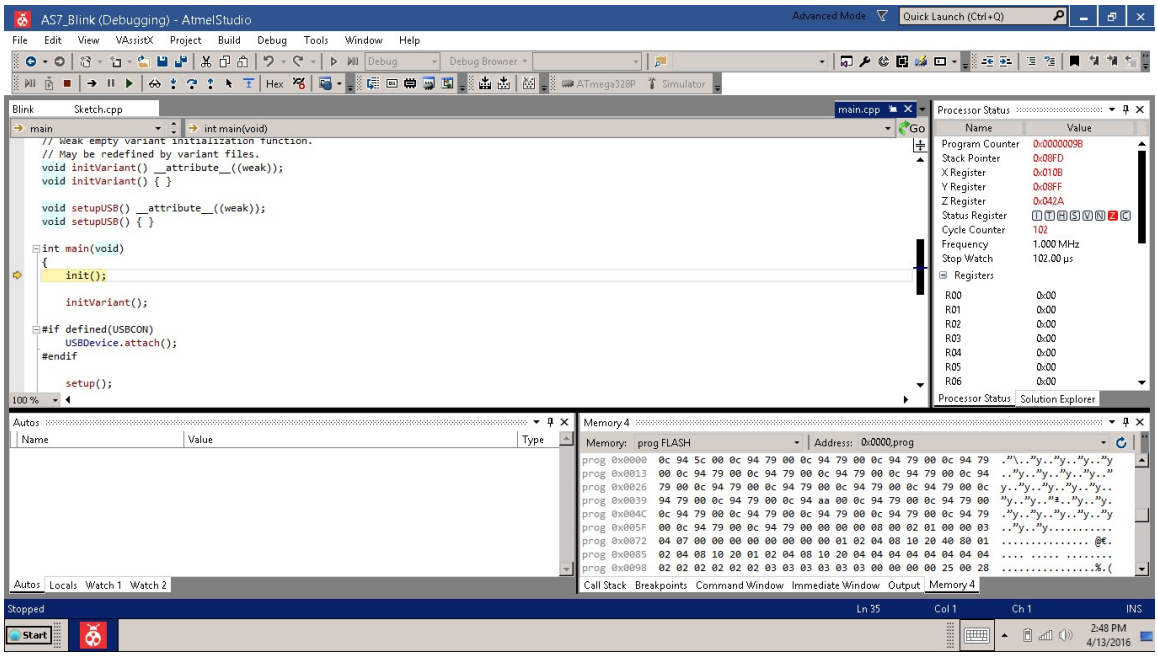



On the properties screen that appears, on the far left, select the “Tool” item, and under the heading of “Selected debugger/programmer”, select “Simulator” from the drop down list.




Now, to run the program in the simulator, simply press “Alt+F5” or Debug->Start Debugging and Break. AS7 will build the Blink project, execute the C-runtime startup code and then halt at the first line of the Arduino program. If you examine your screen, you’ll notice the debugger is pointing to (on the far left) and highlighting the line of code it has paused on:

```
init();
```

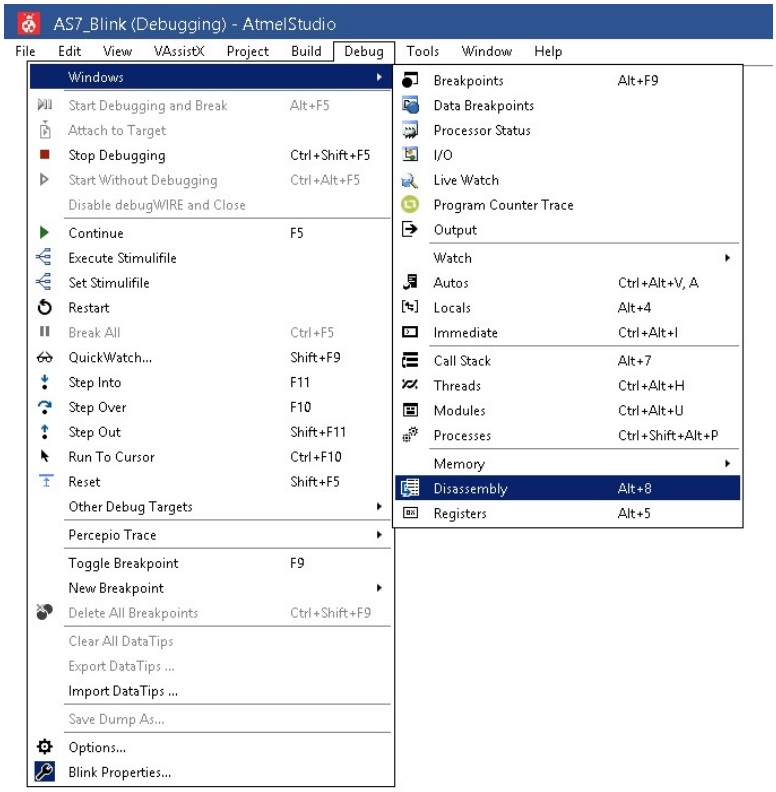



You might be thinking to yourself, this code wasn't in the Blink program. It's actually code that comes from the core Arduino wiring files which silently gets inserted into every Arduino sketch. If you look further down the screen you will eventually see the call to the Blink *Setup* function.

Hitting the "F10" key (Step Over) twice should bring the simulator to the function call to *Setup*. At this time, pressing the "F11" key (Step Into) will cause the simulator to jump into the Blink program and pause execution on the first line inside the *Setup* function:

```
pinMode(led, OUTPUT);
```

Now things should start to look familiar. The last feature I want to demonstrate is the ability to drill down into the underlying assembly language that was created by the compiler. We do this by selecting from the Debug menu, Debug>Windows>Disassembly.



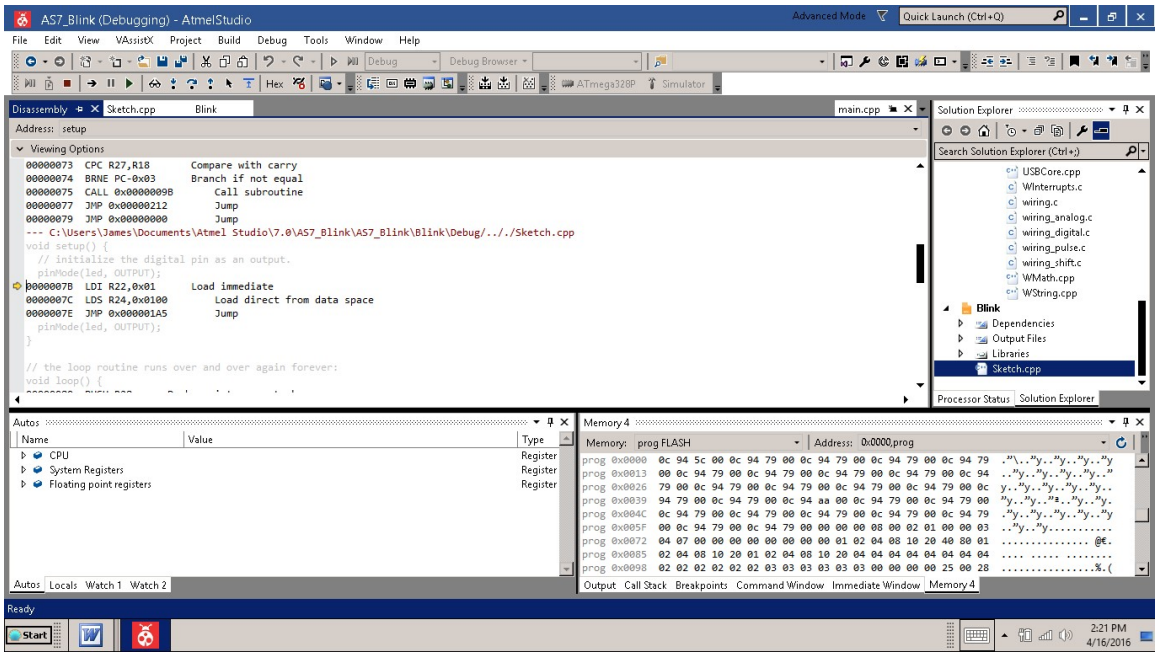

You should now see a mixture of C and assembly code which makes up the executable Blink program. The simulator should be paused on the follow line:

```
      0000007B  LDI R22,0x01          Load immediate
```

I can not overstate the importance of this feature. The ability to examine the assembly code, compare it to the source, and to step through it, line-by-line is an enormous asset to the inline assembly programmer. You can set breakpoints, watch variables, alter register values, and time sections of code. You will want to remember how this is done.

ATMEL has uploaded several informative videos on youtube.com demonstrating the use of the Studio, how to debug and how to use the simulator. A good example is located here:

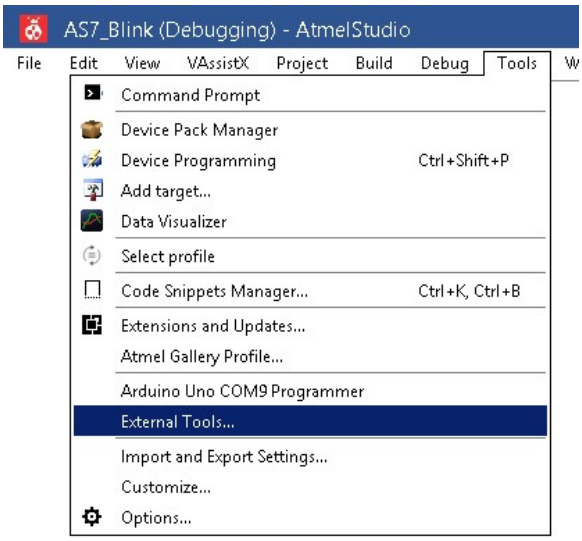
`https://www.youtube.com/watch?v=9QlDSNeuAdY`




## Download Blink Program to Arduino

AS7 terms the process of transfering a compiled program to the Arduino, as "Device Programming", and it expects the user to supply an interface such as an ATMEL-ICS, Dragon, or an STK500-compatible programmer. Fortunately for Arduino users, AS7 also has the ability to also use an "External Tool". In the case of an Arduino, the external tool will be the "avrdude" program that the Arduino IDE utilizes. It simply requires a small setup prior to use.

Access the setup through the menu item, Tools>External Tools.



Enter the following items in the External Tools dialog by first selecting “Add”:

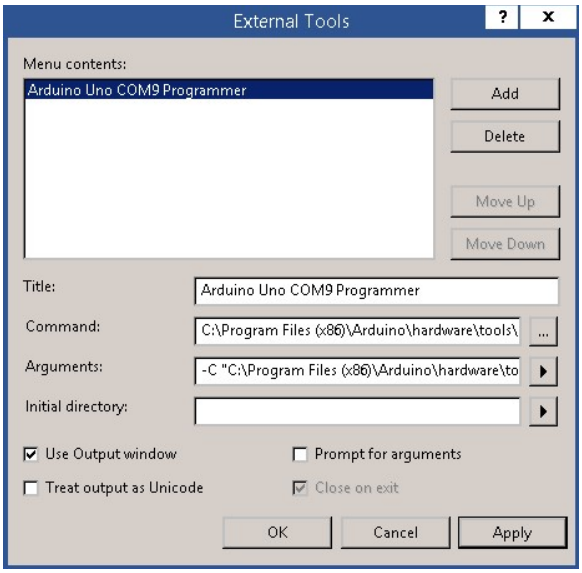
- Give your programming tool a descriptive title. The title will become a menu selection under the Tool menu item.
- Check the “Use Output window” option.
- Enter the location of the avrdude program (Arduino tools directory) under “Command”:

```
C:\Program Files (x86)\Arduino\hardware\tools\avr\bin\avrdude.exe
```

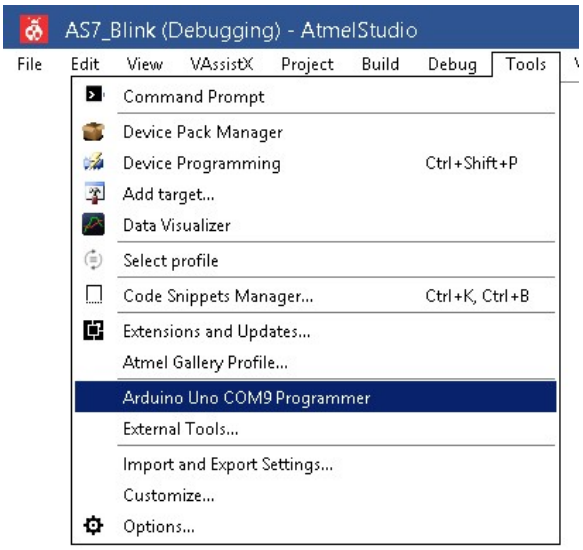
- Carefully enter the following under “Arguments”, substituting the COM Port for your Arduino:

```
-C "C:\Program Files (x86)\Arduino\hardware\tools\avr\etc\avrdude.conf"
–p atmega328p -c arduino -P COM9 -b 115200
-U flash:w:"$(ProjectDir)Debug\$(ItemFileName).hex":i
```

- Leave the “Initial Directory” empty.
- Click “OK” when complete.




You should now have an option under the Tools drop down menu that will allow you to download your program to an Arduino.




When downloading a program via avrdude, you should see a message similar to the following in the "Output Window", once the program has successfully transferred to the Arduino. If you receive a failure message, carefully check your com port and the other items entered above.

```
avrdude.exe: AVR device initialized and ready to accept instructions

Reading | ################################################## | 100% 0.02s

avrdude.exe: Device signature = 0x1e950f
avrdude.exe: NOTE: FLASH memory has been specified, an erase cycle will be
performed
             To disable this feature, specify the -D option.
avrdude.exe: erasing chip
avrdude.exe: reading input file "c:\users\james\Documents\Atmel
Studio\7.0\AS7_Blink\AS7_Blink\Blink\Debug\Blink.hex"
avrdude.exe: writing flash (1066 bytes):

Writing | ################################################## | 100% 0.42s

avrdude.exe: 1066 bytes of flash written
avrdude.exe: verifying flash memory against c:\users\james\Documents\Atmel
Studio\7.0\AS7_Blink\AS7_Blink\Blink\Debug\Blink.hex:
avrdude.exe: load data flash data from input file
c:\users\james\Documents\Atmel
Studio\7.0\AS7_Blink\AS7_Blink\Blink\Debug\Blink.hex:
avrdude.exe: input file c:\users\james\Documents\Atmel
Studio\7.0\AS7_Blink\AS7_Blink\Blink\Debug\Blink.hex contains 1066 bytes
avrdude.exe: reading on-chip flash data:

Reading | ################################################## | 100% 0.37s

avrdude.exe: verifying ...
avrdude.exe: 1066 bytes of flash verified
```

```
avrdude.exe: safemode: Fuses OK

avrdude.exe done.  Thank you.
```

## Exercise in Assembly Language

As an exercise to gain familiarity with AS7, make an assembly language project using the below Blink code.:

- Select “File>New>Project”.
- Assembler.
- AVR Assembler Project.
- Give it a unique name.
- Select the proper device.

Remember to select the Simulator in order to run the program inside the AS7 IDE. Here is the assembly code for the project:

```
.org 0x0000
   rjmp start
start:
    ldi r16, 0           ; reset system status
    out SREG, r16        ; init stack pointer
    ldi r16, low(RAMEND)
    out SPL, r16
    ldi r16, high(RAMEND)
    out SPH, r16

    sbi DDRB, DDB5       ;pinMode(13, OUTPUT);
_loop:
    sbi PORTB, PORTB5    ;turn LED on
    rcall _delay
    cbi PORTB, PORTB5    ;turn LED off
    rcall _delay
    rjmp _loop

_delay:
    ldi r24, 0x00        ;one second delay iteration
    ldi r23, 0xd4
    ldi r22, 0x30
_d1:                     ;delay ~1 second
    subi r24, 1
```

```
    sbci r23, 0
    sbci r22, 0
    brcc _d1
    ret
```

# Chapter 2

# Arduino's Number Formats

## Number Formats

We humans primarily use the base-10 number system, thought primarily due to the fact that we have 10 fingers. But the Arduino, as with all microcontrollers, at its lowest level utilizes the binary numbering system. Yet, to effectively program the microcontroller, in addition to the binary number system we need to understand the hexadecimal number format also.

## Binary Numbers

Microcontrollers (computers) represent data in sets of binary digits. The representation is composed of bits, which in turn are grouped into larger sets such as bytes.

Binary means two, and a bit is a binary digit capable of representing one of two states. The concept of a bit can easily be understood as a value of either 0 or 1, on or off, yes or no, true or false. Additionally, you can think of it being encoded by a switch or toggle of some kind. The lower section of the Arduino's memory (or standard registers, more on that later) are bit-addressable, meaning this portion of the memory can be operated on at the bit level.

While a single bit on its own is able to represent only two values (0 or 1), several bits together, may be used to represent larger values. For example, a number composed of 3 bits can represent up to 8 distinct values. As the number of bits increases, the number of possible 0 and 1 combinations increases exponentially.

The amount of possible combinations doubles with each binary digit added as illustrated in Table 1.

| Comparison of Numbering Systems | | |
|---|---|---|
| Base-10 | Binary | Hexadecimal |
| 0 | 0000 | 0 |
| 1 | 0001 | 1 |
| 2 | 0010 | 2 |
| 3 | 0011 | 3 |
| 4 | 0100 | 4 |
| 5 | 0101 | 5 |
| 6 | 0110 | 6 |
| 7 | 0111 | 7 |
| 8 | 1000 | 8 |
| 9 | 1001 | 9 |
| 10 | 1010 | A |
| 11 | 1011 | B |
| 12 | 1100 | C |
| 13 | 1101 | D |
| 14 | 1110 | E |
| 15 | 1111 | F |
| 16 | 10000 | 10 |

Table 1- Numbering Systems

Groupings with a specific number of bits are used to represent varying things and have specific names. For example, a “byte” is an 8-bit value, having enough bits to represent a character. The 8-bit byte is the fundamental unit used inside the Arduino microcontroller. The Arduino’s ATmega328 processor is one of a family of ATMEL “8-bit” microcontrollers. The byte is used to address memory, otherwise known as byte-addressable memory. Furthermore, the Arduino (as many computers do) reads data in multiples of 8 bits. For example, the basic data types for the Arduino are the byte (8-bits), integer (16-bits), and long (32-bits).

A nibble (sometimes referred to as a “nybble”), is a number composed of four bits. Being half of a byte, the nibble was named as a play on words. Because 4-bits allow for 16 values, a nibble is sometimes known as a hexadecimal digit.

## Hexadecimal Numbers

Programmers often need to write out binary quantities, but in practice writing out a binary number such as 1001001101010001 is tedious and prone to errors.

Therefore, binary quantities are typically written in the base-16, "hexadecimal" or "hex", number format.

In the decimal system, there are 10 digits, 0 through 9, which combine to form numbers. In a hex system, there are 16 digits, 0 through 9 followed by A through F. So the value of a hex "10" is the same as a decimal "16", a hex "20" is a decimal "32", and so on. In the hexadecimal system, there are 16 digits, an example and comparison of numbers in different bases is described in Table 1.

Hexadecimal is a convenient way to represent binary numbers. A byte being 8-bits has a maximum value of 256, or 2 raised to the 8th power. The value of 256, represented in hexadecimal becomes “FF”, which reduces the number to just two digits. When typing numbers, formatting characters are used to describe the number system, for example, the value 248 in base-10 is 0b11111000 in binary (using “0b” as the prefix), and in hexadecimal numbers it is 0xf8, or 0xF8 (using “0x” as the prefix).

## Converting between bases

Each of these number systems are positional systems, but while decimal weights are powers of 10, the binary weights are powers of 2, and hex weights are powers of 16.

To convert from binary to decimal, for each digit, multiply the value of the digit by the value of its position and then adds the results. For example:

```
binary 0b11111000
= (1 * 2^7) + (1 * 2^6) + (1 * 2^5) + (1 * 2^4) + (1 * 2^3)
= (1* 128) + (1 * 64) + (1 * 32) + (1 * 16) + (1 * 8)
= 128 + 64 + 32 + 16 + 8
= decimal 248
```

To convert from hex to decimal, for each digit, multiply the value of the digit by the value of its position and then adds the results. For example:

```
hex 0x3b2
= (3 * 16^2) + (11 * 16^1) + (2 * 16^0)
= (3* 256) + (11 * 16) + (2*1)
= 768 + 176 + 2
= decimal 946
```

To convert from decimal to binary use the “divide by 2” process. Iterate continually dividing the decimal number by 2, keeping track of the remainder. The first remainder will actually be the last digit in the sequence. Repeat the “divide by 2” process until reaching 0. For example:

```
decimal 156
156/2 = 78 remainder 0
78/2  = 39 remainder 0
39/2  = 19 remainder 1
19/2  =  9 remainder 1
9/2   =  4 remainder 1
4/2   =  2 remainder 0
2/2   =  1 remainder 0
1/2   =  0 remainder 1
= binary 0b10011100
```

To convert from decimal to hexadecimal use the “divide by 16” process similar to the above. Iterate continually dividing the decimal number by 16, keeping track of the remainder. The first remainder will actually be the last digit in the sequence. Repeat the “divide by 16” process until reaching 0. For example:

```
decimal 1128
1128/16 = 70 remainder 8
70/16   = 4  remainder 6
4/16    = 0  remainder 4
= hexadecimal 0x468
```

## Floating-point numbers

While both unsigned and signed integers are used in digital systems, even a 32-bit integer is not enough to handle all the range of numbers a calculator can handle, and that's not even including fractions. To approximate the greater range and precision of real number[1], we have to abandon signed integers and go to a "floating point" format.

In the decimal system, we are familiar with floating-point numbers in the form of scientific notation:

$$1.1030402 \times 105 = 1.1030402 \times 100000 = 110304.02$$

or, more compactly:

$$1.1030402E5$$

which means "1.1030402 times 1 followed by 5 zeroes". The certain numeric value of 1.1030402 is known as the "significand", multiplied by a power of 10 (E5 meaning 105 or 100,000), which is known as the "exponent". If we have a

[1] A real number is a value that represents a quantity along a continuous line, including all rational numbers, such as the integer −5 and the fraction 4/3, and all irrational numbers, such as square root of 2 (1.41421356…).

negative exponent, that means the number is multiplied by a 1 with that many places to the right of the decimal point. For example:

```
2.3434E-6 = 2.3434 × 10−6 = 2.3434 × 0.000001 = 0.0000023434
```

The advantage of a scheme using an exponent is we get a much wider range of numbers. Similar a binary floating-point format has been created for computers and adopted for the Arduino. This popular scheme is defined by the Institute of Electrical and Eltronic Engineers (IEEE) and called the IEEE 754-2008 standard. This specification defines a 32-bit floating-point number with a scheme using a 23-bit significand, a sign bit, plus an 8-bit exponent in "excess-127" format (4 total bytes in size). This allows for seven valid decimal digits.

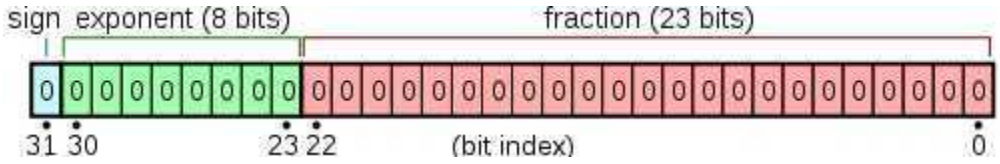



Figure 1 – 32-Bit Floating Point Bits

| | | | | | | | | |
|---|---|---|---|---|---|---|---|---|
| Byte 0: | S | x7 | x6 | x5 | x4 | X3 | x2 | x1 |
| Byte 1: | x0 | m22 | m21 | m20 | m19 | m18 | m17 | m16 |
| Byte 2: | M15 | m14 | m13 | m12 | m11 | m10 | m9 | m8 |
| Byte 3: | M7 | m6 | m5 | m4 | m3 | m2 | m1 | m0 |

Table 2 – 32-Bit Float Composition

The bits are converted to a numeric value with the computation:

<sign> × (1 + <fractional significand>) × 2<exponent> - 127

leading to the following range of numbers:

| **32-Bit Floating Point Number** | | |
|---|---|---|
| | **Maximum** | **Minimum** |
| Positive | 3.402823E+38 | 2.802597E-45 |
| Negative | -2.802597E-45 | -3.402823E+38 |

Table 3 – 32-Bit Float Limits

The specification also defines several special values that are not defined numbers, and are known as NANs, for "Not A Number". These are used to designate invalid operations and the like.

Only a finite range of real numbers can be represented with a given number of bits. Arithmetic operations can overflow or underflow, producing a value too large or too small to be represented. The representation has a limited precision, with only 7 decimal digits representing a 32-bit real number. If a very small floating-point number is added to a large one, the result is just the large one. The small number being too small to even show up in 7 or 8 digits of resolution, and the computer effectively discards it. Analyzing the effect of limited precision is a well-studied problem. Estimates of the magnitude of round-off errors and methods to limit their effect on large calculations are part of any large computation project. The precision limit is different from the range limit, as it affects the significand, not the exponent.

The significand is a binary fraction that doesn't necessarily perfectly match a decimal fraction. In many cases a sum of reciprocal powers of 2 does not matches a specific decimal fraction, and the results of computations will be slightly off. For example, the decimal fraction "0.1" is equivalent to an infinitely repeating binary fraction:

```
0.000110011 ...
```

## Standard Integer Types

In assembly programming, as with any language, it is very important to know the size of the values one is working with. Therefore, the *stdint.h* header file in the AVR-libc Standard C Library (introduced in the C99 Standard) implements a set of typedefs that specify exact-width integer types, together with the defined minimum and maximum allowable values for each type. This header is particularly useful for in embedding programming, and especially useful if you routinely program several different types of microcontrollers. Embedded programming in general and Arduino inline assembly programming specifically involves considerable manipulation of hardware specific registers which require integer data of fixed widths, specific locations and exact alignments. The naming convention for exact-width integer types is *int**N**_t* for a *signed int* and *uint**N**_t* for *unsigned int*, see Table 4.

| **Arduino Standard Data Types**[2] | | | | |
|---|---|---|---|---|
| Standard Type | Number of Bits | Arduino Type | Min Value | Max Value |
| uin8_t | 8 | byte | 0 | 256 |
| int8_t | 1 + 7 | signed char | -128 | 127 |
| char | 8 | char | 0 | 255 |
| uint16_t | 16 | unsigned int | 0 | 65535 |
| int16_t | 1 + 15 | int | -32768 | 32767 |
| uint32_t | 32 | long | 0 | 4294967295 |

[2] The avr-libc Standard C Library for AVR-GCC, which the Arduino uses defines two 64-bit types (uint64_t and int64_t), however we will not be utilizing this size.

| | | | | |
|---|---|---|---|---|
| int32_t | 1 + 31 | unsigned long | -2147483648 | 2147483647 |

Table 4 – Arduino Standard Integer Types

In addition, *stdint.h* along with defines inside the related include file, *limits.h,* provide MACRO values for the range limits of common integer variable types (the comments are mine):

```
//127
#define INT8_MAX  0x7f
//-128, 0x80
#define INT8_MIN  (-INT8_MAX - 1)
//255, 0xff
#define UINT8_MAX (INT8_MAX*2 + 1)
//32767
#define INT16_MAX 0x7fff
//-32768, 0x8000
#define INT16_MIN (-INT16_MAX - 1)
//65535, 0xffff
#define UINT16_MAX(__CONCAT(INT16_MAX, U)*2U + 1U)
//2147483647
#define INT32_MAX 0x7fffffffL
//-2147483648, 0x80000000
#define INT32_MIN (-INT32_MAX - 1L)
//4294967295, 0xffffffff
#define UINT32_MAX (__CONCAT(INT32_MAX, U)*2UL + 1UL)
```

# Chapter 3

# Basic asm

## In the Beginning

An inline assembly statement is a string which specifies assembler code. The string can contain any instructions recognized by the assembler, including directives (we will not discuss assembler directives here). GCC does not parse the assembler instructions and does not know what they mean or even whether they are valid. Multiple assembler instructions can be placed together in a single asm string.

## Simply Inline Assembly

Here is a most basic example of an inline assembler statement:

```
asm ( "nop \n" );
```

NOP is the opcode, or instruction for “No Operation”. This statement simply inserts a NOP into your program. NOP does nothing except take up program memory and waist processor time. NOP might seem to be a worthless instruction, however it does come in handy in several ways.

## Mainline Inline

The general form of an inline assembler statement is:

```
asm( "code \n" );
```

The assembly code is encased in parenthesis preceded by the compiler keyword *asm* or *__asm__*. Each assembler instruction is enclosed inside quotations and terminated with the escape sequence for the linefeed character, "\n". Sometimes you will see the line terminated with "\n\t". The tab sequence is added after the linefeed to simply format the output from the assembler.

The compiler takes each assembly string, strips off the quotation characters and passes the code verbatim onto the assembler. The avr-as assembler requires a single instruction per line, hence the need for the linefeed.

## The Fine Print

Do not expect a sequence of multiple asm statements to remain perfectly consecutive after compilation. If certain instructions need to remain consecutive in the output, put them in a single multi-instruction asm statement. GCC's optimizers can move asm statements relative to other code, including across jumps.

asm statements may not perform jumps into other asm statements. GCC does not know about these jumps, and therefore cannot take account of them when deciding how to optimize. Jumps from asm to C labels are only supported in extended asm.

Under certain circumstances, GCC may duplicate (or remove duplicates of) your assembly code when optimizing. This can lead to unexpected duplicate symbol errors during compilation if your assembly code defines symbols or labels. And since GCC does not parse the Assembler Instructions, it has no visibility of any symbols it references.

The compiler pre-processor does not perform macro expansion within strings. This is the reason why you can't use macro definitions, like so:

```
#define NoOperation NOP
__asm__("NoOperation \n");
```

## Instructions Sans Operands

You may be surprised to learn that the Basic asm statement uses only assembly instructions without operands. An operand[3] is the part of a computer instruction which specifies what data is to be manipulated or operated on. Basic assembly is limited to just pure instructions. That's it. To take advantage of more advanced features and processing, or any of the '%' operators, one must use the "extended asm" (more on the Extended Assembler later).

[3] See https://en.wikipedia.org/wiki/Operand.

## What's An Inline to Do?

So what can we do with the basic inline asm statement besides insert NOPs? There are 24 assembly instructions without operands, 16 of which deal with clearing and setting bits inside the Status Register (SREG[4]). We will cover the Status Register in greater detail later. Beyond the instructions dealing with the SREG, the other instructions are NOP, BREAK, SLEEP, WDR, ICALL/IJMP, RET/RETI.

ICALL, IJMP, RET and RETI are advanced instructions dealing with program flow, and will also be covered at a later time. WDR resets the Watchdog timer and is rarely used in the Arduino. SLEEP instructs the Arduino to go into a low power sleep mode.

However, two operand-less instructions are very useful. These two instructions enable or disable global interrupts. CLI, which is the mnemonic for "*CL*ear global *I*nterrupt flag", clears the Global Interrupt flag (I) in the Status Register (SREG). No interrupt will be executed after the CLI instruction, even if it occurs simultaneously with the CLI instruction.

```
asm ( "cli \n" );
```

SEI is the reverse of CLI, and sets the interrupt flag.

```
asm ( "sei \n" );
```

Both CLI and SEI are defined as C MACROS inside the AVR and arduino libraries, similarly like so:

```
#define CLI()          asm("cli \n")
#define noInterrupts() cli()

#define SEI()          asm("sei \n")
#define Interrupts()   sei()
```

So far we have discovered only minimal usefulness for the inline assembler. But we now know the fundamentals of the inline assembler syntax. In the next chapter we will expand upon these basics with the extended inline assembler.

[4] See chapter 8 on the SREG.

# Chapter 4

# Extended asm

## The Extended asm Statement

The general form of an extended inline assembler statement is:

```
asm(“code” : output operand list : input operand list : clobber list);
```

This statement is divided by colons into (up to) four parts. While the code part is required, the others are optional:

1. Code: the assembler instructions, defined as a single string constant.
2. A list of output operands, separated by commas.
3. A list of input operands, separated by commas.
4. A list of “clobbered” or “accessed” registers.

For now, we are going to ignore parts 2 through 4 and concentrate on the code part of the statement.

## Our First Inline Assembly Program

This is a basic example of storing a value in memory.

```
volatile byte a=0;

void setup() {
  Serial.begin(9600);
```

```
  asm (
    "ldi r26, 42  \n" //load register r26 with 42
    "sts (a), r26 \n" //store r26 in a's memory location
  );

  Serial.print("a = ");
  Serial.println(a);
}

void loop() { }
```

To fully understand what we are doing here first we need to cover some background.

## Memory

The Arduino has 3 basic types of memory, flash, SRAM and EEPROM. Flash is the memory where our program is stored. Arduino flash is non-volatile storage[5] where data can be retrieved even after power has been cycled (turned off and then back on). When you upload an Arduino program, it gets loaded into flash memory.

EEPROM[6] is a form of non-volatile storage used for variable data that we want to maintain between operations of our program.

SRAM is the memory used to store variable information and data. SRAM is volatile storage[7], and anything placed here is immediately lost when power is removed. In our program above, variable a is stored in SRAM. For now, we are not concerned about EEPROM or Flash memory, and will concentrate on SRAM.

## Registers Are Memory Too

Registers are special SRAM memory locations. The Arduino has 32 general purpose registers (labeled *r0* to *r31*). Each register is 8-bits, or 1-byte in size. *r0* occupies SRAM position 0, with each register following incrementally through SRAM position 31. These 32 general purpose registers are important because the Arduino cannot operate (change, do math, compare values, etc.) directly upon memory. Values stored in memory must first be loaded into a register(s).

[5] See https://en.wikipedia.org/wiki/Non-volatile_memory.
[6] See https://en.wikipedia.org/wiki/EEPROM.
[7] See https://en.wikipedia.org/wiki/Volatile_memory.

As you can imagine, the Arduino has more than 32 registers. Registers also have lots more specific details about them, but for now, these first 32 are more than enough.

## Assembly Instructions

Our inline assembly consisted of just two instructions, LDI and STS:

```
    "ldi r26, 42  \n"
    "sts (a), r26 \n"
```

The LDI instruction is a mnemonic for "*L*oa*D I*mmediate". This instruction simply loads an 8 bit constant value directly into a register. The register must be between 16 and 31. For example, trying to use register #1, *r1* with the LDI instruction would cause an error.

In our program, we load the value "42" into register #26, *r26* (pedantically we take note, *r26* is actually the 27th register, since numbering starts at zero). We could have chosen *r18*, *r19* or even *r24* for that matter. Later, our register selection will become crucial, but for now #26 seems like a good choice.
The LDI instruction is followed by an STS instruction. STS is a mnemonic for "*ST*ore direct to data *S*pace". STS stores one byte from a Register into SRAM, the data space. In our inline code, we place the contents of register #26, *r26* into the memory location of variable a. Quietly, behind the scenes, the assembler replaces "a" with the memory location of a. Neat.

Our program finishes by printing the contents of variable location a through the Serial Terminal. Hopefully this produces the output:

```
a = 42
```

## A Few Program Caveats

The variable must be global in scope. This causes the compiler to locate the variable inside SRAM. If we declared the variable a inside of the setup() function, it may have been stored temporarily on the stack or inside a register. If that was the case, the STS instruction would have caused an error because it only works when storing to SRAM. The stack[8] is a subject of a different chapter.

Furthermore, since the default Arduino compilation uses the *–Os* optimization level, we must declare it "volatile", so the optimizer doesn't eliminate it. The optimizer is very good at what it does, like when it sees code that's not necessary, it will artfully remove it. In our case here, we don't want this code to be removed.

[8] See https://ucexperiment.wordpress.com/2015/01/02/arduino-stack-painting/.

## More to Come

With these two beginning chapters we have covered the fundamentals of the inline assembler, basic syntax and a few assembler instructions. We will continue looking at the extended inline assembler, introducing new instructions as we go. Next, we'll look at the "clobber list" then dive into the bizarre world of input and output operands.

# Chapter 5

# Clobbers

## Getting Clobbered

Guess what? Our previous demonstration program in Chapter # has a problem. Here is the inline portion of that code:

```
asm (
  "ldi r26, 42  \n"
  "sts (a), r26 \n"
);
```

Notice in our example, we use register #26, *r26*. Even though we only used this register temporarily, we have trashed (or "clobbered") any value that was previous stored there. The compiler may have been using register *r26* somewhere else in this program, and we've inadvertently replaced any value that may have been inside *r26* with our value of 42. This may have introduced a bug into our program, or worse, it could have caused the program to crash.

Remember, the compiler simply passes our assembly code onto the avr-as assembler. It really has no idea what we are doing. Because of this we need a method to inform the compiler of the registers we use, hence the clobber list.
If you recall the general form of the extended inline assembler statement:

```
asm("code" : output operand list : input operand list : clobber list);
```

The fourth part is a list of "clobbered" or "accessed" registers. The format for this is to simply list the registers we clobber inside quotations. Like so:

```
r26
```

Our inline code should have looked like this:

```
  asm (
    "ldi r26, 42  \n"
    "sts (a), r26 \n"

    : : : "r26"
  );
```

Don't forget the clobber list is the fourth part of the asm statement, and we separate the parts with colons. If we clobbered additional registers, we would simply add them to the list, separating them with commas, like so:

```
"r16", "r17", "r25", "r26"
```

## The Chicken or the Egg

Let's introduce a minor addition to our previous chapter's inline program. Instead of dealing with an 8-bit byte value, lets use a 16-bit (2-byte) integer value. Obviously, an integer value requires two byte-sized memory locations to completely store itself. This introduces a conundrum, which byte comes first?

## Lows, Highs and Endians

Endianness[9] refers to the order of the bytes in computer memory. An integer may be represented in big-endian or little-endian format. The Arduino uses little-endian, which means the least significant byte (LSB) is stored in a lower memory address while the most significant byte is stored at a higher memory address. Before we get to our inline assembly program, here's a diversionary program for the Arduino which demonstrates endianness:

```
//program demonstrating Arduino endianness [little endian]
char text[32];

void setup() {
  uint16_t n16 = 0x1234;     //declare & initialize 16-bit number
  uint32_t n32 = 0x12345678; //declare & initialize 32-bit number

  Serial.begin(9600);

  //declare uint8_t pointer to 1st byte of 16-bit number
```

[9] See https://en.wikipedia.org/wiki/Endianness.

```
  uint8_t* pn16 = (uint8_t *)&n16;

  Serial.println(n16, HEX);

  for (uint8_t i=0; i<2; i++) {
    //iterate through both bytes of n16, noting order of digits
    sprintf(text, "%p: %02x \n", pn16, (uint8_t)*pn16++);
    Serial.print(text);
  }

  Serial.println();

  //declare uint8_t pointer to 1st byte of 32-bit number
  uint8_t* pn32 = (uint8_t *)&n32;

  Serial.println(n32, HEX);

  for (uint8_t i=0; i<4; i++) {
    //iterate through all 4-bytes of n32, noting order of digits
    sprintf(text, "%p: %02x \n", pn32, (uint8_t)*pn32++);
    Serial.print(text);
  }

  Serial.println();
}

void loop(void) { }
```

It is helpful to use a couple of assembly operators which easily determine the LSB and MSB of a 16-bit integer:

- *lo8()* - Takes the least significant 8 bits of a 16-bit integer.
- *hi8()* - Takes the most significant 8 bits of a 16-bit integer.

Lucky for us, when using these operators, we don't need to perform the math to determine that the LSB of 32,767 is 255 (0xff in hexadecimal), and the MSB is 127 (0x7f). The lo8 and hi8 operators do this for us. Armed with this new information let's store the value of 32,767 into our integer variable (a):

## 16-bit Integer Example

```
volatile int a = 0;

void setup() {
  Serial.begin(9600);
```

```
  asm (
    "ldi r24, lo8(32767) \n" //0xff
    "ldi r25, hi8(32767) \n" //0x7f
    "sts (a), r24        \n" //lsb
    "sts (a + 1), r25    \n" //msb

    : : : "r24", "r25"
  );

  Serial.print("a = ");
  Serial.println(a);
}

void loop(void) { }
```

First, notice how we address the 2-byte memory location representing (a), by using the notation of (a + 1) for the MSB, while just (a) equates to the LSB. It's vitally important to keep the correct order, or endianess, otherwise our number would have become 0xff7f in hexadecimal, which is 65,407 as an unsigned integer, or -127 as a signed integer. If all of this sounded foreign to you, you might want to study up on hexadecimal notation and signed[10] vs. unsigned integers[11]. Second, we didn't forget to include the "clobber" list this time.

## Final Answer

Our final example is just a simple adaptation of our very first inline program. Here we are again dealing with byte values, and we are just going to perform a simple variable swap. In C, we would code this something like:

```
byte a=10, b=20, c;

c = a;
a = b;
b = c;
```

In inline assembler:

```
volatile byte a = 10;
volatile byte b = 20;

void setup() {
  Serial.begin(9600);
```

[10] See https://en.wikipedia.org/wiki/Signed_number_representations.
[11] See https://en.wikipedia.org/wiki/Integer_(computer_science).

```
  asm (
    "lds r24, (a) \n"
    "lds r26, (b) \n"
    "sts (b), r24 \n" //exchange registers
    "sts (a), r26 \n"

    : : : "r24", "r26"
  );

  Serial.print("a = ");
  Serial.println(a);
  Serial.print("b = ");
  Serial.println(b);
}

void loop(void) { }
```

Notice, instead of loading an immediate value with the LDI instruction, we use LDS. LDS is the mnemonic for "*L*oad *D*irect from data *S*pace". LDS loads one byte from the data space (SRAM) into a register.

In our program, in the process of loading and then storing, we simply exchange the registers (*r24* for *r26*) in order to perform the swap. Notice we don't need to burden ourselves with the actual addresses of the variables a, and b. In both the LDS and STS instructions, the assembler inserts the SRAM memory addressing for us. Furthermore, we correctly identify the two registers used, as "Clobbered".

## Spoiler Alert

In our next chapter, we will reduce the previous byte swap program into the following rather odd-looking inline assembler code. You might even be tempted to exclaim, "What code?" Believe it or not, this works. Get ready to travel the winding path of input and output operands!

```
  asm (
    "" : "=r" (a), "=r" (b) : "0" (b), "1" (a)
  );
```

# Chapter 6

# Constraints

## Introduction

I have a confession to make. My previous examples were not very efficient assembly code. That might seem like an odd comment, especially since my typical example used just 2-4 lines of code. But, these examples were coded as one would write pure assembly, which is not necessarily the way inline should be written. The sneaky assembler silently inserts some extra code into our programs.

Writing code for the inline assembly requires a paradigm shift. Starting with this chapter, we'll begin to cover the odd method of coding input and output operands for the asm statement. Using them will enable us to produce more efficient inline code.

## Extending Inline

Recall the general form of an extended inline assembler statement is:

```
asm("code" : output operand list : input operand list : clobber list);
```

This statement is divided by colons into (up to) four parts. While the code part is required, the others are optional:

1. Code: the assembler instructions, defined as a single string constant.
2. A list of output operands, separated by commas.
3. A list of input operands, separated by commas.
4. A list of "clobbered" or "accessed" registers.

We previous covered the code portion and the clobber list. We will continue to introduce new assembler instructions with each installment. But now, let's discuss the input and output operands.

## Constraints

Each input or output operand is described by a constraint[12] string followed by a C expression in parentheses. Constraints are primarily letters but can be numbers too. The selection of the proper constraint depends on the range of registers or constants acceptable to the AVR instruction they are used with. Here's an example:

```
      "=r" (status) : "I" (_SFR_IO_ADDR(PINB)), "I" (PINB5)
```

Remember, the C compiler doesn't check your assembly code. But the assembler does check the constraint against your C expression. If you specify the wrong constraints, the compiler may silently pass wrong code to the assembler, causing it to fail. And if this happens, it abruptly terminates giving a very cryptic error message.

For example, if you specify the constraint "r" and you are using this register with an ORI instruction in your code, the assembler may select any register. This would fail if the assembler selects *r2* to *r15*. That's why the correct constraint in that case is "d". But, we're getting ahead of our selves.

## Constraining Constraints

The assembler is free to select any register for storage of the value that meets the constraints of your constraint. Interestingly, the assembler might not explicitly load or store your value, and it may even decide not to include your assembler code at all! All these decisions are part of the optimization strategy. For example, if you never use the variable in the remaining part of the C program, the compiler will most likely remove your code unless you switched off optimization.

## Modified Constraints

A modifier sometimes precedes the constraint. Let's demonstrate with an inline assembler routine using a C char variable (a), and an 8-bit constant value (ANSWER_TO_LIFE). Our inline program will simply save the constant to our variable.

Here is our output constraint string:

[12] See Appendix B, Modifiers and Constraints.

```
"=r" (a)
```

The equal sign is the modifier; '=' means that this operand is written to by this instruction, and the previous value is discarded and replaced by new data. The 'r' is the constraint, and instructs the assembler to place our value into "any general register".

The input constraint string is even simpler:

```
"M" (ANSWER_TO_LIFE)
```

The 'M' defines an 8-bit integer constant in the range of 0-255. Inside the parentheses we place our C MACRO defined value, "ANSWER_TO_LIFE". Remember, we use the colon character, ':' to separate code, outputs, inputs and clobbers.

## Percentage Zero

Take notice of the '%0' and '%1' characters in our inline code below. These represent the substitution locations for the operand values from the constraint strings. The constrained values are substituted into the code in the order they appear at the bottom of the inline routine, 0 first, then 1, 2, etc. The first value (a) as constrained, is substituted where '%0' appears, and the number '42' is substituted for '%1'. If there were additional constraints, the next one in line would be substituted for '%2', and sequentially onward.

Here is our full code:

```
#define ANSWER_TO_LIFE 42

volatile uint8_t a;

void setup() {
  Serial.begin(9600);

  asm (
    "ldi %0, %1 \n"

    : "=r" (a) : "M" (ANSWER_TO_LIFE)
  );

  Serial.print("a = ");
  Serial.println(a);
}

void loop() { }
```

This is where it gets a little confusing. The assembler doesn't simply replace the '%0' with the value in parenthesis. For our example, that would result in an assembler instruction something like:

```
ldi a, ANSWER_TO_LIFE
```

That's invalid syntax for the LDI instruction. Instead, it replaces '%0' with the value as described in the constraint. That's an important difference. As such, it creates a valid assembler instruction like this:

```
ldi r24, 42
```

And that works.

Furthermore, notice we haven't included any code to store the output. Rather, we instructed the assembler to do this for us through the equal sign '=', in our "modified constraint". This may seem like an odd way of writing assembler code, but this is how we do it. It seems natural to want to add a line something like the following after the LDI command:

```
"sts %0, %1 \n"
```

You could. But it's just not necessary.

## Getting To Specifics

Output operands must be write-only and the C expression result must be an lvalue[13], which means that the operands must be valid on the left side of assignments. Note, that the compiler does not check if the operands are a reasonable type for the kind of operation used in the assembler instructions. Input operands are read-only. Read-write operands are not supported in inline assembler code. But there is a solution to this and we cover it below under the heading of "Straight Ahead".

When the compiler selects the registers to use which represent the input or output operands, it does not use any of the registers listed in the "clobbered" section. As a result, clobbered registers are available for any use in the assembler code.

Be forewarned, that accessing data from C programs without using input/output operands (such as by using global symbols directly from the assembler template) may not work as expected. Since the assembler does not parse the inline code, it has no visibility of any symbols it references. This may result in those symbols

[13] See http://ieng9.ucsd.edu/~cs30x/Lvalues%20and%20Rvalues.htm.

being discarded as unreferenced unless they are also listed as input, output operands. The moral of this story is, “USE INPUT AND OUTPUT OPERANDS.”

## A and B Percentage

Here is another example of performing a simple swap between two 16-bit integers:

```
volatile int a = 0xa1a2;
volatile int b = 0xb1b2;

void setup() {
  Serial.begin(9600);

  asm (
    "mov %A0, %A3 \n"
    "mov %B0, %B3 \n"
    "mov %A1, %A2 \n"
    "mov %B1, %B2 \n"

    : "=r" (a), "=r" (b) : "r" (a), "r" (b)
  );

  Serial.print("a = ");
  Serial.println(a, HEX);
  Serial.print("b = ");
  Serial.println(b, HEX);
}

void loop() { }
```

First, notice the letters A and B, as in %A0 and %B0. They refer to two different 8-bit registers, each containing a part of the 2-byte value of %0. Recalling, the Arduino is a little-endian microcontroller, meaning the LSB is stored in the lower memory address and the MSB is stored in the higher address. Therefore, ‘A’ refers to the MSB, while ‘B’ addresses the LSB. If we were dealing with a 4-byte, 32-bit value, we would use the letters A through D.

Second, we address the two variables (a) and (b) separately as both input and output operands. When the compiler fixes up the operands to satisfy the constraints, it needs to know which operands are read by the instructions and which are written by it. Again, ‘=’ identifies an operand which is only written (‘+’ identifies an operand that is both read and written, and all other operands are assumed to be read only).

## Nix Name Your Operands

Although I sometimes find this an additional layer of confusion placed on top of a topic already layered with confusion, operands can be given names. The name is pre-pended in brackets to the constraints in the operand list, and references to the named operand use the bracketed name instead of a number after the % sign. Thus, the above example on the "meaning of life" becomes something like this:

```
asm (
  "ldi %[varA], %[Answer] \n"

  : [varA] "=r" (a) : [Answer] "M" (ANSWER_TO_LIFE)
);
```

I will leave you to yourself to determine if this makes the inline code easier to understand. Take note, that throughout this book we never use this feature.

## Move Over

Last, we introduce the MOV instruction. Did you guess that MOV is the mnemonic for *MOV*e? The MOV instruction makes a copy of one register into another. The source register is left unchanged.

Let's examine the code the assembler produces for this example. We should take note that this code is not very efficient:

```
LDS R24, 0x0102 //a
LDS R25, 0x0103
LDS R18, 0x0100 //b
LDS R19, 0x0101
MOV R18, R18
MOV R19, R19
MOV R24, R24
MOV R25, R25
STS 0x0103, R19
STS 0x0102, R18
STS 0x0101, R25
STS 0x0100, R24
```

## Straight Ahead

Let's make it efficient. Using bytes instead of integers, here is the straight-forward inline method for performing a swap. Again, notice we define both input and output operands. But, for the input operators it is possible to use a single digit in the constraint string. Using a digit "n" tells the compiler to use the same register as the 'n-th' output operand (they start at zero).

Next, hopefully you noticed, in a sneaky fashion we switched the order of the inputs and outputs. Finally, you probably noticed that we don't write any code at all. Because our constraints do it for us!

```
uint8_t a = 10;
uint8_t b = 20;

void setup() {
  Serial.begin(9600);

  asm (
    "" : "=r" (a), "=r" (b) : "0" (b), "1" (a)
  );

  Serial.print("a = ");
  Serial.println(a);
  Serial.print("b = ");
  Serial.println(b);
}

void loop(void) { }
```

The same method works with integers:

```
volatile int a = 0xa1a2;
volatile int b = 0xb1b2;

void setup() {
  Serial.begin(9600);

  asm volatile(
    "" : "=r" (a), "=r" (b) : "0" (b), "1" (a)
  );

  Serial.print("a = ");
  Serial.println(a, HEX);
  Serial.print("b = ");
  Serial.println(b, HEX);
}

void loop() { }
```

## One More Clobber

There is a special type of clobber called "memory" which informs the compiler that the inline assembly code performs memory reads or writes to items other

than those listed in the input and output operands (for example, accessing the memory pointed to by one of the input parameters). This is a “clobber” that is easily missed, and I admit to omitting it often.

To ensure memory contains correct values, the compiler may need to flush specific registers pointing to memory before executing the inline code. Further, the compiler does not assume that any values read from memory before the inline code remain unchanged after that code (it reloads them as needed). Using the “memory” clobber effectively forms a read/write memory barrier for the compiler.

## Wrap Up

We covered a lot of material about constraints in this post, and we’ve only just begun. The proper use of constraints is critical to writing correct and efficient inline assembly code. It took me hours of studying the constraint list to become proficient with them, and at times, I still get frustrated. But as we continue in this series, it will get easier and clearer. With practice comes proficiency.

## AVR family Specific Constraints

| Constraint | Use | Range |
|---|---|---|
| a | Simple upper registers | r16 to r23 |
| b | Base pointer registers pairs | y, z |
| d | Upper register | r16 to r31 |
| e | Pointer register pairs | x, y, z |
| l | Lower registers | r0 to r15 |
| n | Immediate integer operand with known numeric value | |
| q | Stack pointer register | SPH:SPL |
| r | Any register | r0 to r31 |
| s | Immediate integer whose value is not an explicit integer | |
| t | Temporary register | r0 |
| w | Special upper register pairs | r24 to r30 |
| x | Pointer register pair X | x (r27:r26) |
| y | Pointer register pair Y | y (r29:r28) |
| z | Pointer register pair Z | Z (r31:r30) |
| F | Immediate floating point number | |
| G | Floating point constant | 0.0 |
| I | 6-bit positive integer constant | 0 to 63 |
| J | 6-bit negative integer constant | -63 to 0 |
| K | Integer constant | 2 |
| L | Integer constant | 0 |
| M | 8-bit integer constant | 0 to 255 |
| N | Integer constant | -1 |
| O | Integer constant | 8, 16, 24 |
| P | Integer constant | 1 |
| Q | Memory address based on Y or Z pointer w/displacement | |
| R | Integer constant | -6 to 5 |
| X | Any operand whatsoever | |

The x register is r27:r26, the y register is r29:r28, and the z register is r31:r30

## Modifier Characters

‘=’
Means that this operand is written to by this instruction: the previous value is discarded and replaced by new data.

‘+’
Means that this operand is both read and written by the instruction.
When the compiler fixes up the operands to satisfy the constraints, it needs to know which operands are read by the instruction and which are written by it. ‘=’ identifies an operand which is only written; ‘+’ identifies an operand that is both read and written; all other operands are assumed to only be read.

If you specify ‘=’ or ‘+’ in a constraint, you put it in the first character of the constraint string.

‘&’
Means (in a particular alternative) that this operand is an earlyclobber operand, which is written before the instruction is finished using the input operands. Therefore, this operand may not lie in a register that is read by the instruction or as part of any memory address.

‘&’ applies only to the alternative in which it is written. In constraints with multiple alternatives, sometimes one alternative requires ‘&’ while others do not. A operand which is read by the instruction can be tied to an earlyclobber operand if its only use as an input occurs before the early result is written. Adding alternatives of this form often allows GCC to produce better code when only some of the read operands can be affected by the earlyclobber.

Furthermore, if the earlyclobber operand is also a read/write operand, then that operand is written only after it’s used.

‘&’ does not obviate the need to write ‘=’ or ‘+’. As earlyclobber operands are always written, a read-only earlyclobber operand is ill-formed and will be rejected by the compiler.

‘%’
Declares the instruction to be commutative for this operand and the following operand. This means that the compiler may interchange the two operands if that is the cheapest way to make all operands fit the constraints. ‘%’ applies to all alternatives and must appear as the first character in the constraint. Only read-only operands can use ‘%’.

GCC can only handle one commutative pair in an asm; if you use more, the compiler may fail. Note that you need not use the modifier if the two alternatives are strictly identical; this would only waste time in the reload pass.

# Chapter 7

# Ports and Pins

## Mapping Memory

The 3 basic types of memory in the Arduino were discussed in chapter 2. Again, we are interested in SRAM data memory, which the Arduino uses as memory-mapped IO[14]. MMIO means that a part of the SRAM space is reserved for registers that control peripherals (i.e., ports, timers, SPI, USART, etc.). Changing the content of the memory in this area changes the value controlling a peripheral. We have seen that the bottom of the SRAM address space is reserved for direct access to the general purpose registers (r0-r31). The IO register space is mapped into the SRAM just above these 32 GPRs and is divided into two sections. The first section is for Standard IO Registers (primarily the ports, but includes various other peripherals as well). The second section is for the Extended IO Registers, which includes a mishmash of peripherals. The actual locations can be found towards the rear of the data sheet for the Arduino's ATmega microcontroller. Actual SRAM is available behind the IO register area, starting at address0x100 (otherwise known as RAMSTART).

The lower IO register portion of the SRAM is 32 bytes long (addresses 0 through 31 (0x1fhexadecimal) and is further distinguished by the fact that it is bit-accessible using the SBI, CBI, SBIS and SBIC instructions. Above this area, IO registers are addressable only by byte value. The IN and OUT instructions can only access the Standard IO registers below 64 (0x3f and below). The 160 bytes of Extended IO registers are located at 96-255 (0x60 – 0xff). To further confuse matters, when using the LD and ST instructions, 0x20 must be added to the IO

[14] See https://en.wikipedia.org/wiki/Memory-mapped_I/O.

Register number. Not all of these registers are utilized. Hopefully, we can clear up some of this confusion.

## Standard IO Registers (0-64):

| Addr | Name | Bit7 | Bit6 | Bit5 | Bit4 | Bit3 | Bit2 | Bit1 | Bit0 |
|---|---|---|---|---|---|---|---|---|---|
| ... | | | | | | | | | |
| 0x03 | PINB | PINB7 | PINB6 | PINB5 | PINB4 | PINB3 | PINB2 | PINB1 | PINB0 |
| 0x04 | DDRB | DDB7 | DDB6 | DDB5 | DDB4 | DDB3 | DDB2 | DDB1 | DDB0 |
| 0x05 | PORTB | PORTB7 | PORTB6 | PORTB5 | PORTB4 | PORTB3 | PORTB2 | PORTB1 | PORTB0 |
| 0x06 | PINC | x | PINC6 | PINC5 | PINC4 | PINC3 | PINC2 | PINC1 | PINC0 |
| 0x07 | DDRC | x | DDC6 | DDC5 | DDC4 | DDC3 | DDC2 | DDC1 | DDC0 |
| 0x08 | PORTC | x | PORTC6 | PORTC5 | PORTC4 | PORTC3 | PORTC2 | PORTC1 | PORTC0 |
| 0x09 | PIND | PIND7 | PIND6 | PIND5 | PIND4 | PIND3 | PIND2 | PIND1 | PIND0 |
| 0x0A | DDRD | DDD7 | DDD6 | DDD5 | DDD4 | DDD3 | DDD2 | DDD1 | DDD0 |
| 0x0B | PORTD | PORTD7 | PORTD6 | PORTD5 | PORTD4 | PORTD3 | PORTD2 | PORTD1 | PORTD0 |
| ... | | | | | | | | | |
| 0x15 | TIFR0 | x | x | x | x | x | OCF0B | OCF0A | TOV0 |
| 0x16 | TIFR1 | x | x | ICF1 | x | x | OCF1B | OCF1A | TOV1 |
| 0x17 | TIFR2 | x | x | x | x | x | OCF2B | OCF2A | TOV2 |
| ... | | | | | | | | | |
| 0x1B | PCIFR | x | x | x | x | x | PCIF2 | PCIF1 | PCIF0 |
| 0x1C | EIFR | x | x | x | x | x | x | INTF1 | INTF0 |
| 0x1D | EIMSK | x | x | x | x | x | x | INT1 | INT0 |
| 0x1E | GPIOR0 | General Purpose IO Register 0 | | | | | | | |
| 0x1F | EECR | x | x | EEPM1 | EEPM0 | EERIE | EEMPE | EEPE | EERE |
| 0x20 | EEDR | EEPROM Data Register | | | | | | | |
| 0x21 | EEARL | EEPROM Address Register Low Byte | | | | | | | |
| 0x22 | EEARH | EEPROM Address Register High Byte | | | | | | | |
| 0x23 | GTCCR | TSM | x | x | x | x | x | PSRASY | PSRSYNC |
| 0x24 | TCCR0A | COM0A1 | COM0A0 | COM0B1 | COM0B0 | x | x | WGM01 | WGM00 |
| 0x25 | TCCR0B | FOC0A | FOC0B | x | x | WGM02 | CS02 | CS01 | CS00 |
| 0x26 | TCNT0 | Timer/Counter 0 (8-bit) | | | | | | | |
| 0x27 | OCR0A | Timer/Counter 0 Output Compare Register A | | | | | | | |
| 0x28 | OCR0B | Timer/Counter 0 Output Compare Register B | | | | | | | |
| ... | | | | | | | | | |
| 0x2A | GPIOR1 | General Purpose IO Register 1 | | | | | | | |
| 0x2B | GPIOR2 | General Purpose IO Register 2 | | | | | | | |
| 0x2C | SPCR | SPIE | SPE | DORD | MSTR | CPOL | CPHA | SPR1 | SPR0 |
| 0x2D | SPSR | SPIF | WCOL | x | x | x | x | x | SPI2X |
| 0x2E | SPDR | SPI Data Register | | | | | | | |
| ... | | | | | | | | | |
| 0x30 | ACSR | ACD | ACBG | ACO | ACI | ACIE | ACIC | ACIS1 | ACIS0 |
| ... | | | | | | | | | |
| 0x33 | SMCR | x | x | x | x | SM2 | SM1 | SM0 | SE |
| 0x34 | MCUSR | x | x | x | x | WDRF | BORF | EXTRF | PORF |
| 0x35 | MCUCR | x | BODS | BODSE | PUD | x | x | IVSEL | IVCE |
| ... | | | | | | | | | |
| 0x37 | SPMCSR | SPMIE | RWWSB | SIGRD | RWWSRE | BLBSET | PGWRT | PGERS | SPMEN |

```
...
0x3D    SPL       SP7     SP6     SP5     SP4     SP3     SP2     SP1     SP0
0x3E    SPH       x       x       x       x       x       SP10    SP9     SP8
0x3F    SREG      I       T       H       S       V       N       Z       C
```

## Bits of Ports

Notice how the bits of the port registers compose all of the individual pins:

PORTB – The Port B Data Register

| Bit | 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 | |
|---|---|---|---|---|---|---|---|---|---|
| 0x05 (0x25) | PORTB7 | PORTB6 | PORTB5 | PORTB4 | PORTB3 | PORTB2 | PORTB1 | PORTB0 | PORTB |

DDRB – The Port B Data Direction Register

| Bit | 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 | |
|---|---|---|---|---|---|---|---|---|---|
| 0x04 (0x24) | DDB7 | DDB6 | DDB5 | DDB4 | DDB3 | DDB2 | DDB1 | DDB0 | DDRB |

PINB – The Port B Input Pins Address[1]

| Bit | 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 | |
|---|---|---|---|---|---|---|---|---|---|
| 0x03 (0x23) | PINB7 | PINB6 | PINB5 | PINB4 | PINB3 | PINB2 | PINB1 | PINB0 | PINB |

PORTC – The Port C Data Register

| Bit | 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 | |
|---|---|---|---|---|---|---|---|---|---|
| 0x08 (0x28) | – | PORTC6 | PORTC5 | PORTC4 | PORTC3 | PORTC2 | PORTC1 | PORTC0 | PORTC |

DDRC – The Port C Data Direction Register

| Bit | 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 | |
|---|---|---|---|---|---|---|---|---|---|
| 0x07 (0x27) | – | DDC6 | DDC5 | DDC4 | DDC3 | DDC2 | DDC1 | DDC0 | DDRC |

PINC – The Port C Input Pins Address[1]

| Bit | 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 | |
|---|---|---|---|---|---|---|---|---|---|
| 0x06 (0x26) | – | PINC6 | PINC5 | PINC4 | PINC3 | PINC2 | PINC1 | PINC0 | PINC |

PORTD – The Port D Data Register

| Bit | 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 | |
|---|---|---|---|---|---|---|---|---|---|
| 0x0B (0x2B) | PORTD7 | PORTD6 | PORTD5 | PORTD4 | PORTD3 | PORTD2 | PORTD1 | PORTD0 | PORTD |

DDRD – The Port D Data Direction Register

| Bit | 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 | |
|---|---|---|---|---|---|---|---|---|---|
| 0x0A (0x2A) | DDD7 | DDD6 | DDD5 | DDD4 | DDD3 | DDD2 | DDD1 | DDD0 | DDRD |

PIND – The Port D Input Pins Address[1]

| Bit | 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 | |
|---|---|---|---|---|---|---|---|---|---|
| 0x09 (0x29) | PIND7 | PIND6 | PIND5 | PIND4 | PIND3 | PIND2 | PIND1 | PIND0 | PIND |

The fact that the Standard IO Registers are addressable by bit is a noteworthy feature of Arduinos. The typical method for addressing MMIO is through the use of a Read-Modify-Write strategy. Since port registers contain bits controlling several pins (typically 0 through 7), simply writing to the port without regard to the other bits would overwrite their status. A Read-Modify-Write strategy properly preserves the unaffected bits of register. The Read-Write-Modify procedure is covered in our next chapter which discusses bit manipulation.

Here are two key bit instructions: SBI is the mnemonic for *S*et *B*it in *I*/o register. SBI sets a specified bit in the register. This instruction operates on the lower 32 MMIO addresses. This is not to be confused with the registers r0-r31. And keep in mind, the CBI and SBI instructions work with registers 0x00 to 0x1F only.

CBI is the mnemonic for *C*lear *B*it in *I*/o register, and is the antipode to SBI. CBI clears a specified bit in an I/O register. Again, this instruction operates on the lower 32 I/O registers.

IN and the antipode OUT are also used to load and store I/O locations to and from registers. Remember, the IN and OUT instructions can only access the IO Registers between 0x00-0x3f.

IN performs virtually the same function as LDS (we covered LDS in an earlier chapter, if you would like to refresh your memory). This may lead you to ask, "why use IN vs. LDS?" IN has the following advantages over LDS:

- IN executes twice as fast as LDS (one machine cycle vs. two).
- IN is a 2-byte instruction and LDS is 4-bytes long.
- LDS works on all SRAM, IN only on the Standard IO Registers.

## Digital Right?

Here is an example of how to set the Arduino digital pin #13 HIGH (that's pin #5 of Port B) high. This is the equivalent to the Arduino command, digitalWrite(13, HIGH). However, our inline version uses approximately 170 less bytes, digitalWrite being somewhat of an exercise in Yak Shaving[15].

```
asm (
  "sbi %0, %1 \n"

  : : "I" (_SFR_IO_ADDR(PORTB)), "I" (PORTB5)
);
```

[15] See this blog reference Yak Shaving: https://ucexperiment.wordpress.com/2013/03/25/arduino-digitalwritepin-value/

One curious thing you should have noticed was the use of the C MACRO, _SFR_IO_ADDR. Since the port names are defined with their "memory" address in mind, in order to use them as parameters of the instructions SBI/CBI, (including IN, OUT, SBIS/SBIC), 32 (0x20 hexadecimal) must be subtracted first. Otherwise we will be addressing the wrong locations. For example, PORTB is defined as the following:

```
//sfr_defs.h
#define PORTB _SFR_IO8(0x05)
#define __SFR_OFFSET 0x20
#define _SFR_IO8(io_addr) _MMIO_BYTE ((io_addr) + __SFR_OFFSET)
#define _MMIO_BYTE(mem_addr) (*(volatile uint8_t *)(mem_addr))
#define _SFR_IO_ADDR(sfr) ((sfr) - __SFR_OFFSET)
```

Did you enjoy weaving your way through that convolution? The result is that PORTB = 0x25, however we want to address location 0x05. For simplicity sake, we'll just use the _SFR_IO_ADDR macro. Note that all of the peripherals use this scheme, not just the IO pins. I suggest reading the iom328p.h header-file if you're interested in the locations for other peripherals.

Also, keep in mind, on the Arduino, when using a pin that is also connected to a PWM timer (pins 3, 5, 6, 9, 10 or 11), PWM should be disabled first. However, that's a topic of a later chapter.

## Go LOW

SBI is not enough to completely control the ports by itself. To conversely set a pin LOW, we need to use the CBI instruction. The following is the equivalent of the Arduino command, digitalWrite(13, LOW) [16].

```
asm (
  "cbi %0, %1 \n"

  : : "I" (_SFR_IO_ADDR(PORTB)), "I" (PORTB5)
);
```

Again, this saves many bytes.

## Don't Forget to Mode the Pin

Prior to using a pin, it needs to be configured to behave either as an input or an output. The Arduino function for setting the pin mode is pinMode(13,

[16] See the Arduino website for further information on digitalWrite command: http://www.arduino.cc/en/Reference/DigitalWrite.

OUTPUT)[17]. Remember, pins default as input, so they don't need to be explicitly declared for input. Here is a simple method to set the pin direction:

```
asm (
  "sbi %0, %1          \n"

    : : "I" (_SFR_IO_ADDR(DDRB)), "I" (DDB5)
);
```

Notice we are not referring to the PORT definition of the pin, but rather the Data Direction Register for Port B (DDRB and DDB5). However, it's still necessary to use the SFR_IO_ADDR MACRO with the DDR.

If you know the status for all the pins on a particular port, it is far easier to set the entire port at once, rather than setting individual pins. This is easily accomplished using a bit definition which helps visualize the pins, like so:

```
#define PB_PIN_DIRS 0b00101000 //PIN 3 & 5 output,
//same as #define PB_PIN_DIRS (1<<DDB3) | (1<<DDB5)

asm (
  "out %0, %1 \n"

  : : "I" (_SFR_IO_ADDR(DDRB)), "r" (PB_PIN_DIRS)
);
```

In the event you haven't taken notice, all of the above digital port-writing examples result in the inclusion of one assembly instruction into your program. That's a pretty phenomenal thing if you ask me.

## Do You Read Digital?

The Arduino command to read a digital pin is digitalRead(pin)[18]. Again, we can emulate the Arduino command and consume far less memory by using the SBIS instruction. SBIS stands for *S*kip if *B*it in *I*/o register is Set. SBIS tests a single bit in an I/O register and skips the next instruction if the bit is set. This instruction operates on the lower 32 I/O registers – addresses 0-31. Suppose we want to read digital pin #13 (Port B bit 5), and place the result (low/high, true/false) into the variable status:

```
volatile uint8_t status;
```

[17] See the Arduino website for further information on the pinMode command: https://www.arduino.cc/en/Reference/pinMode.
[18] See the Arduino website for further information on the digitalRead command: https://www.arduino.cc/en/Reference/DigitalRead.

```
asm (
  "in __tmp_reg__, __SREG__  \n"
  "cli                       \n" //disable interrupts
  "ldi %0, 1                 \n"
  "sbis %1, %2               \n" //skip next if pin high
  "clr %0                    \n"
  "out __SREG__, __tmp_reg__ \n"

  : "=r" (status) : "I" (_SFR_IO_ADDR(PINB)), "I" (PINB5)
);
```

As you may have guessed, the SBIS instruction has it's opposite relative, called SBIC. SBIC, or *S*kip if *B*it in *I*/o register is Cleared tests a single bit in an I/O register and skips the next instruction if the bit is cleared. Likewise, this instruction operates on the lower 32 I/O registers – addresses 0-31.

## You're Both Write

Previous examples demonstrated separately setting a port bit and clearing a port bit. Fine, but how about demonstrating a routine that operates like the Arduino's digitalWrite, allowing either HIGH or LOW (set/clear together)? Unfortunately, this requires a comparison and a conditional and unconditional branch, which are subjects of future chapters. However, without explanation, here it is (it's really quite simple):

```
//digitalWrite(output)
volatile uint8_t output = HIGH; //LOW or HIGH

asm (
    "cpi %2, 0  \n"
    "breq 1f    \n"
    "sbi %0, %1 \n"
    "rjmp 2f    \n"
  "1:           \n"
    "cbi %0, %1 \n"
  "2:           \n"

  : : "I" (_SFR_IO_ADDR(PORTB)), "I" (PORTB5), "r" (output)
);
```

## P.S. How Does One Turn Off PWM?

Here is an example of turning off the PWM for Arduino digital pin #11. This is as simple as disconnecting OC2A output compare function from pin #3 on Port B (digital pin #11). But, you need to read the next chapter about the nitty-gritty of port manipulation to understand what is happening in this example:

```
asm (
  "ld  r16, Z \n"
  "ldi r17, 0xff \n"
  "eor r17, %1 \n"
  "and r16, r17 \n"
  "st  Z, r16 \n"

  : : "z" (_SFR_MEM_ADDR(TCCR2A)), "d" (COM2A1) : "r16", "r17"
);
```

# Chapter 8

# Status Register

Like all microcontrollers the Arduino's AVR μC has a flag register to indicate various conditions. The Status Register is referred to in short as the "SREG", and can be directly referenced inline using __SREG__. It is an 8-bit wide register, containing 6 condition flags, a control register and a special bit instruction register.

**SREG – AVR Status Register**

| Bit | 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 | |
|---|---|---|---|---|---|---|---|---|---|
| 0x3F (0x5F) | I | T | H | S | V | N | Z | C | SREG |
| Initial Value | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | |

The status register bits are:

```
C – Carry flag
Z – Zero flag
N – Negative flag
V – Overflow flag
S – Sign flag
H – Half carry
T – Bit Copy storage
I – Global Interrupt Enable flag
```

The following is a brief explanation of each bit of the Status Register.

## C – Carry Flag

This flag is set whenever there is a carry out of the 7th-bit position (MSB). This bit is affected after an 8-bit addition or subtraction operation.

## Z – Zero Flag

The zero flag reflects the result of an arithmetic or logic operation. If the result is zero, then this bit = 1, conversely, if this bit = 0, the result was not zero. This flag is very useful for controlling program loops.

## N – Negative Flag

The negative flag reflects the result of an arithmetic operation. Signed numbers use the 7th-bit (MSB) as a sign bit. If the 7th-bit of the result is zero, then this flag = 0, meaning the result is positive. Conversely, if the 7th-bit of the result is set, then this flag =1, and the result is negative. The Negative (N) and Overflow (V) flags are used for signed number arithmetic operations.

## V – Overflow Flag

This flag is set whenever the result of a signed number operation is too large, causing the high-order bit to overflow into the sign bit. In general, the Carry flag (C) is used to detect errors in unsigned arithmetic and the Overflow flag (V) is used to detect errors in signed arithmetic operations. Both the Overflow (V) and Negative (N) flags are used for signed arithmetic operations.

## S – Sign Flag

This flag is the result of an Exclusive-OR of the Overflow (V) and Negative (N) flags.

## H – Half Carry Flag

If there is a carry from bit 3 to 4 during an ADD or SUB operation, this bit is set, otherwise it is cleared. This flag is used for Binary Coded Decimal[19] arithmetic.

## T – Bit copy Storage Bit

The T bit is used as a temporary storage location for bit operations involving the BLD and BST instructions. BLD, or *B*it *Lo*a*D* copies the T flag in the Status Register to a particular bit in a register.

[19] See https://en.wikipedia.org/wiki/Binary-coded_decimal.

BST, is *B*it *ST*ore from a bit in a register to T flag in the Status Register. Here is how we can use these instructions:

```
asm (
  //Copy bit
  "clr r0 \n"
  "ldi r16, 0b00000100 \n"
  "bst r16, 2 \n" //Store bit 2 of r1 in T flag
  "bld r0, 4 \n" //Load T into bit 4 of r0

  : : : "r0", "r16"
);
```

There are several additional instructions that specific manipulate or reference the T-flag. SET (*SE*t *T* Flag), CLT (*CL*ear *T* Flag), BRTS (*BR*anch if the *T* Flag is *S*et), and BRTC (*BR*anch if the *T* Flag is *C*leared).

## I – Global Interrupt Enable

This bit must be set for interrupts to be enabled. It is cleared by the microcontroller after an interrupt has occurred, but may be set again manually with the *sie()* instruction to allow nested interrupts. It is also set by the RETI instruction upon completion of the interrupt. This enables subsequent interrupts. In addition to this Global Interrupt Enable flag, each individual interrupt enable bit(s) must be set in the appropriate control register.

For example, for timer counter 1, both the timer 1 overflow interrupt enable bit (TOIE1) and global interrupts need to be enabled, to allow the TCNT1 over flow to cause an interrupt.

To set the Global Interrupt Flag, use the SEI, or *SE*t global *I*nterrupt Flag instruction. The instruction following SEI will be executed before any pending interrupts.

To clear the Global Interrupt Flag, use the CLI, or *CL*ear global *I*nterrupt Flag instruction. Interrupts will be immediately disabled. No interrupt will be executed after the CLI instruction, even if it occurs simultaneously with the CLI instruction.

```
                          asm (“sei \n”);
                          asm (“cli \n”);
```

## Instructions That Affect Flag Bits

```
Inst.   Z  C  N  V  S  H  T  I
-----------------------------
```

| | | | | | | | | |
|---|---|---|---|---|---|---|---|---|
| ADD | Z | C | N | V | S | H | - | - |
| ADC | Z | C | N | V | S | H | - | - |
| ADIW | Z | C | N | V | S | - | - | - |
| AND | Z | - | N | V | S | - | - | - |
| ANDI | Z | - | N | V | S | - | - | - |
| ASR | Z | C | N | V | - | - | - | - |
| BST | - | - | - | - | - | - | T | - |
| CBR | Z | - | N | V | S | - | - | - |
| CLC | - | C | - | - | - | - | - | - |
| CLH | - | - | - | - | - | H | - | - |
| CLI | - | - | - | - | - | - | - | I |
| CLN | - | - | N | - | - | - | - | - |
| CLR | Z | - | N | V | S | - | - | - |
| CLS | - | - | - | - | S | - | - | - |
| CLT | - | - | - | - | - | - | T | - |
| CLV | - | - | - | V | - | - | - | - |
| CLZ | Z | - | - | - | - | - | - | - |
| COM | Z | C | N | V | S | - | - | - |
| CP | Z | C | N | V | - | H | - | - |
| CPC | Z | C | N | V | - | H | - | - |
| CPI | Z | C | N | V | - | H | - | - |
| DEC | Z | - | N | V | S | - | - | - |
| EOR | Z | - | N | V | S | - | - | - |
| INC | Z | - | N | V | S | - | - | - |
| LSL | Z | C | N | V | - | H | - | - |
| LSR | Z | C | N | V | - | - | - | - |
| NEG | Z | C | N | V | - | H | - | - |
| OR | Z | - | N | V | S | - | - | - |
| ORI | Z | - | N | V | S | - | - | - |
| ROL | Z | C | N | V | - | H | - | - |
| ROR | Z | C | N | V | - | - | - | - |
| SBR | Z | | N | V | - | - | - | - |
| SBCI | Z | C | N | V | - | H | - | - |
| SBIW | Z | C | N | V | - | - | - | - |
| SBC | Z | C | N | V | - | H | - | - |
| SEC | - | C | - | - | - | - | - | - |
| SEH | - | - | - | - | - | H | - | - |
| SEI | - | - | - | - | - | - | - | I |
| SEN | - | - | N | - | - | - | - | - |
| SES | - | - | - | - | S | - | - | - |
| SET | - | - | - | - | - | - | T | - |
| SEV | - | - | - | V | - | - | - | - |
| SEZ | Z | - | - | - | - | - | - | - |
| SUB | Z | C | N | V | S | H | - | - |
| SUBI | Z | C | N | V | S | H | - | - |
| TST | Z | - | N | V | S | - | - | - |

## Stupid SREG Tricks

This Arduino program will iterate through the flags in the Status Register, turning them on individually:

```
//SREG demo
#define SREG_IBIT 0b10000000

char s[6], f[8] = { 'C', 'Z', 'N', 'V', 'S', 'H', 'T', 'I' };

void PrintSreg(uint8_t sreg) {
  for (uint8_t i=0; i<8; i++) {
    sprintf(s, " %c: %d", f[i], (sreg & (1 << i) ? 1 : 0));
    Serial.print(s);
  }
  Serial.println();
}

void setup() {
  Serial.begin(9600);

  //Carry Flag
  asm (
    "out __SREG__, %0 \n"
    "sec              \n"

    : : "r" (SREG_IBIT)
  );

  PrintSreg(SREG);
  //Zero Flag
  asm (
    "out __SREG__, %0 \n"
    "sez              \n"

    : : "r" (SREG_IBIT)
  );

  PrintSreg(SREG);
  //Negative Flag
  asm (
    "out __SREG__, %0 \n"
    "sen              \n"
```

```
    : : "r" (SREG_IBIT)
  );

  PrintSreg(SREG);
  //Overflow Flag
  asm (
    "out __SREG__, %0 \n"
    "sev              \n"

    : : "r" (SREG_IBIT)
  );

  PrintSreg(SREG);
  //Sign Flag
  asm (
    "out __SREG__, %0 \n"
    "ses              \n"

    : : "r" (SREG_IBIT)
  );

  PrintSreg(SREG);
  //Half Carry Flag
  asm (
    "out __SREG__, %0 \n"
    "seh              \n"

    : : "r" (SREG_IBIT)
  );

  PrintSreg(SREG);
  //T bit
  asm (
    "out __SREG__, %0 \n"
    "set              \n"

    : : "r" (SREG_IBIT)
  );

  PrintSreg(SREG);
}

void loop() { }
```

# Chapter 9

# Basic Bit Operations

## Introduction

Bit operations[20] are the foundation upon which all microcontroller programming builds. A good understanding of bit functions is essential for efficient Arduino programming in general, and specifically for inline assembly programming. Its that simple.

It really is simple too. Bit manipulation basically comes down to the capacity to set, clear and toggle bits inside of a byte. The operations which perform these manipulations are termed bitwise operations (AND, OR, XOR, NOT, and shifts).

[20] See https://en.wikipedia.org/wiki/Bit_manipulation.

I don't plan to cover the very basics here. If you don't already have at least a fundamental understanding of bit operations, I suggest starting with a good tutorial[21].

## Truth in a Table

Having said that, I find truth tables are helpful in understanding each bitwise operation. In a truth table, a 1 bit stands for true, and a 0 stands for false. Here are the truth tables for AND, OR and XOR operations:

| AND | | |
|---|---|---|
| A | B | Output |
| 0 | 0 | 0 |
| 0 | 1 | 0 |
| 1 | 0 | 0 |
| 1 | 1 | 1 |

| OR | | |
|---|---|---|
| A | B | Output |
| 0 | 0 | 0 |
| 0 | 1 | 1 |
| 1 | 0 | 1 |
| 1 | 1 | 1 |

| XOR | | |
|---|---|---|
| A | B | Output |
| 0 | 0 | 0 |
| 0 | 1 | 1 |
| 1 | 0 | 1 |
| 1 | 1 | 0 |

In addition to AND, OR and XOR, there is also the NOT operation, which inverts the bit; 1 becomes 0, and 0 becomes 1.

## SO CAN I

I use the mnemonic "SO CAN I" to help remember basic bit functions. "SO", or "Set a bit with Or", and "CAN 1", for "Clear a bit with And Not 1". It works for me!

## Setting the Stage

It's time to twiddle some bits. We will start by setting one bit inside of a byte. For example, let's set bit 0 in byte-sized variable "foo" (storing the result back into foo), which requires the use of the OR operation. We OR, or "SO" between the variable that we want to change and a constant. The constant is referred to as a BIT MASK, or simply the MASK. The mask is used to identify the bit (or bits) that we want to affect. In our example, we just want to set bit 0, so our mask is simply 1, or 0b00000001 (binary). Note, that 1 occupies the right-most bit position, or bit #0. Using the binary form of the number (i.e. 0b00000001) for the mask is helpful because it provides a pictorial representation of the affected bits.

The ORI instruction is the mnemonic for Logical OR with Immediate. ORI performs a logical OR between the contents of a register and a constant and places the result back into the register. The OR instruction is simply a logical OR between the contents of two registers.

[21] One such tutorial can be found here: http://www.avrfreaks.net/forum/tut-c-bit-manipulation-aka-programming-101?page=all.

Finally, if you don't like my "SO CAN I" mnemonic, you can use the SBR instruction, which is the mnemonic for *S*et *B*it in *R*egister. However, the assembler often replaces the SBR instruction with an ORI instruction.

```
//Set bit to immediate value
//C language equivalent: b = b | 0x01;
volatile uint8_t b = 0;

asm (
  "ori %0, %1 \n"

  :"+r" (b) : "M" (0x01)
);
```

If you are already comfortable using the typical C notation to define multiple bits, like so:

```
(1<<3) | (1<<2) | (1<<1) | (1<<0)
```

You can continue to use this technique in your inline assembly code for constant expressions. For example:

```
//corresponds to "ori %0, 15 \n"
volatile uint8_t b = 0;

asm (
  "ori %0, (1<<3) | (1<<2) | (1<<1) | (1<<0) \n"

  :"+r" (b)
);
```

## Clear a Register, And Not

To clear bit 0 in variable foo requires 2 bit operations ("CAN 1"), both the AND, and NOT operations. To clear a bit, the bit mask must first be NOT'd (inverted) and then AND'd with the variable. The first hurdle here, is there is no 'NOT' instruction. Did I just state a "double negative?" However, we realize when EOR'ing something with a one, that bit flips. So to get the inverse bits (or to perform a NOT operation), we EOR it with a byte filled with 1's (0b11111111, or 255):

```
volatile uint8_t foo = 0xff, bit = 1;

asm (
  "eor %1, %2 \n"
  "and %0, %1 \n"
```

```
  :"+r" (foo) : "r" (bit), "r" (0xff)
);
```

Of course if we wanted to perform the NOT operation with a constant value, we can use the C language negation operator '~' (or tilde) with the ANDI instruction, like so:

```
volatile uint8_t foo = 0xff;

asm (
  "andi %0, %1 \n"

  :"+r" (foo) : "i" (~1)
);
```

## Getting One's Bits Inverted

When I said, there is no NOT instruction (was that a double negative again?), I was only partially correct. While there is no instruction termed "NOT", there is a COM instruction which performs a "one's-*COM*plement" of the register. One's complementing[22] a number is the same as inverting it's bits. This allows us to rewrite our initial example for clearing a bit:

```
volatile uint8_t foo = 0xff, bit = 1;

asm (
  "com %1     \n"
  "and %0, %1 \n"

  :"+r" (foo) : "r" (bit)
);
```

## Making Flippy-Flop

Another handy tool flips or toggles a byte on and off, even if you don't know and/or don't care what state the bit is currently. This can easily be accomplished by using the XOR operator:

```
//C language equivalent: foo = foo ^ 0x01;
volatile uint8_t foo = 1;

asm (
  "ldi r16, 1  \n"
```

[22] See https://en.wikipedia.org/wiki/Ones%27_complement.

```
  "eor %0, r16 \n"

  : "+r" (foo) : : "r16"
);
```

## Reading, Modifying and Writing

We previously covered how to set and clear bits within PORT registers using the SBI/CBI instructions. But, can we have a general method for setting and/or clearing several bits on a Port without altering the others? Since Port registers contain bits controlling several pins (typically 0-7), simply writing to the Port without regard to the other bits would overwrite their status. This is the essence of the Read-Modify-Write strategy we briefly talked about previously. The R-M-W technique properly preserves the unaffected bits of register.

Suppose we wanted to set digital pins #11 and #13 (Port B bits 3 & 5). Our mask becomes 0b00101000, 40, or 0x28 hexadecimal. The process is to first read the port, then modify the byte (set our bits with an OR operation), and finally write back to the Port. Conversely, if we EOR and AND the bits, we clear them.

Turn on multiple bits:

```
//Set multiple bits on a port
volatile uint8_t BitMask = 0b00101000;

asm (
  "in r18, %0  \n"
  "or r18, %1  \n"
  "out %0, r18 \n"

  : : "I" (_SFR_IO_ADDR(PORTB)), "r" (BitMask) : "r18"
);
```

Turn off multiple bits:

```
//Clear multiple bits on a port
volatile uint8_t foo = 0xff, bit = 1;

asm (
  "in r18, %0  \n"
  "eor %1, %2  \n"
  "and r18, %1 \n"
  "out %0, r18 \n"

  : : "I" (_SFR_IO_ADDR(PORTB)), "r" (BitMask) , "r" (0xff)
  : "r18"
```

```
);
```

Remember the restrictions on using IN/OUT? These two instructions only work with the Standard IO Registers 0x00 through 0x3f. If we wanted to use the Read-Modify-Write technique on IO Registers above 0x3f, we need to substitute LDS and STS for IN and OUT.

## Not So Exclusive Or

BTW, EOR has several other useful functions. For example, an EOR of itself is the same as CLR (clear), or an LDI with 0 (Load Immediate). Each instruction impacts the Status Register too:

```
"eor r16, r16 \n" //clears SREG S, V and N and sets Z bit
"clr r16      \n" //clears SREG S, V and N and sets Z bit
"ldi r16, 0   \n" //does not effect the SREG
```

An EOR can also be used to compare two registers to determine if they are the same. For example:

```
EOR r0, r1
```

This will perform an Exclusive OR of registers 0 and 1, putting the result in r0. If both registers had the same value, r0 will contain zero. This will form the basis for a following chapter on branching.

## Shifty Bit(s)

A confusing issue with setting or clearing bits arises from the difference between the bit number(s) and the bit mask. The bit number and the bit mask are not the same. For example, the bit position #0 requires a mask of 0b00000001 (0x01), and position #1uses a mask of 0b00000010 (or 0x02), followed by 0x04 for bit #3, and so on.

```
Bit : Mask [bit position]
  0 : 1    [ 0b00000001 ]
  1 : 2    [ 0b00000010 ]
  2 : 4    [ 0b00000100 ]
  3 : 8    [ 0b00001000 ]
  4 : 16   [ 0b00010000 ]
  5 : 32   [ 0b00100000 ]
  6 : 64   [ 0b01000000 ]
  7 : 128  [ 0b10000000 ]
```

## A Practical Bit

Here are two simple yet practical examples of AND'ing and OR'ing. The difference between an ASCII upper and lower case character of the alphabet is simply the status of bit #5. If bit #5 is set, the character is lower case, but when bit #5 is clear the character is upper case. We can use this knowledge to write two simple inline functions (noting of course, that these functions[23] are already included in the "ctype.h" header file of the standard C library):

```
inline char _tolower(char c) {
  asm (
    "ori %0, 0x20 \n" //set bit 5, 'A' becomes 'a'

    : "=r" (c)
  );

  return c;
}

inline char _toupper(char c) {
  asm (
    "andi %0, ~0x20 \n" //clear bit 5, 'a' becomes 'A'

    : "=r" (c)
  );

  return c;
}
```

## Shifting Into Gear

As you can see, there's much to cover on the topic of bit manipulation. Bit operations are used throughout embedded programming in general, and Arduino programming in particular. We've just skimmed the basics of setting, clearing and inverting bits. Our next chapter will continue with bit shift operations.

[23] See http://www.nongnu.org/avr-libc/user-manual/group__ctype.html.

# Chapter 10

# Bit Shifts

## Introduction

In this chapter we continue our examination of bit operations. Specifically we discuss left and right shifts. We'll end this chapter with an assortment of short C language MACRO code segments that demonstrate checking bits.

## Shift to Multiply

The instruction which performs a left bit shift is LSL. LSL is a mnemonic for *L*ogical *S*hift *L*eft, which shifts all bits in the register one place to the left. Bit 0 is cleared, and bit 7 is loaded into the C flag of the Status Register (SREG). This operation effectively multiplies signed and unsigned values by two.

Why would we want to move each bit in a register, one place to the left? Let's look at the example of left-shifting the value of three:

```
volatile uint8_t foo = 0;

asm (
  "ldi %0, 3 \n" //foo = 0b00000011 = 3
  "lsl %0    \n" //foo = 0b00000110 = 3x2 = 6
  "lsl %0    \n" //foo = 0b00001100 = 3x2x2 = 12

  : "=r" (foo)
);
```

Each left shift is also the equivalent of multiplying by 2. If we want to multiply something by eight, we would left-shift it three times, and so on.

LSL has a cousin named ROL, or *RO*tate *L*eft through carry. ROL shifts all bits in one place to the left. The C flag is shifted into bit 0, and bit 7 is shifted into the C flag.

Since LSL always shifts a zero into the lowest bit, and ROL shifts the contents of the carry flag into the lowest bit, using both allows us to multiply numbers larger than one byte. To multiply a 16-bit number by two, first LSL the lower byte, then ROL the high byte. This has the net effect of "rolling" the high bit of the lower byte into the first bit of the 2nd byte. This technique can be expanded to multiply even larger numbers.

Multiplying a 32-bit number by two:

```
volatile uint32_t foo = 0x00f0f0f0;

asm (
  "lsl %A0 \n" //r24=0x00f0f0f0=15790320x2=0x01e1e1e0 (31580640)
  "rol %B0 \n" //number shifted out of %A0 into %B0
  "rol %C0 \n" //number shifted out of %B0 into %C0
  "rol %D0 \n" //number shifted out of %C0 into %D0

  : "+r" (foo)
);
```

## Right Shift to Divide and Conquer

The right-shift operation moves each bit in a register, one place to the right. Why would we want to do this? Again, examine this example of right-shifting the value of twelve:

```
volatile uint8_t foo;

asm (
  "ldi %0, 12 \n" //foo = 0b00001100 = 12
  "lsr %0     \n" //foo = 0b00000110 = 12/2 = 6
  "lsr %0     \n" //foo = 0b00000011 = 12/4 = 3

  : "=r" (foo)
);
```

Each time we shift a value one bit to the right, we are dividing that value by two. If we wanted to divide something by eight, we would right-shift three times.

What the Logical Shift Right command (LSR) does is shift a zero into the highest bit, shift the contents to the right and shift the lowest bit into the carry flag. The C flag can be used to round the result.

LSR always shifts a zero into the highest bit, while ROR shifts the contents of the carry flag into the highest bit. Using both we can divide numbers larger than one byte. To divide a sixteen bit number by two, first LSR the highest byte, then ROR the lower byte, this has the net effect of "rolling" the low bit of the high byte into the high bit of the lower byte. This technique can be expanded to divide even larger numbers.

Dividing a 32-bit number by two:

```
volatile uint32_t foo = 0x01e1e1e0;

asm (
  "lsr %D0 \n" //foo=0x01e1e1e0(31580640/2) =0x00f0f0f0(15790320)
  "ror %C0 \n" //number shifted out of %D0 into %C0
  "ror %B0 \n" //number shifted out of %C0 into %B0
  "ror %A0 \n" //number shifted out of %B0 into %A0

  : "+r" (foo)
);
```

Notice, we are not covering general multiplication and division operations. That topic is beyond the scope of this chapter. Furthermore, it would be very difficult to improve upon the highly efficient routines found inside the arduino/AVR library. Additional information on general integer multiply and divide routines can be found in ATMEL Application Note #200[24].

## Let's Jump

Now we introduce the concept of "jumping." The RJMP, or *R*elative *J*u*MP* instruction jumps to an address within 2048 (words). In assembly, we don't calculate the relative operand portion of the jump, or the amount of instructions to jump over. We use a label as the destination of our jump and let the assembler/linker calculate the amount to jump for us.

A jump instruction causes the µC to start execution of instructions at a location defined by the label. The label's location can be anywhere (forward, backward, and even in the same place) within the distance limits defined. However, for inline assembly, the label must be located inside the same block of inline code. Take a close look at the following code. It does nothing except cause execution to jump between the two labels, "here" and "there". The 3 "NOP" instructions in

[24] See http://www.atmel.com/Images/doc0936.pdf.

between are never executed.

```
here:
  rjmp there
  nop
  nop
  nop
there:
  rjmp here
```

## You're Both Write

In a previous chapter we demonstrated a routine that operates like the arduino's digitalWrite function. It uses an unconditional branch (RJMP) to skip over the clear bit instruction (CBI), if the result of the comparison (CPI) with 0 is true.

Here it is again:

```
//digitalWrite(output)
volatile uint8_t output = HIGH; //LOW or HIGH

asm (
  "cpi %2, 0     \n"
  "breq 1f       \n"
  "sbi %0, %1    \n"
  "rjmp 2f       \n"
"1:              \n"
  cbi %0, %1     \n"
"2:              \n"

  : : "I" (_SFR_IO_ADDR(PORTB)), "I" (PORTB5), "r" (output)
);
```

## Bit Checking MACROs

These C language MACROs are useful for checking the status of particular bits. The first two use relative jump instructions. Newer, C-language versions of these MACROs are defined inside the file "sfr_defs.h".

```
#define _loop_until_bit_is_set(port, bit)           \
  asm volatile (                                    \
    "L_%=: sbis %0, %1 \n"                          \
    "rjmp L_%=         \n"                          \
    : : "I" ((uint8_t)(port)), "I" ((uint8_t)(bit)) \
)
```

```
#define _loop_until_bit_is_clear(port, bit)       \
  asm volatile (                                  \
  "L_%=: sbic %0, %1 \n"                          \
  "rjmp L_%=         \n"                          \
  : : "I" ((uint8_t)(port)), "I" ((uint8_t)(bit)) \
)
```

```
#define _bit_is_clear(port, bit) ({               \
  uint8_t t;                                      \
  asm volatile (                                  \
    "clr %0      \n"                              \
    "sbis %1, %2 \n"                              \
    "inc %0      \n"                              \
    : "=r" (t)                                    \
    : "I" ((uint8_t)(port)), "I" ((uint8_t)(bit)) \
  );                                              \
  t;                                              \
})
```

```
#define _bit_is_set(port, bit)                    \
  ({ uint8_t t;                                   \
  asm volatile (                                  \
    "clr %0      \n"                              \
    "sbic %1, %2 \n"                              \
    "inc %0      \n"                              \
    : "=r" (t)                                    \
    : "I" ((uint8_t)(port)), "I" ((uint8_t)(bit)) \
  );                                              \
  t; })
```

An example using the above MACROs:

```
Wait_Until_Clear:
  //pause here in loop checking until until bit is clear
  if ( _bit_is_set(_SFR_IO_ADDR(PORTB), PORTB5) )
    goto Wait_Until_Clear;

  //or, this which performs the same thing
  _loop_until_bit_is_clear(_SFR_IO_ADDR(PORTB), PORTB5);
```

C MACROs offer a convenient way to insert assembly code into your program, but they demand extra care because a MACRO expands into a single logical line. Writing a proper C MACRO is an art. If you desire to reuse your assembler language parts, it is useful to define them as MACROs and put them into include files. AVR Libc comes with a bunch of them, which could be found in the

directory “avr/include.” You may want to peruse these files and study a few examples.

Do note, that a problem with reused macros arises if you are using labels. In such cases you may make use of the special pattern =, which is replaced by a unique number in each asm statement. As in the above _loop_until_bit_is_clear MACRO, notice how the relative jump label is both defined ( L_%=: ) and referenced ( rjmp L_%= ). When the MACROS is used for the first time, for example, L_= may be translated to L_1404, while the next usage might create L_1405. This results in each instance of the label becoming unique.

# Chapter 11

# Basic Math

## Increment, Decrement, Add and Subtract

INC is a mnemonic for "*INC*rement". INC adds one to the contents of the register placing the result in the register. The C flag in the Status Register (SREG) is not affected by the operation, thus allowing the INC instruction to be used on a loop counter in multiple-precision computations.

```
volatile char a = 0;

asm (
  "inc %0 \n"

  : "+r" (a)
);
```

Remember, you might be tempted to write something similar to the following. While it is not wrong, it's just inefficient:

```
asm (
  "mov __tmp_reg__, %0 \n"
  "inc __tmp_reg__     \n"
  "mov %0, __tmp_reg__ \n"

  : "+r" (a)
);
```

DEC is a mnemonic for "*DEC*rement". DEC subtracts one from the contents of the register placing the result in the register. The C flag in SREG is not affected by the operation, thus allowing the DEC instruction to be used on a loop counter in multiple-precision computations.

```
volatile char a = 2;

asm (
  "dec %0 \n"

  : "+r" (a)
);
```

SUBI is a mnemonic for "*SUB*tract *I*mmediate", which subtracts a register and a constant and places the result in the register. This instruction works with registers *R16* to *R31*.

```
volatile char x = 2;

asm (
  "subi %0, %1 \n"

  : "+r" (x) : "K" (2)
);
```

The ADD instruction is really the mnemonic for "*ADD* without carry". However, without getting ahead of ourselves, try to ignore the reference to "carry" for now. ADD, simply adds two registers without the C flag and places the result in the destination register.

```
volatile char x = 10, y = 20, z = 0;

asm (
  "add %1, %2 \n"

  : "=r" (z) : "r" (x), "r" (y)
);
```

## Where is ADDI?

Guess what? There's no Add Immediate instruction. Not to fret, "add immediate" is the same as a SUBI operation using the "two's complement" of a number.

## Two Complements?

First, a “One’s Complement” is to negate a number. Negate by flipping the bits (make 0 bits 1, and 1 bits 0). Thus, 12 which is 00001100 in binary, becomes 11110011 (-12 in binary). As in a signed magnitude, the leftmost bit indicates the sign (1 is negative, 0 is positive).

For a “Two’s Complement[25]”, begin with the number’s “One’s Complement” and add 1 if the number is negative. Thus, continuing the previous example, twelve represented as 00001100 (binary), negated becomes 11110011 (-12 in binary), and subtracting one, becomes 11110100 (binary).

```
volatile char x = 2;

asm (
  "subi %0, %1 \n"      //subi twos-complement = add immediate

  : "+r" (x) : "i" (-2) //(-2)=0xfe, and (2 – (0xfe))=4
);
```

## Carry What?

The carry flag is a single bit in the Status Register (SREG). It indicates when an arithmetic carry or borrow has been generated out of the most significant bit position. The carry flag is affected by most arithmetic instructions. Several arithmetic instructions have two forms, which either read or ignore the carry. The carry flag enables multi-byte add, subtract, shift, and rotate operations.

For example, if one were to add 255 and 1 using 8-bit registers, the result should be 256, or 100000000 in binary. You will notice that number requires 9 bits. The 8 least significant bits, or the result stored into a byte-sized register would be 00000000 binary (0 in decimal). There’s a need to carry a one out of bit 7 (the eighth bit). When this occurs, the SREG carry bit is set, indicating that the result needs to spill over into the next byte. The valid 9-bit result is the concatenation of the carry flag with the result.

Note, that in an 8-bit two’s complement interpretation, this operation is (−1) + (−1) and yields the correct result of −2, with no overflow, even if the carry is ignored.

## Carry or Borrow?

While the carry flag works as described for addition, when subtracting, one uses the bit as a borrow flag. Set the flag if a < b when computing a − b, and borrowing must occur. A subtract with carry (SBC) instruction will compute a − b − C = a − (b + C), while subtract without carry (SUB) acts as if the borrow bit were clear.

[25] See https://en.wikipedia.org/wiki/Two%27s_complement.

You may want to re-read that section.

ADC is a mnemonic for "*AD*d with *C*arry", which adds two registers and the contents of the C flag and places the result in the destination register.

```
volatile int x = 0x00ff, y = 0x0001, z = 0;

asm (
  "add %A1, %A2  \n" //Add low byte
  "adc %B1, %B2  \n" //Add with carry high byte

  : "=r" (z) : "r" (x), "r" (y)
);
```

## Integer Add with Carry the Easy Way

ADIW is the mnemonic for "*AD*d *I*mmediate to *W*ord". The ADIW instruction adds an immediate value between 0 and 63 to a register pair and places the result in the register pair. Why only values between 0, and 63? I don't know. However, notice that there is a constraint explicitly designated for this range of numbers, 'I', and it would be wise to use it here.

```
volatile int x = 0x00ff, y = 0;

asm (
  "adiw %1, %2 \n"

  : "=r" (y) : "r" (x), "P" (1)
);
```

SBC is a mnemonic for "*S*u*B*tract with *C*arry", which subtracts two registers and subtracts with the C flag and places the result in the destination register.

```
volatile int x = 0x0100, y = 0x0001, z = 0;

asm (
  "sub %A1, %A2 \n" //Subtract low byte
  "sbc %B1, %B2 \n" //Subtract with carry high byte

  : "=r" (z) : "r" (x), "r" (y)
);
```

However, as with addition, there is an easier method also within limitations. SBIW is the mnemonic for "*S*u*B*tract *I*mmediate from *W*ord". SBIW subtracts an

immediate value between 0 and 63 from a register pair and places the result in the register pair. Notice the use of 'I' constraint again.

```
volatile int x = 0x0100;

asm (
  "sbiw %0, %1  \n"

  : "+r" (x) : "I" (1)
);
```

## My Cup Overflows

When performing math (even basic addition and subtraction) with signed numbers an overflow problem sometimes arises. The Arduino microcontroller indicates the existence of an overflow error by setting the overflow flag in the SREG. Here's a demonstration of the overflow problem with a simple addition operation:

```
volatile int8_t n1=0x70; //112
volatile int8_t n2=0x35; //53
volatile int8_t answer;

void setup() {
  Serial.begin(9600);

  asm(
    "add %1, %2 \n"

    : "=r" (answer) : "r" (n1), "r" (n2)
  );

  Serial.print("answer = "); Serial.println(answer);
}
```

The result to the above addition is, *answer = -91*, or 0xA5 hexadecimal. That's wrong! The reason the answer turns out wrong is because the result is larger than the 8-bit register can hold. The largest signed 8-bit number is +127, or 0x7f hexadecimal. However, the overflow flag (V flag) was set during this addition to warn us that the result is erroneous. However, it's completely up to us, the programmer to deal with this issue.

## What's Your Sign?

In 8-bit signed number operations, the overflow flag is set when either of the following two conditions occur:

- There is a carry from bit 6 to bit 7, but no carry out of bit 7 (C flag not set).
- There is a carry out of bit 7 (C flag set), but no carry from bit 6 to bit 7.

I bring these two cases to your attention, because we can perform addition on two negative numbers and the sign bit can remain correct, but the addition could still overflow. For example, when adding -2 (0x80) and -128 (0xFE), the result becomes 0x7E (+126), which again is incorrect.

When adding two numbers with different signs, the absolute value of the result is a smaller number than the absolute value of the operands prior to addition. In this case, an overflow is impossible. Therefore overflow is only possible when adding two numbers with the same sign. Furthermore, when adding two same-signed numbers, the sign of the result must be the same. The conclusion here is, for signed number addition, if the overflow flag is set, the result is invalid, and in unsigned addition, if the carry flag is set, the result is invalid. In signed number operations, overflow is possible, and overflow corrupts the result and negates the sign bit.

## Multiplication is Hard(ware)

The Arduino's AVR microcontroller has several instructions dedicated to multiplication. We will primarily discuss the MUL instruction, however there are additional instructions for signed numbers (MULS) and fractional multiplication (FMUL).

MUL, which is the obvious mnemonic for "*MUL*tiply", performs a byte-by-byte multiplication. This means the operands (multiplicand and multiplier) must be inside registers. After the multiplication operation, the result is placed in r0 (low byte) and r1 (high byte). If r0 or r1 was selected as an operand register, the result will overwrite those registers. The same register can be selected for both multiplicand and multiplier in order to accomplish a "squaring".

Here is an example of a 8x8 multiplication producing a 16-bit result:

```
volatile uint8_t m1=20, m2=25;
volatile uint16_t result;

asm (
  "mul  %2, %1       \n"
  "movw  %0, r0      \n"
  "clr  __zero_reg__ \n" //r1 trashed by mul
  : "+a" (result) : "a" (m1), "a" (m2)
);
```

Here is a 16x16-bit multiplication producing a 16-bit result:

```
volatile uint16_t m1=255, m2=256, result;

asm (
  "mul  %A2, %A1 \n" //al*bl
  "movw %A0, r0  \n"
  "mul  %B2, %A1 \n" //ah*bl
  "add  %B0, r0  \n"
  "mul  %B1, %A2 \n" //bh*al
  "add  %B0, r0  \n"
  "clr  __zero_reg__ \n"

  : "=&a" (result) : "a" (m1), "a" (m2)
);
```

A 16x16-bit multiplication which produces a 32-bit result:

```
volatile uint16_t m1=256, m2=256;
volatile uint32_t result;

asm (
  "clr  r2 \n"
  "mul  %B2, %B1 \n" //ah*bh
  "movw %C0, r0  \n"
  "mul  %A2, %A1 \n" //al*bl
  "movw %A0, r0  \n"
  "mul  %B2, %A1 \n" //ah*bl
  "add  %B0, r0  \n"
  "adc  %C0, r1  \n"
  "adc  %D0, r2  \n"
  "mul  %B1, %A2 \n" //bh*al
  "add  %B0, r0  \n"
  "adc  %C0, r1  \n"
  "adc  %D0, r2  \n"
  "clr  __zero_reg__ \n"

  : "=&a" (result) : "a" (m1), "a" (m2)
);
```

And finally, a 16x16-bit signed multiplication which produces a 32-bit result:

```
volatile int16_t m1=-127, m2=127;
volatile int32_t result;

asm(
  "clr   r2       \n"
```

```
  "muls  %B2, %B1 \n" //(signed)ah*(signed)bh
  "movw  %C0, r0  \n"
  "mul   %A2, %A1 \n" //al*bl
  "movw  %A0, r0  \n"
  "mulsu %B2, %A1 \n" //(signed)ah*bl
  "sbc   %D0, r2  \n"
  "add   %B0, r0  \n"
  "adc   %C0, r1  \n"
  "adc   %D0, r2  \n"
  "mulsu %B1, %A2 \n" //(signed)bh*al
  "sbc   %D0, r2  \n"
  "add   %B0, r0  \n"
  "adc   %C0, r1  \n"
  "adc   %D0, r2  \n"
  "clr   __zero_reg__ \n"

  : "=&a" (result) : "a" (m1), "a" (m2)
);
```

## Remain Divided

The Arduino's AVR has no instruction for division operations. A simple method to implement integer division is by repeated subtraction. To divide a byte by a byte, the denominator is subtracted repeatedly from the number, with the quotient being the number of times the subtraction occurred. The remainder is placed in the number upon completion.

```
volatile uint8_t number=95, denominator=10;
volatile uint8_t quotient;

asm(
  "1:           \n"
    "inc %0     \n"
    "sub %1, %2 \n"
    "brpl 1b    \n"
    "dec %0     \n"
    "add %1, %2 \n"

    : "=&a" (quotient), "+a" (number) : "a" (denominator)
);
```

## Carry On

Our next topic will cover “branch” operations, which will wrap up the basics of assembly language programming. This will enable us to start writing actual short snippets of code, suitable for inclusion in an Arduino program. Topics for future chapters will include functions and interrupts.

# Chapter 12

# Branching

## Loop and Branch

Branching is a fundamental feature of computers. Branching allows a computer to repeat instruction sequences. The most basic form of repetition is a “loop”. The loop is probably the most widely used programming technique.

There are two type of branches, unconditional and conditional. An unconditional branch is basically a JUMP. We previously briefly discussed jumps. This chapter will examine loops utilizing conditional branches.

## Not Equal

Basic loops in the C language use the “for” construct. Here is a very simple countdown for loop in C that repeats 8 times:

```
for (i=8; i>0; i--) {
  //repeat instructions here
}
```

Let’s duplicate the loop with inline assembler:

```
volatile uint8_t counter = 8;

asm (
  "1:           \n"
```

```
    "nop        \n" //repeating code goes here
    "dec %1     \n"
    "brne 1b    \n"

    : : "r" (counter)
  );
}
```

First, we're not performing any function inside the body of this loop, with the exception of killing time with a NOP. Obviously, in an actual loop we would replace the NOP with functional code.

Second, notice how we decrement the counter value and then use the instruction BRNE (*BR*anch if *N*ot *E*qual) to loop back to the label '1'. We add 'b' to the label to inform the assembler we are branching to a label located "before" the current instruction. If the label was after the branch instruction, we would be branching "forward", and use "1f" instead of "1b".

BRNE is a conditional relative branch. It tests the Zero flag (Z) of the Status Register (SREG) and branches if Z is cleared. When the counter value is decremented to 0, the Zero flag is set, and the branch doesn't occur.

Finally, the counter must be pre-loaded with the number of loop iterations. If counter was zero when this inline code gets executed, then the loop will iterate 255 times! Which leads us to ask, how can we get a loop to repeat more than 255 times?

## Nesting

One possible solution is to nest loops. Placing a loop inside of another loop would allow for255 * 255 = 65,025 iterations. More iterations would require using a 16-bit or even a 32-bit integer as the loop counter (see our delay example at the end of this post).

Here is an example of a nested loop:

```
volatile uint8_t outer = 0xff;
volatile uint8_t inner = 0xff;

asm (
  "1:                   \n" //outer loop
    "mov __tmp_reg__, %0 \n" //(re)load inner loop counter

  "2:                   \n" //inner loop
                            //perform stuff here...
```

```
    "dec __tmp_reg__      \n" //DEC inner loop counter
    "brne 2b              \n" //branch to '1' if tmp_reg not 0

    "dec %1               \n" //DEC outer loop counter
    "brne 1b              \n" //branch to '2' if "outer" not 0

    : : "r" (inner), "r" (outer)
);
```

## More Branches

Prior to a branch instruction, there must be an operation which sets a flag in the Status Register (SREG). In the above examples we used the DEC instruction to perform this operation. However, there are several methods for setting flags besides INC, DEC, ADD and SUB. We can also conduct simple comparisons and tests. Keep in mind, probably the two most widely used comparison instructions are CP and CPI.

CP, CPI, CPC, CPSE and TST are explicitly designed for this purpose.
As you may have guessed, there are many more branch instructions beside just BRNE. We can branch if equal (BREQ), if same or higher (BRSH), if lower (BRLO), if carry is clear (BRCC), or carry set (BRCS). Also, one must be careful to use the appropriate instruction when working with signed vs. unsigned values. Its important to realize that comparisons involving both signed and unsigned values require different branch instructions. Unsigned values are between 0 and 255, and signed values are between -128 and 127. The following two tables summarize the comparisons and branch instructions in order to accomplish the desired result. Notice that a few of the comparison require two branch instructions.

| Unsigned Conditional Branches | | |
|---|---|---|
| CP rX, rY (or) CPI rX, Num | | |
| **Wanted** | **Branch(es)** | **Flags** |
| rX > rY/Num | BREQ <no><br>BRLO <no><br><yes> | C or Z=0<br>(both cleared) |
| rX >= to rY/Num | BRSH <or><br>BRCC | C=0 |
| rX = rY/Num | BREQ | Z=1 |
| rX <= to rY/Num | BREQ <yes><br>BRLO <yes><br><no> | C or Z=1<br>(either set) |
| rX < rY/Num | BRLO (or)<br>BRCS | C=1 |
| rX != rY/Num | BRNE | Z=0 |

The comparison instructions operate like a subtraction (without saving the result). Therefore, these tables are valid to subtraction operations as well.

| Signed Conditional Branches | | |
|---|---|---|
| CP rX, rY (or) CPI rX, Num | | |
| **Wanted** | **Branch(es)** | **Flags** |
| rX > rY/Num | BREQ <no><br>BRLT <no><br><yes> | Z or S=0<br>(both cleared) |
| rX >= to rY/Num | BRGE | S=0 |
| rX = rY/Num | BREQ | Z=1 |
| rX <= to rY/Num | BREQ <yes><br>BRLT <yes><br><no> | Z or S=1<br>(either set) |
| rX < rY/Num | BRLT | S=1 |
| rX != rY/Num | BRNE | Z=0 |

One last item of note, many times when using a particular branch instruction, the assembler will substitute a different. Keep this in mind if you are examining the code produced by the compiler and assembler and you see a different instruction than the one you used.

## Practically Speaking

Lets look at a few practical, yet simple examples (isspace, isdigit, and isalpha). These are all standard C library functions defined inside the "ctype.h" header file. They all take an ASCII[26] integer char as input and return an integer. Take note however, our perverted versions accept and return a char-sized parameter.

## A Space

The standard C function isspace(c) returns true for the standard "white-space" characters:

```
' '  (0x20)
'\t' (0x09)
'\n' (0x0a)
'\v' (0x0b)
'\f' (0x0c)
'\r' (0x0d)
```

Our function only detects a space character, ' ' (0x20). In this code we demonstrate the use of the BREQ instruction:

```
uint8_t _isspace(unsigned char c) {
  uint8_t result;

  asm (
      "cpi  %1, ' ' \n"
      "breq 1f      \n" //branch if equal
      "clr  %0      \n" //false
      "rjmp 2f      \n"
    "1:             \n"
      "ldi  %0, 1   \n" //true
    "2:             \n" //exit

      : "=r" (result) : "r" (c)
  );

  return result;
}
```

## Going Digital

The standard C function isdigit(c) returns true for the characters '0' (0x30) through '9' (0x39) and the negative sign '-' (0x2d). Our function only detects digits, neglecting the negative sign. This code demonstrates the use of the BRMI and BRPL instructions:

[26] See http://www.asciitable.com/.

```
uint8_t _isdigit(unsigned char c) {
  uint8_t result;

  asm (
    "subi %1, 0x30 \n"
    "brmi 2f       \n" //branch if minus
    "subi %1, 10   \n"
    "brpl 2f       \n" //branch if plus
    "ldi  %0, 1    \n" //true
    "rjmp 3f       \n"
    "2: clr  %0    \n" //false
    "3:            \n" //exit
    : "=r" (result) : "r" (c)
  );

  return result;
}
```

## Alphabet Soup

The standard C function isalpha(c) returns true for the characters ‘a’ (0x61) through ‘z’ (0x7a), and ‘A’ (0x41) through ‘Z’ (0x5a). Our function does the same. This code demonstrates the use of the BREQ and BRPL instructions:

```
uint8_t _isalpha(unsigned char c) {
  uint8_t result;

  asm (
      "sbrs %1, 6     \n" //check bit 6
      "rjmp 1f        \n" //bit 6 is clear, cannot be alpha
      "andi %1, ~0x60 \n" //clear bit 5&6
      "breq 1f        \n" //0 cannot be alpha
      "subi %1, 27    \n" //26 letters
      "brpl 1f        \n" //>z cannot be alpha
      "ldi %0, 1      \n" //true
      "rjmp 2f        \n"
    "1:               \n"
      "clr  %0        \n" //false
    "2:               \n" //exit

      : "=r" (result) : "r" (c)
  );

  return result;
}
```

## Labels

Previously we showed how to use numbers as labels. There are other valid methods. A problem arises when reusing macros using labels. In such cases you may make use of the special pattern %=, which is replaced by a unique number on each asm statement. The following code had been taken from "avr/include/iomacros.h":

```
#define loop_until_bit_is_clear(port,bit) \
  asm (                                   \
    "L_%=:       \n"                      \
    "sbic %0, %1 \n"                      \
    "rjmp L_%=   \n"                      \
    : : "I" (_SFR_IO_ADDR(port)), "I" (bit))
```

When used for the first time, L_%= may be translated to L_1404, the next usage might create L_1405 or whatever. In any case, the labels become unique. Another option is to use actual names for the labels. The above example would then look like:

```
#define loop_until_bit_is_clear(port,bit) \
  asm (                                   \
    "start:      \n"                      \
    "sbic %0, %1 \n"                      \
    "rjmp start  \n"                      \
    : : "I" (_SFR_IO_ADDR(port)), "I" (bit))
```

## Wait, One More Thing

Finally, an assembly language tutorial series without a home-grown version of a delay routine would be like a swimming pool without water. Here is where we present our version. The following code produces approximately a 1 second delay on an Arduino. The included C language MACROs allow adapting this code for other delay periods. Take note, this code does not load the MSB of the 32-bit "DELAY_VALUE", so a delay longer than ~1.048 seconds (DELAY_VALUE larger than 0x00ffffff) would require a slight modification (by now, you should be able to handle that). Also, take note of the label we use:

```
#define CLOCK_MHZ       16UL
#define DELAY_LENGTH_MS 1000UL
#define DELAY_VALUE     (uint32_t)((CLOCK_MHZ * 1000UL *
DELAY_LENGTH_MS) / 5UL)

asm (
"loop:          \n"
```

```
  "subi %A2, 1  \n"
  "sbci %B2, 0  \n"
  "sbci %C2, 0  \n" //note: 1 byte short of full 32-bits
  "brcc loop    \n"

  : : "r" (DELAY_VALUE)
);
```

# Chapter 13

# Strings

## Addressing Modes

When loading and storing data, there are several addressing methods available for use. The Arduino's AVR microcontroller supports 13 address modes for accessing the Program memory (Flash) and Data memory (SRAM, Register file, I/O Memory, and Extended I/O Memory). Six modes use "direct addressing", and as such are very basic. The direct modes are generally inherent in the assembly instruction. The good news is that, we covered all six in past chapters, so there is no need to address them here (pun intended). Four additional modes incorporate indirect addressing, and will be the focus of this chapter.

```
                        ADDRESSING MODES
*Register Direct, Single Register-
*Register Direct, Two Registers
*IO Direct
*Data Direct
Data Indirect
Data Indirect w/Displacement
Data Indirect w/Pre-Decrement
Data Indirect w/Post-Increment
Program Memory Constant
Program Memory w/Post-Inc
*Direct Program
Indirect Program
*Relative Program
```

---
* denotes previously covered.

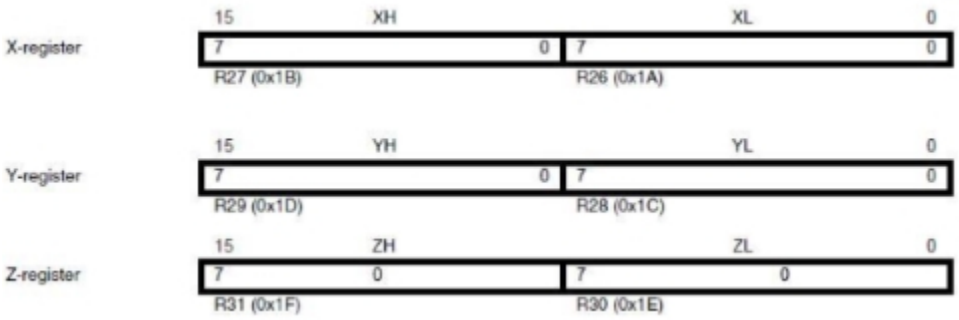
## String Theory

Indirect addressing can be said to involve "pointers[27]". In the C language, the word "pointer" scares people. Hopefully we can calm these irrational fears, by coding an assortment of string routines using simple indirect addressing modes. By the end of this chapter, we should have a good basis for a library of string functions.

## X, Y AND Z

The six registers, *r26* through r31 can be paired together and referenced using the letters X, Y and Z. (the X register is *r27:r26*, the Y register is *r29:r28*, and the Z register is *r31:r30*). When combined, these registers are 16-bit "address pointers" for indirect addressing of the data space. In use, the X, Y and Z register pairs are loaded with an address of interest.

The three indirect address registers X, Y, and Z are defined as described here:




## Speaking Indirectly

Previously we used the LDS instruction to load the value stored inside SRAM memory. For example, this code loads the number 42 into register *r24*:

```
volatile uint8_t x=42;

asm (
  "lds r24, (x) \n"
);
```

[27] See https://ucexperiment.wordpress.com/2015/01/07/arduino-c-pointers/.

But with the X, Y and Z pointer registers, we load the SRAM address into the register pairs (not the value stored there). Hence, we use the the term "indirect addressing". For example, the following code loads the "address" of the string, (src) into the X register pair via the constraint, "x" (src). When we want the first character of the string, or as in this case, 'a', we load it "indirectly" from the X register pair (address) like so:

```
const char src[4] = "abc";

asm (
  "ld __tmp_reg__, X \n"

  : : "x" (src)
);
```

## Fetch Me Z Pointer

Here is an example directly out of the AVR Inline Assembler Cookbook[28] involving a true C-pointer. In this code snippet, *ptr* is a pointer to variable number. The 'e' constraint requests that *ptr* (which is the address of variable number) be loaded into one of the X, Y or Z register pairs, at the assembler's choice.

Then, the value at the "address" inside the pointer register pair (or 0x11) is loaded into the temporary register (__tmp_reg__). It is incremented, and finally stored back through the pointer *ptr* into the variable number. At the completion of this inline code, number = 0x12, and of course, the value of *ptr* hasn't changed.

```
volatile uint8_t number=0x11, *ptr = &number;

asm volatile(
  "ld __tmp_reg__, %a0 \n"
  "inc __tmp_reg__     \n"
  "st %a0, __tmp_reg__ \n"

  : : "e" (ptr) : "memory"
);
```

If you have don't have a good grasp of C pointers, this could be slightly confusing. It might be helpful to examine the assembler code produced to see exactly what is happening here (note the compiler selected the Z register pair for the pointer, *ptr*):

```
0000029E e0.91.00.01  LDS R30, 0x0100 //load address into ptr (0x0102)
```

[28] See http://www.nongnu.org/avr-libc/user-manual/inline_asm.html.

```
000002A0 f0.91.01.01  LDS R31, 0x0101
000002A2 00.80        LDD R0, Z+0     //load number into r0 (0x11)
000002A3 03.94        INC R0          //increment r0 to 0x12
000002A4 00.82        STD Z+0, R0     //store back into number (0x0102)

Address locations:
ptr:    0x0102  uint8_t* @0x0100
p:  0x11    uint8_t  @0x0102
```

## How Long is a String?

Now, onto strings. The following code calculates the length of the string *str*, not including the terminating NUL, or '\0' character. It places the number of characters inside *str* into *len*:

```
const char src[4] = "abc";
volatile uint8_t len;

asm (
  "_loop:                  \n"
    "ld   __tmp_reg__, Z+ \n"
    "tst  __tmp_reg__     \n"
    "brne _loop           \n"
    //Z points one character past the terminating NUL
    "subi %A1, 1          \n" //subtract post-increment
    "sbci %B1, 0          \n"
    "sub  %A1, %A2        \n" //length = end - start
    "sbc  %B1, %B2        \n"
    "mov  %0, %A1         \n" //save len (uint8_t)

    : "=r" (len) : "z" (src), "x" (src) : "memory"
);
```

First, notice we define input constraints for the string, *str* twice, using both X and Z pairs. These constraints place the address of the string inside of the *r30:r31* and *r26:r27* register pairs. The reason for this will become clear in a moment.

Studying the code further, notice we load the first character of the string (pointed to by the "Z" register), placing it into the temp register (__tmp_reg__). Further, take note that the instruction has a plus sign '+' appended to the 'Z'. This means the Z register is incremented by 1 after the load operation. It's as if we combine two instructions into one! This is termed "Indirect Addressing with Post-Increment".

Next, the temp register is tested (tst __tmp_reg__), and if it is NOT zero, execution will loop back and fetch another character. This repeats until finding the NUL character at the end of the string. This terminates the loop, however at

this point, because of the post-increment operation, the Z register points one location past the end of the string.

We complete the routine by subtracting 1 for extra post-increment, and then subtract the ending string address from the start address. The result of this math is the length of the string.

Here is a slightly more efficient version, but I will leave it to you to determine the details of the shortened arithmetic (the embedded comment explains the math in cryptic fashion):

```
const char src[4] = "abc";
volatile uint8_t len;

asm (
  "_loop:                \n"
    "ld  __tmp_reg__, Z+ \n"
    "tst __tmp_reg__     \n"
    "brne _loop          \n"
    //len=Z - 1 – src = (-1 - src) + Z = ~src + Z
    "com %A2             \n"
    "com %B2             \n"
    "add %A2, %A1        \n"
    "adc %B2, %B1        \n"

    : "=r" (len) : "z" (src), "x" (src)
);
```

## Zerox a String

Lets do another one. This code copies the *src* string (including the terminating NUL character) to the array pointed to by *dst*. However, the strings may not overlap, and the *dst* string must be large enough to receive the copy. If the destination string is not large enough, anything could happen…

```
const char src[4] = "abc";
char dst[4] = "   ";

asm (
  "_copy:                 \n"
    "ld   __tmp_reg__, Z+ \n"  //load tmp reg w/src char
    "st   X+, __tmp_reg__ \n" //store tmp reg to dst
    "tst  __tmp_reg__     \n" //check if 0 (end)
    "brne _copy           \n"

  : : "x" (dst) , "z" (src)
```

```
);
```

Wow, only 4-lines of inline assembly code can copy a string! As you can see, this is very straight forward and quite simple. It utilizes the X and Z register pairs, incorporating post-increment addressing with both.

## Whose String is Bigger?

This code compares the two strings *s1* and s2. It returns an integer (in result) less than, equal to, or greater than zero if *s1* is found to be less than, to match, or be greater than *s2*. Again, it utilizes the X and Z register pairs, incorporating post-increment addressing with both. Hopefully you are starting to recognize the power of "indirect addressing" combined with "post-indexing".

```
char s1[4] = "abc";
char s2[4] = "xyz";
volatile int16_t result;

asm (
  "_compare:                          \n"
    "ld   %A0, X+                     \n"
    "ld   __tmp_reg__, Z+             \n"
    "sub  %A0, __tmp_reg__            \n"
    "cpse __tmp_reg__, __zero_reg__ \n"
    "breq _compare                    \n"
    "sbc  %B0, %B0                    \n"

    : "=&r" (result) : "x" (s1) , "z" (s2)
);
```

## String Cat

This code appends the *src* string to the *dst* string overwriting the NUL character at the end of *dst*, and then adds a terminating NUL character. The strings may not overlap, and the *dst* string must have enough space for the result. This example is slightly more involved, but with a little study the details should become clear.

```
const char src[4] = "def";
char dst[7] = "abc";

asm (
  "_dst:                   \n" //find end of destination
    "ld   __tmp_reg__, X+ \n"
    "tst  __tmp_reg__     \n"
    "brne _dst            \n"
```

```
    "sbiw %A0, 1          \n" //undo post-increment
  "_src:                  \n" //X==end of dst string
    "ld   __tmp_reg__, Z+ \n"  //copy src to dst
    "st   X+, __tmp_reg__ \n"
    "tst  __tmp_reg__     \n" //test for 0 (end)
    "brne _src            \n"

    : : "x" (dst), "z" (src) : "memory"
);
```

## Charred String?

Finally, this code finds the first occurrence of the character *val* in the string *src*. Here "character" means "byte" (no wide or multi-byte characters allowed). The location of the matched character is placed in a pointer, *c* or a NUL if the character is not found.

```
const char s[4] = "abc", *c;
volatile int16_t val = 0x63;

asm (
  "_loop:          \n"
    "ld   %A0, Z+  \n" //fetch char from string
    "cp   %A0, %A2 \n" //compare char with val
    "breq _found   \n"
    "tst  %A0      \n" //end of string (0)?
    "brne _loop    \n" //not at end
    "clr  %B0      \n" //not found, NULL pointer
    "rjmp _end     \n"
  "_found:         \n"
    "sbiw %A1, 1   \n" //undo post-increment
    "movw %A0, %A1 \n" //save pointer
  "_end:           \n"

  : "=x" (c) : "z" (s), "r" (val)
);
```

# Chapter 14

# Functions

At first consideration, the topic of functions seems simple and trite. Just discuss how to “CALL” and “RETURN” to and from a function, right? However, there are many subtopics involved as well. For example, passing and returning parameters, prologue and epilogue code, the stack frame and mixing assembly and C are topics deserving of separate chapters. Hopefully, we can do all of these justice, but first, the basics…

## Convert Snippet into a Function

How about a simple demonstration of turning an inline code snippet into a function? In a previous chapter on indirect addressing, several inline pieces of code were developed to perform various string operations. One such operation determined the character length of a string. The code is below.

### String Length, Sounds Like strlen

```
const char src[4] = "abc";
volatile uint8_t len;

asm (
  "_loop:                 \n"
    "ld   __tmp_reg__, Z+ \n"
    "tst  __tmp_reg__     \n"
    "brne _loop           \n"
    //Z points one character past the terminating NUL
```

```
    "subi %A1, 1          \n" //subtract post-increment
    "sbci %B1, 0          \n"
    "sub  %A1, %A2        \n" //length = end - start
    "sbc  %B1, %B2        \n"
    "mov  %0, %A1         \n" //save len (uint8_t)

    : "=r" (len) : "z" (src), "x" (src)
);
```

While this code could easily be included "inline", it certainly would be more useful if it was defined as a general function. This would make it much easier to use throughout a program, and also reduce overall program size by incorporating only one instance of the code. So how is this accomplished?

## Stub Your Code

The official Cookbook[29] refers to this techniques as a "C Stub Function," which is nothing more than a function definition containing only inline assembler code. Typically, in a "C Stub Function", the function parameters and local variables define the data used in, and the value returned (if any) by the function. This is an easy method to pass data to/from the inline function, without the need to understand the underlying details of how it is done. And it therefore, eliminates the necessity of writing additional code.

The above "string length" snippet easily becomes a full blown function, *_strlen()* using this method. Notice the transformed function below receives a string, (s) as a parameter, and returns the length, which is defined as a local variable. We refer to these same variables in the input and output constraints:

```
inline uint8_t _strlen(const char *s) {
  uint8_t len;

  asm (
    "_loop:                \n"
      "ld  __tmp_reg__, Z+ \n"
      "tst __tmp_reg__     \n"
      "brne _loop          \n"
      //len=Z - 1 – src = (-1 - src) + Z = ~src + Z
      "com %A2             \n"
      "com %B2             \n"
      "add %A2, %A1        \n"
      "adc %B2, %B1        \n"
```

[29] See http://www.nongnu.org/avr-libc/user-manual/inline_asm.html.

```
      : "=r" (len) : "z" (s), "x" (s)
  );

  return len;
}
```

## Placing a Call

An extension to the "C Stub Function" technique is calling another C function from inside inline assembly code. The following bit of code demonstrates the CALL instruction. This instruction "calls" a subroutine located within the program memory (if we remember to properly define the function to avoid linkage errors). The C Stub Function even handles the return (RET) for us. An additional detail required here, is the need to encapsulate the "called" function inside the extern "C" { } declaration (see below example). The extern "C", C++ keyword prevents the function name from becoming "mangled[30]", thus preventing the linker from locating the called function.

```
extern "C" {
  void foo() {
    // do something here...
  }
}

void test() {
  asm (
    "call foo \n"
  );
}
```

## Playing Catch

Next, we present a basic example of passing and returning parameters to and from C Stub Functions. The purpose of the following code is to convert an upper case ASCII character into its lower case equivalent. We've created two functions here, *_isupper* and *_tolower*, which validate the input character and then perform the conversion.

Take a look at the code below.

Notice, the first thing *_tolower* does is call the function, *_isupper*. Since *_tolower* hasn't done anything yet, the C Stub Function simply hands the input character, *c* the parameter to *_tolower* directly onto the *_isupper* function. Neat!

[30] See https://en.wikipedia.org/wiki/Name_mangling.

Next, *_isupper* checks the character to confirm its actually an upper case character. If so, it returns the character, otherwise it returns a zero. Upon returning to *_tolower*, the next instruction which is executed is "tst r24", a test of the contents of register *r24*. If register #24 (*r24*) is not zero, the character is converted and the function returns.

Again, notice the use of the C++ keyword "extern C { }" here:

```
extern "C" {
  unsigned char _isupper(unsigned char c) {
    //bind variable to a specific register r18
    register unsigned char ch asm("r18");

    asm (
      "mov  %1, %0 \n" //save input
      "subi %1, 'A'\n" //subtract 0x41
      "brmi 2f     \n" //branch if minus
      "subi %1, 26 \n" //26 letters
      "brpl 2f     \n" //branch if plus
      "ret         \n" //c==upper, return
    "2:            \n"
      "clr  %0     \n" //false

      : "+r" (c) : "r" (ch)
    );

    return c;
  }
}

char _tolower(unsigned char c) {
  asm (
    "call _isupper \n" //validate char
    "tst r24       \n" //0 = not alpha char
    "breq 1f       \n" //not alpha char
    "ori %0, 0x20  \n" //make lower
    "1:            \n"

    : "+r" (c)
  );

  return c;
}
```

## Insider Information

Why did function *_tolower* choose to test register #24 (*r24*)? Because the above two functions relied on "insider" information when using register *r24*. These routines knew that an 8-bit, byte-sized value is passed to and from a function via the *r24* register. The C Compiler always passes function arguments and returns values in specific register locations. Knowing these locations is essential to writing efficient inline assembly code, especially when interfacing with the C language.

This is a good time to review the data type sizes: a char is 8 bits, an integer is 16-bits, a long is 32-bits, a long long is 64-bits, floats are 32-bits, and pointers are 16-bits (function pointers are word addresses). Arguments are allocated left to right, starting in register *r25* descending through register *r8*. All arguments are aligned to start in even-numbered registers (odd-sized arguments, like char, have one free register above them), for example, a single 8-bit value is passed via the *r24* register (*r25* is assumed empty), a single 16-bit value is passed via the *r25:r24* register pair, and a 32-bit value would be passed via *r25:r24:r23:r22* register combination.

Return values are expected to be passed in a similar fashion. An 8-bit value is passed via *r24*, a 16-bit value in *r25:r24*, and 32-bits in *r22:r23:r24:r25*. An 8-bit return value may be zero/sign-extended to 16-bits by the called function.

## What's the Use of a Register?

Function "call-used" registers are *r18-r27*, and *r30-r31*. Any, or all of these registers may be allocated by the compiler for local data. However, we may use them freely in assembler subroutines. Calling C subroutines can clobber any of them, and the caller is responsible for saving and restoring before and after use. Function "call-saved" registers are *r2-r17,* and *r28-r29*. They may also be allocated by the compiler for local data, but C subroutines leaves them unchanged. Assembler subroutines are responsible for saving and restoring any of these registers, if changed. The Y register pair (*r29:r28*) is used as a frame pointer (pointing to local data placed on the stack) if necessary.

Fixed registers, *r0*, and *r1* are never allocated by the compiler for local data. The temporary register, r0 can be clobbered by any C code (except interrupt handlers which save it), and may be used freely. The zero register is *r1*, and assumed to be always zero in any C code. It may be used for other purposes within a piece of assembler code, but must then be cleared after use (clr r1). Interrupt handlers save and clear *r1* on entry, and restorer1 on exit (in case it was non-zero).

# Chapter 15

# Interrupts

## Pardon the Interruption

The previous chapter covered the basics of writing inline functions. A close relative of the function is the Interrupt Service Routine (ISR), which is the topic here. Portions of this chapter may pertain to functions as well.

As a warning, this chapter assumes an understanding of the basic concepts of interrupts in general, and interrupt handlers on the Arduino (AVR μC) specifically. Hopefully, you have already written a few Arduino interrupts in C, using the internal Arduino functionality[31]. If not, you may want to study some of the links given in the reference section of this chapter before continuing.

## The Deck is Stacked

Basic knowledge of the stack is essential to understanding functions and interrupt handlers. The basic purpose of the stack is to support function calls and interrupts. Whenever a program makes a function call or whenever an interrupt occurs, the stack is used to store critical information which will be restored upon completion of the function or interrupt.

First and primary, during a function call or interrupt, the hardware places the return address on the stack. The saving and restoration of the return address is accomplished transparently by the CALL and RET (RETI) instructions. It is not necessary to perform any special instruction(s) to make this occur.

[31] See http://playground.arduino.cc/Code/Interrupts.

Second, if any "call-saved" registers will be "clobbered" inside the function, these registers are "pushed" onto the stack. In the case of an interrupt service routine, all of the registers used inside the ISR (and always the temporary and zero registers, r0 and r1) get pushed onto the stack. Additionally, during an ISR the SREG is saved and restored.

Finally, if the compiler deems it necessary, space is reserved for any local variables on the stack. Many times the compiler will place local variables into specific registers, and therefore doesn't use the stack for temporary storage. Here is an example of how the compiler uses the stack to store local variables inside of a function. This is sometimes referred to as "setting up a stack frame." We will reserve 16 bytes for a character array (note: unrelated code has been removed for the purpose of clarity). The compiler performs all of this stack manipulation for us behind the scenes, so-to-speak:

```
void example(void) {
  char buffer[16]; //space will be reserved on the stack

  //
  //do something here. . .
  //

}
```

Result in this machine code:

```
;prologue
  PUSH r28          ;save registers on stack
  PUSH r29
  IN   r28, SPL     ;get stack pointer
  IN   r29, SPH
  SBIW r28, 16      ;reserve 16 bytes space on stack
                    ;the stack grows downward, hence the
subtraction
  OUT  SPH, r29     ;update new stack pointer
  OUT  SPL, r28

;
;do something here. . .
;

;epilogue
  ADIW r28, 16      ;remove the 16 bytes from the stack
  OUT  SPH, r29     ;restore stack pointer
  OUT  SPL, r28
```

```
  POP  r29          ;restore registers from stack
  POP  r28
  RET
}
```

Upon return from the interrupt or function, all the preserved values are restored, or “popped” from the stack. Obviously, during the pro and epilogue code, the order of the push and pop instructions is very critical.

## Interrupt Before and After

Below, I wrote a very basic interrupt routine that simply increments a byte so we can examine the prologue and epilogue code generated by the compiler:

```
//here is an example ISR coded in C:
volatile uint8_t a;

ISR(INT0_vect) {
  a++;
}

//this is the generated assembly code:
;prologue
0000027F 1f.92               PUSH r1       ;save r1 register
00000280 0f.92               PUSH r0       ;save r0 register
00000281 0f.b6               IN r0, SREG   ;get status register
00000282 0f.92               PUSH r0       ;save sreg
00000283 11.24               CLR r1
00000284 8f.93               PUSH r24      ;save r24 register

;increment byte (a) here
00000285 80.91.c3.01         LDS r24, (a)
00000287 8f.5f               SUBI r24, 0xFF
00000288 80.93.c3.01         STS (a), r24

;epilogue
0000028A 8f.91               POP r24       ;restore r24 register
0000028B 0f.90               POP r0        ;restore sreg
0000028C 0f.be               OUT SREG, r0
0000028D 0f.90               POP r0        ;restore r0 register
0000028E 1f.90               POP r1        ;restore r1 register
0000028F 18.95               RETI          ;ret from interrupt
```

As you can see, the meat of the ISR is only 10 bytes long. However, together the prologue and epilogue add another 24 bytes, for a total of 34. It might be possible

to save a few bytes and program cycles by tightly writing your own ISR pro and epilogue. GCC has a provision which allows writing your own pro and epilogues, which will be covered later.

## We Interrupt This Program to Blink

It is now time to write an interrupt handler, or ISR in inline assembler. I can't think of a better example than to adapt the basic Blink sketch to use the Timer #1 Overflow interrupt. Please note, because this code alters the Timer #1 registers, it will render any use of the Arduino Timer #1 as nonfunctional (i.e. analogWrite pins 9 & 10, the Servo Library, etc.).

## Handle It

The first order of business is to write the interrupt handler for the Timer #1 Overflow. This is the routine that is called when the Timer #1 counter (TCNT1) rolls over from 0xffff to zero. Our the ISR is very basic, and as always, it should be kept as short as possible. Inside the handler we perform two functions:
Reset the counter (TCNT1) allowing the next overflow to reoccur at 1 second intervals.

## Toggle the LED.

An ISR can be coded using inline assembler just as in a "C Stub Function", relying upon the compiler to insert the necessary prologue and epilogue code. I suggest you use this stub technique at first before graduating to writing the entire "naked" ISR. Here is a stub version of our ISR:

```
#define TCNT_BASE   0x0bdc
#define TCNT_BASE_H (((TCNT_BASE)>>8)&0xff)
#define TCNT_BASE_L ((TCNT_BASE)&0xff)

ISR(TIMER1_OVF_vect) {
  asm (
    //reload TCNT1 counter for 1sec interrupt
    "ldi r24, %3           \n"
    "st  Z+, r24           \n" //TCNT1L
    "ldi r24, %4           \n"
    "st  Z, r24            \n" //TCNT1H
    //toggle LED
    "in   __tmp_reg__, %0  \n" //read port
    "ldi  r24, %1          \n" //LED bit mask
    "eor  __tmp_reg__, r24 \n" //toggle LED bit
    "out  %0, __tmp_reg__  \n" //write port
```

```
    : : "I" (_SFR_IO_ADDR(PORTB)),
    "I" (_BV(PORTB5)),
    "z" (_SFR_MEM_ADDR(TCNT1)),
    "M" (TCNT_BASE_L),
    "M" (TCNT_BASE_H)
    : "r24"
  );
}
```

Having said all that, the boilerplate code the compiler inserts is not always the most efficient, and many times inadequate. For these reasons, and for the academic exercise, we will also select the “ISR_NAKED” attribute when defining the ISR. This gives us full control over all of the code inside the ISR. Full control is a good thing:

```
ISR(TIMER1_OVF_vect, ISR_NAKED)
```

Eleven instructions encompass the prologue and epilogue, which is more than the code required for the main purpose of the interrupt. Notice inside the handler, we utilize 3 registers, r24, r30 and r31. This means we need to preserve the content of these registers since the interrupt could be triggered at any time, even precisely when these registers may be in use. Additionally we need to preserve the status register (SREG). The SREG holds critical information on the state of the program when the interrupt fired. Neglecting to reserve any of this information would probably cause the program to crash.

Don’t forget to include the terminating RETI instruction also. By comparison, this ISR_NAKED version is 10 bytes shorter than the “Stub” version:

#include "k328p.h"

```
#define TCNT_BASE   0x0bdc
#define TCNT_BASE_H (((TCNT_BASE)>>8)&0xff)
#define TCNT_BASE_L ((TCNT_BASE)&0xff)

ISR(TIMER1_OVF_vect, ISR_NAKED) {
  asm (
    "push r31           \n" //save r30, r31 contents
    "push r30           \n"
    "push r24           \n"
    //preserve SREG
    "in   r24, __SREG__ \n"
    "push r24           \n"

    //reload TCNT1 counter for 1sec interrupt
    "clr r31            \n"
```

```
    "ldi r30, %2        \n"
    "ldi r24, %3        \n"
    "st  Z+, r24        \n" //TCNT1L
    "ldi r24, %4        \n"
    "st  Z, r24         \n" //TCNT1H
    //toggle LED
    "in   r30, %0       \n" //read port
    "ldi  r31, %1       \n" //LED bit mask
    "eor  r30, r31      \n" //toggle LED bit
    "out  %0, r30       \n" //write port

    //restore old SREG
    "pop  r24           \n"
    "out  __SREG__, r24 \n"
    //restore r30, r31
    "pop r24            \n"
    "pop  r30           \n"
    "pop  r31           \n"
    "reti               \n"

    : : "I" (kPORTB), "I" (_BV(PORTB5)),
    "M" (kTCNT1), "M" (TCNT_BASE_L), "M" (TCNT_BASE_H)
  );
}
```

The initiation code required for the Timer #1 interrupt (setting the prescaler, loading the counter and enabling the overflow interrupt) is completely contained inside the Setup function. Obviously, it is not necessary to write this in inline assembly, it's just good practice:

```
#include "k328p.h"

#define TCNT_BASE   0x0bdc
#define TCNT_BASE_H (((TCNT_BASE)>>8)&0xff)
#define TCNT_BASE_L ((TCNT_BASE)&0xff)

void setup() {
  uint16_t TNCTBase = TCNT_BASE;

  asm (
    "cli                \n" //disable gloal interrupts
    "sbi %0, %1         \n" //pinMode(13, OUTPUT);

    //set 256 prescale (CS12)
    "st  Z+, __zero_reg__ \n" //TCCR1A
    "ldi r24, %3          \n"
```

```
    "st  Z+, r24          \n" //zero TCCR1B
    "st  Z, __zero_reg__  \n" //zero TCCR1C
    //load counter for 1sec interrupt
    "ldi r30, %4          \n"
    "st  Z+, %A5          \n" //TCNT1L
    "st  Z, %B5           \n" //TCNT1H
    //enable overflow interrupt
    "ldi r30, %6          \n"
    "ldi r24, %7          \n"
    "st  Z, r24           \n" //TIMSK1
    "sei                  \n" //enable global interrupts

    : : "I" (_SFR_IO_ADDR(DDRB)), "I" (PORTB5),
    "z" (_SFR_MEM_ADDR(TCCR1A)), "I" (_BV(CS12)),
    "M" (kTCNT1), "r" (TNCTBase),
    "M" (kTIMSK1), "I" (_BV(TOIE1)) : "r24", "memory"
  );
}

void loop() { }
```

Finally, we are introducing a new header file “k328p.h” (contents listed below) which contains all of the IO register defines in such a way that we can use them inside our inline assembly routines. The definitions in this file use the same standard ATMEL mnemonics for the IO registers with the letter ‘k’ pre-pended. They are the LSB of the IO register address, and allow greater flexibility in inline assembler code when referring to the IO registers (when using pointer registers with the LD/ST instructions). A close examination of the above code will reveal the method of use.

## Arduino IO Register Defines

```
//k328p.h - definitions for ATmega328P
//4.4.2016
#ifndef _k328P_H_
#define _k328P_H_

//standard registers
//0-0x1f: bit addressable
//0-0x3f: IN/OUT compatible
//0-0x3f: add 0x20 when using LD/ST
#define kPINB   0x03
#define kDDRB   0x04
#define kPORTB  0x05
#define kPINC   0x06
#define kDDRC   0x07
```

```
#define kPORTC  0x08
#define kPIND   0x09
#define kDDRD   0x0A
#define kPORTD  0x0B

#define kTIFR0  0x15
#define kTIFR1  0x16
#define kTIFR2  0x17

#define kPCIFR  0x1B
#define kEIFR   0x1C
#define kEIMSK  0x1D
#define kGPIOR0 0x1E
#define kEECR   0x1F
//end bit addressable

#define kEEDR   0x20
#define kEEAR   0x21
#define kEEARL  0x21
#define kEEARH  0x22
#define kGTCCR  0x23
#define kTCCR0A 0x24
#define kTCCR0B 0x25
#define kTCNT0  0x26
#define kOCR0A  0x27
#define kOCR0B  0x28

#define kGPIOR1 0x2A
#define kGPIOR2 0x2B
#define kSPCR   0x2C
#define kSPSR   0x2D
#define kSPDR   0x2E

#define kACSR   0x30

#define kMCUSR  0x34
#define kMCUCR  0x35

#define kSPMCSR 0x37

#define kSPL    0x3D
#define kSPH    0x3E
#define kSREG   0x3F
//end IN/OUT compatible

//extended registers begin
```

```
#define kWDTCSR 0x60
#define kCLKPR  0x61

#define kPRR    0x64

#define kOSCCAL 0x66

#define kPCICR  0x68
#define kEICRA  0x69

#define kPCMSK0 0x6B
#define kPCMSK1 0x6C
#define kPCMSK2 0x6D
#define kTIMSK0 0x6E
#define kTIMSK1 0x6F
#define kTIMSK2 0x70

#define kADC    0x78
#define kADCW   0x78
#define kADCL   0x78
#define kADCH   0x79
#define kADCSRA 0x7A
#define kADCSRB 0x7B
#define kADMUX  0x7C

#define kDIDR0  0x7E
#define kDIDR1  0x7F

#define kTCCR1A 0x80
#define kTCCR1B 0x81
#define kTCCR1C 0x82

#define kTCNT1  0x84
#define kTCNT1L 0x84
#define kTCNT1H 0x85
#define kICR1   0x86
#define kICR1L  0x86
#define kICR1H  0x87
#define kOCR1A  0x88
#define kOCR1AL 0x88
#define kOCR1AH 0x89
#define kOCR1B  0x8A
#define kOCR1BL 0x8A
#define kOCR1BH 0x8B

#define kTCCR2A 0xB0
```

```
#define kTCCR2B 0xB1
#define kTCNT2  0xB2
#define kOCR2A  0xB3
#define kOCR2B  0xB4
#define kASSR   0xB6

#define kTWBR   0xB8
#define kTWSR   0xB9
#define kTWAR   0xBA
#define kTWDR   0xBB
#define kTWCR   0xBC
#define kTWAMR  0xBD

#define kUCSR0A 0xC0
#define kUCSR0B 0xC1
#define kUCSR0C 0xC2

#define kUBRR0  0xC4
#define kUBRR0L 0xC4
#define kUBRR0H 0xC5
#define kUDR0   0xC6
//end extended registers

//0-0x3f for LD/ST instructions
#define k2PINB   0x23
#define k2DDRB   0x24
#define k2PORTB  0x25
#define k2PINC   0x26
#define k2DDRC   0x27
#define k2PORTC  0x28
#define k2PIND   0x29
#define k2DDRD   0x2A
#define k2PORTD  0x2B
#define k2TIFR0  0x35
#define k2TIFR1  0x36
#define k2TIFR2  0x37
#define k2PCIFR  0x3B
#define k2EIFR   0x3C
#define k2EIMSK  0x3D
#define k2GPIOR0 0x3E
#define k2EECR   0x3F
#define k2EEDR   0x40
#define k2EEAR   0x41
#define k2EEARL  0x41
#define k2EEARH  0x42
#define k2GTCCR  0x43
```

```
#define k2TCCR0A 0x44
#define k2TCCR0B 0x45
#define k2TCNT0  0x46
#define k2OCR0A  0x47
#define k2OCR0B  0x48
#define k2GPIOR1 0x4A
#define k2GPIOR2 0x4B
#define k2SPCR   0x4C
#define k2SPSR   0x4D
#define k2SPDR   0x4E
#define k2ACSR   0x50
#define k2MCUSR  0x54
#define k2MCUCR  0x55
#define k2SPMCSR 0x57
#define k2SPL     0x5D
#define k2SPH     0x5E
#define k2SREG    0x5F

#endif //_k328P_H_
```

# Chapter 16

# Tables

Often, the fastest way to compute something on an Arduino is to not compute it all.

Huh?

For example, trigonometric functions are costly operations and can abruptly slow your application to the pace of a crawl. And many times, the result is computed with far more precision than needed for the situation. Most often you just want the periodic wave-like characteristics of sine or cosine, which can easily be approximated. With a trigonometric function, it's easy to substitute a lookup-table[32] populated with pre-computed values at discrete steps. If your program can handle the loss of precision, yet requires as much speed as possible, this alternative is a good option.

## The Ivy League Microcontroller

Since the Arduino's ATMEL AVR μC is based upon the modified Harvard architecture[33], the data and program instructions are stored in different memory. The program instructions are stored in flash, while data is stored in SRAM. These separate pathways are primarily implemented to enhance performance, but it also prohibits executing program instructions from data memory. Yet it may seem paradoxical, data is allowed to be stored inside program memory (using

[32] See https://en.wikipedia.org/wiki/Lookup_table.
[33] See https://en.wikipedia.org/wiki/Modified_Harvard_architecture.

the PROGMEM[34] attribute).

Placing a table in SRAM is simple, and shouldn't present problems for an inline programmer (especially at our stage!). Consequentially, in this chapter, we will store a table inside program memory.

## Did He Say Frogmen?

Placing the table into program memory is easy. It is accomplished via a C language floating-point array, incorporating the special keyword, "PROGMEM". PROGMEM instructs the compiler to place this data into flash memory:

```
static const float PROGMEM SineTable[91] = {
  0.0, 0.017452, 0.034899, 0.052336, 0.069756,
  . . .
  0.997564, 0.998630, 0.999391, 0.999848, 1.0
};
```

Previously, when accessing SRAM (data memory) we used the LDSinstruction. However, accessing program memory requires the use of the LPM instruction. LPM is the mnemonic for *L*oad from *P*rogram *M*emory, and it loads a data byte from flash program memory into a register.

## Details

The Flash program memory is organized as 16 bits words, while the registers and SRAM are organized as eight bits bytes. The Z-register is used to access the program memory. This 16 bits register pair is used as a 16 bit pointer to the Program memory. The 15 most significant bits selects the word address in Program memory. Because of this, the word address is multiplied by two before it is put in the Z-register. However, the good news is that in the code presented below all of these details are transparent.

## Table Legs

The function below first limits the input value to a range between 0-90. If the input is out-of-range, it returns the floating-point Not-A-Number (*NAN*) value. It then multiplies the input by 4 to produce an index into our table. We multiply by four because our table is populated with floating point numbers, each of which is 4-bytes long. The index is simply added to the (PROGMEM) address of the start of the table. The functions finishes by retrieving the 4-byte float value and returning.

[34] See http://deans-avr-tutorials.googlecode.com/svn/trunk/Progmem/Output/Progmem.pdf.

Note, floating point support inside the inline assembler is scarce. In this function we treat the float variable transparently, like any 32-bit variable. We get away with this because we're not performing any operation on the value.

```
float _Sine(uint16_t angle) {
  float tmp;

  asm (
    //validate angle >= 0 && angle <= 90
    "cpi  %A1, 90+1 \n"
    "cpc  %B1, __zero_reg__ \n"
    "brcc _NaN      \n" //out of range

     //calculate table index
    "lsl  %A1       \n" //float is 4 bytes wide
    "rol  %B1       \n" //index = angle * 4
    "lsl  %A1       \n"
    "rol  %B1       \n"

    //add index to start of SineTable
    "add  r30, %A1  \n"
    "adc  r31, %B1  \n"

    //get sine value (4-bytes)
    "lpm  %A0, Z+   \n"
    "lpm  %B0, Z+   \n"
    "lpm  %C0, Z+   \n"
    "lpm  %D0, Z    \n"
    "ret            \n" //exit

    //return NAN
    "_NaN:              \n"
    "ldi  %A0, lo8(%3)  \n" //NAN = 0x7fc00000
    "ldi  %B0, hi8(%3)  \n"
    "ldi  %C0, hlo8(%3) \n"
    "ldi  %D0, hhi8(%3) \n"

    : "=r" (tmp) : "r" (angle), "z" (SineTable), "F" (NAN)
  );

  return tmp;
}
```

## The Full Table

```
#include <avr/pgmspace.h>

//max errror ~0.017452 [91*4=364 bytes]
static const float PROGMEM SineTable[91] = {
  0.0, 0.017452, 0.034899, 0.052336, 0.069756, 0.087156,
  0.104528, 0.121869, 0.139173, 0.156434, 0.173648, 0.190809,
  0.207912, 0.224951, 0.241922, 0.258819, 0.275637, 0.292372,
  0.309017, 0.325568, 0.34202, 0.358368, 0.374607, 0.390731,
  0.406737, 0.422618, 0.438371, 0.45399, 0.469472, 0.48481,
  0.5, 0.515038, 0.529919, 0.544639, 0.559193, 0.573576,
  0.587785, 0.601815, 0.615661, 0.62932, 0.642788, 0.656059,
  0.669131, 0.681998, 0.694658, 0.707107, 0.71934, 0.731354,
  0.743145, 0.75471, 0.766044, 0.777146, 0.788011, 0.798636,
  0.809017, 0.819152, 0.829038, 0.838671, 0.848048, 0.857167,
  0.866025, 0.87462, 0.882948, 0.891007, 0.898794, 0.906308,
  0.913545, 0.920505, 0.927184, 0.93358, 0.939693, 0.945519,
  0.951057, 0.956305, 0.961262, 0.965926, 0.970296, 0.97437,
  0.978148, 0.981627, 0.984808, 0.987688, 0.990268, 0.992546,
  0.994522, 0.996195, 0.997564, 0.99863, 0.999391, 0.999848, 1.0
};
```

## What Are You Doing For Me?

After a cursory comparison test between the table *_Sine()* function and the Arduino floating point *sin()* function, we can draw some basic conclusions. Even though the table itself consumes 364 (91 x 4 = 364) bytes of flash (on top of the function code), the Arduino library *sin()* function (and it's required peripheral floating point support) uses approximately 900 bytes more flash memory. However, saving space wasn't necessarily the goal of this exercise, speed was the primary concern. Comparing 1,000 calls to both functions yielded an average duration of 121.7uS per sin() vs. 2.92uS for *_Sine()*. One final but obvious concern is the precision of the result. This will need to be evaluated to determine if it is sufficient for your application.

## Bigger, Better, More

Various modifications can expand and improve the accuracy of the table code, but are beyond the scope of this chapter. However, here are some basic ideas. The obvious method is to expand the table to decrease the step interval. Another technique is to incorporate interpolation similar to the following pseudo code:

```
float _iSine(uint16_t angle) {
     uint16_t x1 = floor( angle );
     float y1 = SineTable[x1];
     float y2 = SineTable[x1 + 1];
     return y1 + ( y2 - y1 ) * ( x - x1 )
```

```
}
```

For full 0-360 angle coverage, do something like:

```
float Sine(uint16_t i) {
  while (i > 359)
    i -= 360;

  if (i < 90)
    return iSine(i);
  else if (i < 180)
    return iSine(179 - i);
  else if (i < 270)
    return (-1*iSine(i - 180));
  else if (i < 360)
    return (-1*iSine(359 - i));
}
```

Another easy expansion with the sine table is to calculate cosine and tangent values:

```
float _Cosine(uint16_t a) {
  return _Sine( a + 90 );
}

float _Tangent(uint16_t a) {
  return ( _Sine(a) / _Cosine(a) );
}
```

# Chapter 17

# Examples

As the final chapter in this series, we present four example inline assembly functions and a complete inline program for the Arduino. Specifically, these cover the conversion of a byte to a hexadecimal string, SPI Mode 0 hardware transfer, SPI Mode 0 Bit-banging, the C library *atoi* function, and the example Blink program. Do not take these functions as archetypical examples of high-quality coding practice or brilliantly efficient inline code. They are neither.

Most of the previous examples in this series were simple "snippets of code", and as such gave a myopic view of inline assembly. The goal here is to show complete and working demonstrations of how to include inline assembly into the typical Arduino program. Each example includes explanatory comments covering the key portions of code.

## Stringing Hexadecimals

The following code converts a byte value into a hexadecimal string. Notice at the start of the code, that the constraint #0 value (*val*) is temporarily saved in the *r25* register. The function then converts the first nibble. When the conversion process is complete, the function loops back and converts the second nibble. Note how the code uses the SREG T-bit to flag the first vs. second nibble.

```
void ByteToHexStr(uint8_t val, char *str) {
  asm (
    "set           \n" //flag first nibble
    "mov  r25, %0  \n" //save val
    "swap %0       \n" //swap for correct nibble order
```

```
  "1:              \n"
    "andi %0, 0xf  \n" //mask a nibble
    "cpi  %0, 0xa  \n" //>10?
    "brcc 2f       \n" //yes
    "subi %0, 0xd0 \n" //convert numeral (0-9)
    "rjmp 3f       \n" //skip next

  "2:              \n"
    "subi %0, 0xc9 \n" //convert letter (A-F)

  "3:              \n"
    "st   Z+, %0   \n" //put into string
    "brtc 4f       \n" //upper nibble?
    "clt           \n" //clear nibble flag
    "mov  %0, r25  \n" //get upper nibble
    "rjmp 1b       \n" //repeat conversion

  "4:              \n" //exit

    : : "r" (val), "z" (str) : "memory"
  );
}
```

## I SPI with My Little Eye

Serial Peripheral Interface (SPI)[35] is a synchronous serial data protocol used by microcontrollers for communicating with one or more peripheral devices, or for communication between two microcontrollers. The SPI standard is loose and each device implements it a little differently, which means you must pay close attention to the device's datasheet when implementing the protocol. Generally speaking, there are four modes of transmission, defined by the clock phase and polarity.

Here are two versions of the SPI transfer function. The first of these programs incorporates the Arduino hardware SPI. The second is a bit-bang version using different pins.

## SPI Mode 0 Hardware Transfer

```
static __attribute__ ((noinline)) uint8_t SpiXfer(uint8_t data) {

  asm (
```

[35] See http://www.arduino.cc/en/Reference/SPI.

```
    "out  %1, %0          \n" //put data out SPDR register
    "nop                  \n" //pause

  "1:                     \n"
    “in   __tmp_reg__, %2 \n" //check xmit complete
    "sbrs __tmp_reg__, %3 \n"
    "rjmp 1b              \n"
    "in   %0, %1          \n" //get incoming data

    : "+r" (data) : "M" (_SFR_IO_ADDR(SPDR)),
    "M" (_SFR_IO_ADDR(SPSR)), "I" (SPIF)
  );

  return data;
}
```

## SPI Bit-Bang

```
#define MOSI_PORT  PORTD
#define MOSI_BIT   PORTD5
#define MISO_PORT  PIND
#define MISO_BIT   PIND6
#define CLOCK_PORT PORTD
#define CLOCK_BIT  PORTD7

static uint8_t SpiBitBang(uint8_t data) {
  register uint8_t tmp, i=8;

  //save and restore sreg because t-bit is utilized
  asm (
    "in   __tmp_reg__, __SREG__ \n"

  "1:               \n"
    "sbrs %0, 0x07  \n" //is output data bit high?
    "rjmp 2f        \n" //no
    "sbi  %3, %4    \n" //output a high bit
    "rjmp 3f        \n"

  "2:               \n"
    "cbi  %3, %4    \n" //output a low bit

  "3:               \n"
    "lsl  %0        \n" //shift to next bit
    "in   %1, %5    \n" //get input
    "tst  %1        \n" //anything here?
```

```
    "breq 4f        \n" //nope
    "bst  %1, %6    \n" //set t-bit if input bit is high
    "clr  %1        \n" //zeroize register
    "bld  %1, 0     \n" //set bit 0
    "or   %0, %1    \n" //or low bit with data for return value

  "4:               \n"
    "sbi  %7, %8    \n" //toggle clock bit high
    "nop            \n" //pause
    "cbi  %7, %8    \n" //toggle clock bit low
    "subi %2, 1     \n" //more bits?
    "brne 1b        \n" //do next bit
    "out  __SREG__, __tmp_reg__ \n"

    : "+r" (data), "=&r" (tmp): "a" (i),
    "M" (_SFR_IO_ADDR(MOSI_PORT)), "I" (MOSI_BIT),
    "M" (_SFR_IO_ADDR(MISO_PORT)), "I" (MISO_BIT),
    "M" (_SFR_IO_ADDR(CLOCK_PORT)),  "I" (CLOCK_BIT)
  );

  return data;
}
```

## A Toy

Atoi is a function in the that converts a string into an integer numerical representation (*atoi* stands for ASCII to integer). It is included in the C standard library header file stdlib.h. It is prototyped as follows:

```
int atoi(const char *str);
```

The *str* argument is a string, represented by an array of characters, containing the characters of a signed integer number. The string must be null-terminated. Here is the basic idea of the *atoi* function implemented in C language:

```
int16_t atoi(char s[]) {
  uint8_t i, sign;
  int16_t n;

  //skip white space
  for (i=0; s[i]<=' '; i++);

  //sign
  sign = 0;
  if (s[i] == '-') {
```

```
    sign = 1;
    i++;
  }

  //convert
  for (n=0; s[i]>='0' && s[i]<='9'; i++)
    n = 10*n + s[i] - '0';

  if (sign)
    return (-1*n);
  else
    return n;
}
```

## atoi Inline

Here is our implementation, which is only 64 bytes in length. By comparison, the Arduino AVR libc atoi() function is 76 bytes long. This version is basically functionally equivalent, however there are a few detail differences (this function steps over all leading ASCII characters 0x2F and below, not just whitespace):

```
int16_t _atoi(const char *s) {
#pragma GCC diagnostic push
#pragma GCC diagnostic ignored "-Wuninitialized"
  //sign & c are initialized inside inline asm code
  register uint8_t sign, c;
#pragma GCC diagnostic pop
  //force result into return registers
  register int16_t result asm("r24");

  asm (
    "ldi  %A0, 0x00         \n" //result = 0
    "ldi  %B0, 0x00         \n"

  "1:                       \n"
    "ld   %2, Z+            \n" //fetch char
    "cpi  %2, '-'           \n" //negative sign?
    "brne 2f                \n"
    "ldi  %3, 0x01          \n" //sign = TRUE

  "2:                       \n"
    "cpi  %2, '/' + 1       \n" //step over whitespace/garbage
    "brcc 3f                \n"
    "rjmp 1b                \n"
```

```
  "3:                         \n"
    "rjmp 5f                  \n"

  "4:                         \n"
    "ldi  r23, 10             \n" //result *= 10
    "mul  %B0, r23            \n"
    "mov  %B0, r0             \n"
    "mul  %A0, r23            \n"
    "mov  %A0, r0             \n"
    "add  %B0, r1             \n"
    "clr  __zero_reg__        \n" //r1 trashed by mul
    "add  %A0, %2             \n" //result += new digit
    "adc  %B0, __zero_reg__   \n"
    "ld   %2, Z+              \n" //fetch next digit char

  "5:                         \n"
    "subi %2, '0'             \n" //convert char to 0-9
    "cpi  %2, 10              \n" //end of string?
    "brlo 4b                  \n"

    "cpi  %3, 0               \n" //negative?
    "breq 6f                  \n"
    "com  %B0                 \n" //negate result
    "neg  %A0                 \n"
    "sbci %B0, -1             \n"

  "6:                         \n"
    : "+r" (result) : "z" (s), "a" (c), "a" (sign) : "memory"

  );

  return result;
}
```

## Blink

At less than 490 bytes, this version is less than half the size of the Arduino C language example program. Obviously, there are several ways to make it smaller yet.

```
#define CLOCK_MHZ       16UL
#define DELAY_LENGTH_MS 1000UL
#define DELAY_VALUE     (uint32_t)((CLOCK_MHZ * 1000UL *
DELAY_LENGTH_MS) / 5UL)
```

```
void setup() {
  asm volatile (
    "sbi %0, %1 \n" //pinMode(13, OUTPUT);

    : : "I" (_SFR_IO_ADDR(DDRB)), "I" (DDB5)
  );
}

void loop() {
  asm volatile (
    "mov r18, %D2 \n" //save for second delay iteration
    "mov r20, %C2 \n"
    "mov r21, %B2 \n"

    "sbi %0, %1   \n" //turn LED on

  "1:             \n" //delay ~1 second
    "subi %A2, 1  \n"
    "sbci %B2, 0  \n"
    "sbci %C2, 0  \n"
    "brcc 1b      \n"

    "cbi %0, %1   \n" //turn LED off

  "2:             \n" //delay ~1 second
    "subi r18, 1  \n"
    "sbci r19, 0  \n"
    "sbci r20, 0  \n"
    "brcc 2b      \n"

    : : "I" (_SFR_IO_ADDR(PORTB)),
    "I" (PORTB5),
    "r" (DELAY_VALUE)
    : "r18", "r19", "r20"
  );
}
```

## Conclusion

While there are countless more topics to cover, and many more rabbit-holes to dive down, I believe I have covered enough of the basics in this series. I sure enjoyed researching and writing this book. And, hopefully you gained a few insights into the funky world of Arduino (AVR) inline assembly programming. Now, get inline with your programming!

# Chapter 18

# Assembly Listing

## Obtaining Assembly Language Listing of Compiled Sketch

We will use the very basic blink sketch as an example:

```
void setup() {
  pinMode(13, OUTPUT);
}

void loop() {
  digitalWrite(13, HIGH);   //set the LED on
  delay(1000);              //wait for a second
  digitalWrite(13, LOW);    //set the LED off
  delay(1000);              //wait for a second
}
```

## Step by Step Example:

1. Compile the sketch inside the Arduino IDE.

2. Go to your command prompt and navigate to the temporary compile folder. As an example, the location of my folder was here:

```
C:\Users\XXX\AppData\Local\Temp\build5233078615707877954.tmp
```

Note: Substitute your Windows Login ID for “XXX” in the above example, and the numbers after “build” will vary with each instance of compilation. Look for a folder that begins with “build” which was created most recently inside of the “\Users_your_id_here_\AppData\Local\Temp\” folder.

3. Enter the following command:

```
avr-objdump -S Blink.cpp.elf > list.txt
```

Note: “Blink.cpp.elf” is the name of your sketch with “.cpp.elf” appended to it, and “list.txt” is the name we selected for the file that will contain the assembly listing.

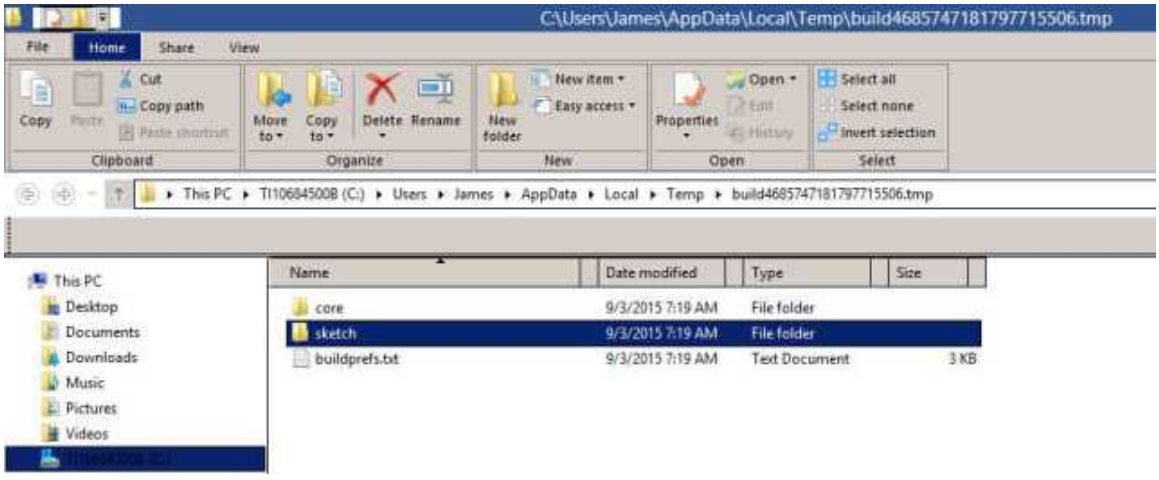
**Note:** The folder location of “avr-objdump.exe” must be in your PATH environment variable for this method to work:

```
\arduino-00xx\hardware\tools\avr\bin
```

**Note:** somewhere along the line of Adruino IDE updates, the temporary file folder structure has changed. While using IDE version 1.6.5, I noticed this new location for the output after compiling the example “blink” program:

```
C:\Users\James\AppData\Local\Temp\build4685747181797715506.tmp\sk
etch\Blink.cpp.elf
```

Here is the new temporary directory structure:




Note the file contents of each of these folders.

The “sketch” folder:

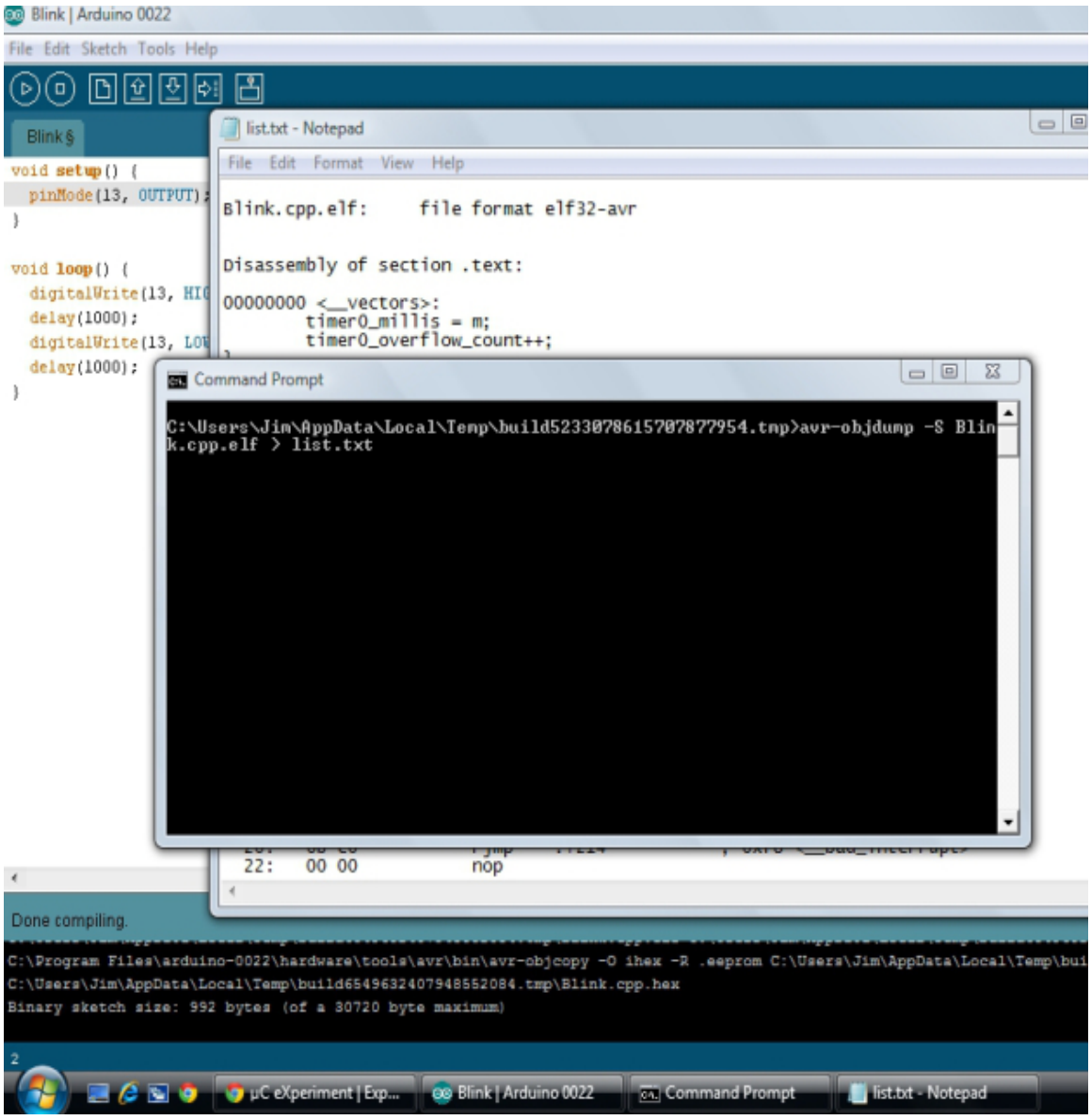
```
09/02/2015 08:12 AM 987     Blink.cpp
09/02/2015 08:12 AM 1,152   Blink.cpp.d
```

```
09/02/2015 08:13 AM 13      Blink.cpp.eep
09/02/2015 08:13 AM 15,539  Blink.cpp.elf
09/02/2015 08:13 AM 2,918   Blink.cpp.hex
09/02/2015 08:12 AM 4,176   Blink.cpp.o
09/02/2015 08:13 AM 4,272   Blink.cpp.with_bootloader.hex
```

The “core” folder:

```
09/02/2015 08:12 AM 166     abi.cpp.d
09/02/2015 08:12 AM 2,996   abi.cpp.o
09/02/2015 08:12 AM 1,062   CDC.cpp.d
09/02/2015 08:12 AM 2,492   CDC.cpp.o
09/02/2015 08:13 AM 397,254 core.a
09/02/2015 08:12 AM 1,282   HardwareSerial.cpp.d
09/02/2015 08:12 AM 28,104  HardwareSerial.cpp.o
09/02/2015 08:12 AM 1,284   HardwareSerial0.cpp.d
09/02/2015 08:12 AM 21,136  HardwareSerial0.cpp.o
09/02/2015 08:12 AM 1,284   HardwareSerial1.cpp.d
09/02/2015 08:12 AM 2,516   HardwareSerial1.cpp.o
09/02/2015 08:12 AM 1,284   HardwareSerial2.cpp.d
09/02/2015 08:12 AM 2,516   HardwareSerial2.cpp.o
09/02/2015 08:12 AM 1,284   HardwareSerial3.cpp.d
09/02/2015 08:12 AM 2,516   HardwareSerial3.cpp.o
09/02/2015 08:12 AM 166     hooks.c.d
09/02/2015 08:12 AM 2,168   hooks.c.o
09/02/2015 08:12 AM 1,342   IPAddress.cpp.d
09/02/2015 08:12 AM 14,324  IPAddress.cpp.o
09/02/2015 08:12 AM 1,152   main.cpp.d
09/02/2015 08:12 AM 4,844   main.cpp.o
09/02/2015 08:12 AM 166     new.cpp.d
09/02/2015 08:12 AM 4,744   new.cpp.o
09/02/2015 08:12 AM 1,173   PluggableUSB.cpp.d
09/02/2015 08:12 AM 2,512   PluggableUSB.cpp.o
09/02/2015 08:12 AM 1,066   Print.cpp.d
09/02/2015 08:12 AM 56,260  Print.cpp.o
09/02/2015 08:12 AM 1,068   Stream.cpp.d
09/02/2015 08:12 AM 43,148  Stream.cpp.o
09/02/2015 08:12 AM 1,064   Tone.cpp.d
09/02/2015 08:12 AM 19,764  Tone.cpp.o
09/02/2015 08:12 AM 1,163   USBCore.cpp.d
09/02/2015 08:12 AM 2,500   USBCore.cpp.o
09/02/2015 08:12 AM 545     WInterrupts.c.d
09/02/2015 08:12 AM 6,768   WInterrupts.c.o
09/02/2015 08:12 AM 535     wiring.c.d
09/02/2015 08:12 AM 8,400   wiring.c.o
09/02/2015 08:12 AM 549     wiring_analog.c.d
```

```
09/02/2015 08:12 AM 6,356   wiring_analog.c.o
09/02/2015 08:12 AM 551     wiring_digital.c.d
09/02/2015 08:12 AM 13,172  wiring_digital.c.o
09/02/2015 08:12 AM 547     wiring_pulse.c.d
09/02/2015 08:12 AM 10,108  wiring_pulse.c.o
09/02/2015 08:12 AM 1,688   wiring_pulse.S.o
09/02/2015 08:12 AM 547     wiring_shift.c.d
09/02/2015 08:12 AM 6,948   wiring_shift.c.o
09/02/2015 08:12 AM 170     WMath.cpp.d
09/02/2015 08:12 AM 8,292   WMath.cpp.o
09/02/2015 08:13 AM 262     WString.cpp.d
09/02/2015 08:13 AM 115,472 WString.cpp.o
```



## Resulting Assembly Listing (Editted):

```
blink.cpp.elf:     file format elf32-avr
Disassembly of section .text:

00000000 <__vectors>:
   2:   00 00           nop
   4:   79 c0           rjmp .+242       ; 0xf8 <__bad_interrupt>
   6:   00 00           nop
   8:   77 c0           rjmp .+238       ; 0xf8 <__bad_interrupt>
   a:   00 00           nop
   c:   75 c0           rjmp .+234       ; 0xf8 <__bad_interrupt>
   e:   00 00           nop
  10:   73 c0           rjmp .+230       ; 0xf8 <__bad_interrupt>
  12:   00 00           nop
  14:   71 c0           rjmp .+226       ; 0xf8 <__bad_interrupt>
  16:   00 00           nop
  18:   6f c0           rjmp .+222       ; 0xf8 <__bad_interrupt>
  1a:   00 00           nop
  1c:   6d c0           rjmp .+218       ; 0xf8 <__bad_interrupt>
  1e:   00 00           nop
  20:   6b c0           rjmp .+214       ; 0xf8 <__bad_interrupt>
  22:   00 00           nop
  24:   69 c0           rjmp .+210       ; 0xf8 <__bad_interrupt>
  26:   00 00           nop
  28:   67 c0           rjmp .+206       ; 0xf8 <__bad_interrupt>
  2a:   00 00           nop
  2c:   65 c0           rjmp .+202       ; 0xf8 <__bad_interrupt>
  2e:   00 00           nop
  30:   63 c0           rjmp .+198       ; 0xf8 <__bad_interrupt>
  32:   00 00           nop
  34:   61 c0           rjmp .+194       ; 0xf8 <__bad_interrupt>
  36:   00 00           nop
  38:   5f c0           rjmp .+190       ; 0xf8 <__bad_interrupt>
  3a:   00 00           nop
  3c:   5d c0           rjmp .+186       ; 0xf8 <__bad_interrupt>
  3e:   00 00           nop
  40:   6f c0           rjmp .+222       ; 0x120 <__vector_16>
  42:   00 00           nop
  44:   59 c0           rjmp .+178       ; 0xf8 <__bad_interrupt>
  46:   00 00           nop
  48:   57 c0           rjmp .+174       ; 0xf8 <__bad_interrupt>
  4a:   00 00           nop
  4c:   55 c0           rjmp .+170       ; 0xf8 <__bad_interrupt>
  4e:   00 00           nop
```

```
  50:   53 c0           rjmp .+166       ; 0xf8 <__bad_interrupt>
  52:   00 00           nop
  54:   51 c0           rjmp .+162       ; 0xf8 <__bad_interrupt>
  56:   00 00           nop
  58:   4f c0           rjmp .+158       ; 0xf8 <__bad_interrupt>
  5a:   00 00           nop
  5c:   4d c0           rjmp .+154       ; 0xf8 <__bad_interrupt>
  5e:   00 00           nop
  60:   4b c0           rjmp .+150       ; 0xf8 <__bad_interrupt>
  62:   00 00           nop
  64:   49 c0           rjmp .+146       ; 0xf8 <__bad_interrupt>
    ...

00000068 <port_to_mode_PGM>:
  68:   00 00 00 00 24 00 27 00 2a 00
...

00000072 <port_to_output_PGM>:
  72:   00 00 00 00 25 00 28 00 2b 00
...

0000007c <port_to_input_PGM>:
  7c:   00 00 00 00 23 00 26 00 29 00
...

00000086 <digital_pin_to_port_PGM>:
  86:   04 04 04 04 04 04 04 04 02 02 02 02 02 02 03 03
  96:   03 03 03 03
...

0000009a <digital_pin_to_bit_mask_PGM>:
  9a:   01 02 04 08 10 20 40 80 01 02 04 08 10 20 01 02

000000ae <digital_pin_to_timer_PGM>:
  ae:   00 00 00 07 00 02 01 00 00 03 04 06 00 00 00 00
  be:   00 00 00 00
...

000000c2 <__ctors_end>:
  c2:   11 24           eor  r1, r1
  c4:   1f be           out  0x3f, r1
  c6:   cf ef           ldi  r28, 0xFF   ;255
  c8:   d8 e0           ldi  r29, 0x08   ;8
  ca:   de bf           out  0x3e, r29   ;62
  cc:   cd bf           out  0x3d, r28   ;61
```

```
000000ce <__do_copy_data>:
  ce:   11 e0           ldi  r17, 0x01   ;1
  d0:   a0 e0           ldi  r26, 0x00   ;0
  d2:   b1 e0           ldi  r27, 0x01   ;1
  d4:   e0 ee           ldi  r30, 0xE0   ;224
  d6:   f3 e0           ldi  r31, 0x03   ;3
  d8:   02 c0           rjmp .+4   ;0xde <.do_copy_data_start>

000000da <.do_copy_data_loop>:
  da:   05 90           lpm  r0, Z+
  dc:   0d 92           st   X+, r0

000000de <.do_copy_data_start>:
  de:   a0 30           cpi  r26, 0x00   ;0
  e0:   b1 07           cpc  r27, r17
  e2:   d9 f7           brne .-10  ; 0xda <do_copy_data_loop>

000000e4 <__do_clear_bss>:
  e4:   11 e0           ldi  r17, 0x01   ;1
  e6:   a0 e0           ldi  r26, 0x00   ;0
  e8:   b1 e0           ldi  r27, 0x01   ;1
  ea:   01 c0           rjmp .+2   ;0xee <.do_clear_bss_start>

000000ec <.do_clear_bss_loop>:
  ec:   1d 92           st   X+, r1

000000ee <.do_clear_bss_start>:
  ee:   a9 30           cpi  r26, 0x09   ;9
  f0:   b1 07           cpc  r27, r17
  f2:   e1 f7           brne .-8   ;0xec <.do_clear_bss_loop>
  f4:   6f d1           rcall .+734   ; 0x3d4 <main>
  f6:   72 c1           rjmp .+740   ; 0x3dc <_exit>

000000f8 <__bad_interrupt>:
  f8:   83 cf           rjmp .-250   ; 0x0 <__vectors>

000000fa <loop>:
  digitalWrite(13, HIGH);   // set the LED on
  fa:   8d e0           ldi  r24, 0x0D   ; 13
  fc:   61 e0           ldi  r22, 0x01   ; 1
  fe:   12 d1           rcall .+548   ; 0x324 <digitalWrite>
  delay(1000);              // wait for a second
 100:   68 ee           ldi  r22, 0xE8   ; 232
 102:   73 e0           ldi  r23, 0x03   ; 3
 104:   80 e0           ldi  r24, 0x00   ; 0
 106:   90 e0           ldi  r25, 0x00   ; 0
```

```
 108:   53 d0           rcall .+166   ; 0x1b0 <delay>
  digitalWrite(13, LOW);    // set the LED off
 10a:   8d e0           ldi  r24, 0x0D   ; 13
 10c:   60 e0           ldi  r22, 0x00   ; 0
 10e:   0a d1           rcall .+532   ; 0x324 <digitalWrite>
  delay(1000);              // wait for a second
 110:   68 ee           ldi  r22, 0xE8   ; 232
 112:   73 e0           ldi  r23, 0x03   ; 3
 114:   80 e0           ldi  r24, 0x00   ; 0
 116:   90 e0           ldi  r25, 0x00   ; 0
 118:   4b c0           rjmp .+150      ; 0x1b0 <delay>

0000011a <setup>:
  pinMode(13, OUTPUT);
 11a:   8d e0           ldi  r24, 0x0D   ; 13
 11c:   61 e0           ldi  r22, 0x01   ; 1
 11e:   dc c0           rjmp .+440   ; 0x2d8 <pinMode>

00000120 <__vector_16>:
 120:   1f 92           push r1
 122:   0f 92           push r0
 124:   0f b6           in   r0, 0x3f    ; 63
 126:   0f 92           push r0
 128:   11 24           eor  r1, r1
 12a:   2f 93           push r18
 12c:   3f 93           push r19
 12e:   8f 93           push r24
 130:   9f 93           push r25
 132:   af 93           push r26
 134:   bf 93           push r27
    unsigned long m = timer0_millis;
 136:   80 91 04 01     lds  r24, 0x0104
 13a:   90 91 05 01     lds  r25, 0x0105
 13e:   a0 91 06 01     lds  r26, 0x0106
 142:   b0 91 07 01     lds  r27, 0x0107
    unsigned char f = timer0_fract;
 146:   30 91 08 01     lds  r19, 0x0108
    m += MILLIS_INC;
 14a:   01 96           adiw r24, 0x01  ; 1
 14c:   a1 1d           adc  r26, r1
 14e:   b1 1d           adc  r27, r1
    f += FRACT_INC;
 150:   23 2f           mov  r18, r19
 152:   2d 5f           subi r18, 0xFD  ; 253
    if (f >= FRACT_MAX) {
 154:   2d 37           cpi  r18, 0x7D   ; 125
```

```
 156:   20 f0           brcs .+8   ; 0x160 <__vector_16+0x40>
        f -= FRACT_MAX;
 158:   2d 57           subi r18, 0x7D  ; 125
        m += 1;
 15a:   01 96           adiw r24, 0x01  ; 1
 15c:   a1 1d           adc  r26, r1
 15e:   b1 1d           adc  r27, r1
    timer0_fract = f;
 160:   20 93 08 01     sts  0x0108, r18
    timer0_millis = m;
 164:   80 93 04 01     sts  0x0104, r24
 168:   90 93 05 01     sts  0x0105, r25
 16c:   a0 93 06 01     sts  0x0106, r26
 170:   b0 93 07 01     sts  0x0107, r27
    timer0_overflow_count++;
 174:   80 91 00 01     lds  r24, 0x0100
 178:   90 91 01 01     lds  r25, 0x0101
 17c:   a0 91 02 01     lds  r26, 0x0102
 180:   b0 91 03 01     lds  r27, 0x0103
 184:   01 96           adiw r24, 0x01   ; 1
 186:   a1 1d           adc  r26, r1
 188:   b1 1d           adc  r27, r1
 18a:   80 93 00 01     sts  0x0100, r24
 18e:   90 93 01 01     sts  0x0101, r25
 192:   a0 93 02 01     sts  0x0102, r26
 196:   b0 93 03 01     sts  0x0103, r27
 19a:   bf 91           pop  r27
 19c:   af 91           pop  r26
 19e:   9f 91           pop  r25
 1a0:   8f 91           pop  r24
 1a2:   3f 91           pop  r19
 1a4:   2f 91           pop  r18
 1a6:   0f 90           pop  r0
 1a8:   0f be           out  0x3f, r0    ; 63
 1aa:   0f 90           pop  r0
 1ac:   1f 90           pop  r1
 1ae:   18 95           reti

000001b0 <delay>:
void delay(unsigned long ms) {
     uint16_t start = (uint16_t)micros();

     while (ms > 0) {
          if (((uint16_t)micros() - start) >= 1000) {
               ms--;
               start += 1000;
```

```
          }
     }
}
 1b0:   9b 01           movw r18, r22
 1b2:   ac 01           movw r20, r24

unsigned long micros() {
    uint8_t oldSREG = SREG, t;
 1b4:   7f b7           in   r23, 0x3f   ; 63
    cli();
 1b6:   f8 94           cli
    m = timer0_overflow_count;
 1b8:   80 91 00 01     lds  r24, 0x0100
 1bc:   90 91 01 01     lds  r25, 0x0101
 1c0:   a0 91 02 01     lds  r26, 0x0102
 1c4:   b0 91 03 01     lds  r27, 0x0103
    t = TCNT0;
 1c8:   66 b5           in   r22, 0x26   ; 38
    if ((TIFR0 & _BV(TOV0)) && (t < 255))
 1ca:   a8 9b           sbis 0x15, 0 ; 21
 1cc:   05 c0           rjmp .+10  ; 0x1d8 <delay+0x28>
 1ce:   6f 3f           cpi  r22, 0xFF   ; 255
 1d0:   19 f0           breq .+6   ; 0x1d8 <delay+0x28>
        m++;
 1d2:   01 96           adiw r24, 0x01   ; 1
 1d4:   a1 1d           adc  r26, r1
 1d6:   b1 1d           adc  r27, r1
    SREG = oldSREG;
 1d8:   7f bf           out  0x3f, r23   ; 63

    uint16_t start = (uint16_t)micros();
     return ((m<<8) + t)*(64/clockCyclesPerMicrosecond());
 1da:   ba 2f           mov  r27, r26
 1dc:   a9 2f           mov  r26, r25
 1de:   98 2f           mov  r25, r24
 1e0:   88 27           eor  r24, r24
 1e2:   86 0f           add  r24, r22
 1e4:   91 1d           adc  r25, r1
 1e6:   a1 1d           adc  r26, r1
 1e8:   b1 1d           adc  r27, r1
 1ea:   62 e0           ldi  r22, 0x02   ; 2
 1ec:   88 0f           add  r24, r24
 1ee:   99 1f           adc  r25, r25
 1f0:   aa 1f           adc  r26, r26
 1f2:   bb 1f           adc  r27, r27
 1f4:   6a 95           dec  r22
```

```
 1f6:   d1 f7           brne .-12  ; 0x1ec <delay+0x3c>
 1f8:   bc 01           movw r22, r24
 1fa:   2d c0           rjmp .+90  ; 0x256 <delay+0xa6>

unsigned long micros() {
    uint8_t oldSREG = SREG, t;
 1fc:   ff b7           in   r31, 0x3f   ; 63
    cli();
 1fe:   f8 94           cli
    m = timer0_overflow_count;
 200:   80 91 00 01     lds  r24, 0x0100
 204:   90 91 01 01     lds  r25, 0x0101
 208:   a0 91 02 01     lds  r26, 0x0102
 20c:   b0 91 03 01     lds  r27, 0x0103
    t = TCNT0;
 210:   e6 b5           in   r30, 0x26   ; 38
    if ((TIFR0 & _BV(TOV0)) && (t < 255))
 212:   a8 9b           sbis 0x15, 0 ; 21
 214:   05 c0           rjmp .+10  ; 0x220 <delay+0x70>
 216:   ef 3f           cpi  r30, 0xFF   ; 255
 218:   19 f0           breq .+6   ; 0x220 <delay+0x70>
        m++;
 21a:   01 96           adiw r24, 0x01   ; 1
 21c:   a1 1d           adc  r26, r1
 21e:   b1 1d           adc  r27, r1
    SREG = oldSREG;
 220:   ff bf           out  0x3f, r31   ; 63
    while (ms > 0) {
        if (((uint16_t)micros() - start) >= 1000) {
     return ((m<<8) + t)*(64/clockCyclesPerMicrosecond());
 222:   ba 2f           mov  r27, r26
 224:   a9 2f           mov  r26, r25
 226:   98 2f           mov  r25, r24
 228:   88 27           eor  r24, r24
 22a:   8e 0f           add  r24, r30
 22c:   91 1d           adc  r25, r1
 22e:   a1 1d           adc  r26, r1
 230:   b1 1d           adc  r27, r1
 232:   e2 e0           ldi  r30, 0x02   ; 2
 234:   88 0f           add  r24, r24
 236:   99 1f           adc  r25, r25
 238:   aa 1f           adc  r26, r26
 23a:   bb 1f           adc  r27, r27
 23c:   ea 95           dec  r30
 23e:   d1 f7           brne .-12  ; 0x234 <delay+0x84>
```

```
 240:   86 1b           sub  r24, r22
 242:   97 0b           sbc  r25, r23
 244:   88 5e           subi r24, 0xE8   ; 232
 246:   93 40           sbci r25, 0x03   ; 3
 248:   c8 f2           brcs .-78  ; 0x1fc <delay+0x4c>
           ms--;
 24a:   21 50           subi r18, 0x01   ; 1
 24c:   30 40           sbci r19, 0x00   ; 0
 24e:   40 40           sbci r20, 0x00   ; 0
 250:   50 40           sbci r21, 0x00   ; 0
           start += 1000;
 252:   68 51           subi r22, 0x18   ; 24
 254:   7c 4f           sbci r23, 0xFC   ; 252
    while (ms > 0) {
 256:   21 15           cp   r18, r1
 258:   31 05           cpc  r19, r1
 25a:   41 05           cpc  r20, r1
 25c:   51 05           cpc  r21, r1
 25e:   71 f6           brne    .-100       ; 0x1fc <delay+0x4c>
 260:   08 95           ret

00000262 <init>:
; code removed
...
2d6:   08 95           ret

000002d8 <pinMode>:
   uint8_t bit = digitalPinToBitMask(pin);
 2d8:   48 2f           mov  r20, r24
 2da:   50 e0           ldi  r21, 0x00   ; 0
 2dc:   ca 01           movw r24, r20
 2de:   86 56           subi r24, 0x66   ; 102
 2e0:   9f 4f           sbci r25, 0xFF   ; 255
 2e2:   fc 01           movw r30, r24
 2e4:   24 91           lpm  r18, Z+
   uint8_t port = digitalPinToPort(pin);
 2e6:   4a 57           subi r20, 0x7A   ; 122
 2e8:   5f 4f           sbci r21, 0xFF   ; 255
 2ea:   fa 01           movw r30, r20
 2ec:   84 91           lpm  r24, Z+
   if (port == NOT_A_PIN) return;
 2ee:   88 23           and  r24, r24
 2f0:   c1 f0           breq .+48  ; 0x322 <pinMode+0x4a>
   reg = portModeRegister(port);
 2f2:   e8 2f           mov  r30, r24
```

```
 2f4:   f0 e0           ldi  r31, 0x00   ; 0
 2f6:   ee 0f           add  r30, r30
 2f8:   ff 1f           adc  r31, r31
 2fa:   e8 59           subi r30, 0x98   ; 152
 2fc:   ff 4f           sbci r31, 0xFF   ; 255
 2fe:   a5 91           lpm  r26, Z+
 300:   b4 91           lpm  r27, Z+
    if (mode == INPUT) {
 302:   66 23           and  r22, r22
 304:   41 f4           brne .+16  ; 0x316 <pinMode+0x3e>
        uint8_t oldSREG = SREG;
 306:   9f b7           in   r25, 0x3f   ; 63
                cli();
 308:   f8 94           cli
        *reg &= ~bit;
 30a:   8c 91           ld   r24, X
 30c:   20 95           com  r18
 30e:   82 23           and  r24, r18
 310:   8c 93           st   X, r24
        SREG = oldSREG;
 312:   9f bf           out  0x3f, r25   ; 63
 314:   08 95           ret
    } else {
        uint8_t oldSREG = SREG;
 316:   9f b7           in   r25, 0x3f   ; 63
                cli();
 318:   f8 94           cli
        *reg |= bit;
 31a:   8c 91           ld   r24, X
 31c:   82 2b           or   r24, r18
 31e:   8c 93           st   X, r24
        SREG = oldSREG;
 320:   9f bf           out  0x3f, r25   ; 63
 322:   08 95           ret

00000324 <digitalWrite>:
    uint8_t timer = digitalPinToTimer(pin);
 324:   48 2f           mov  r20, r24
 326:   50 e0           ldi  r21, 0x00   ; 0
 328:   ca 01           movw r24, r20
 32a:   82 55           subi r24, 0x52   ; 82
 32c:   9f 4f           sbci r25, 0xFF   ; 255
 32e:   fc 01           movw r30, r24
 330:   24 91           lpm  r18, Z+
    uint8_t bit = digitalPinToBitMask(pin);
 332:   ca 01           movw r24, r20
```

```
 334:   86 56           subi r24, 0x66   ; 102
 336:   9f 4f           sbci r25, 0xFF   ; 255
 338:   fc 01           movw r30, r24
 33a:   34 91           lpm  r19, Z+
    uint8_t port = digitalPinToPort(pin);
 33c:   4a 57           subi r20, 0x7A   ; 122
 33e:   5f 4f           sbci r21, 0xFF   ; 255
 340:   fa 01           movw r30, r20
 342:   94 91           lpm  r25, Z+
    if (port == NOT_A_PIN) return;
 344:   99 23           and  r25, r25
 346:   09 f4           brne .+2   ; 0x34a <digitalWrite+0x26>
 348:   44 c0           rjmp .+136 ; 0x3d2 <digitalWrite+0xae>
    if (timer != NOT_ON_TIMER) turnOffPWM(timer);
 34a:   22 23           and  r18, r18
 34c:   51 f1           breq .+84  ; 0x3a2 <digitalWrite+0x7e>
static void turnOffPWM(uint8_t timer)
    switch (timer)
 34e:   23 30           cpi  r18, 0x03   ; 3
 350:   71 f0           breq .+28  ; 0x36e <digitalWrite+0x4a>
 352:   24 30           cpi  r18, 0x04   ; 4
 354:   28 f4           brcc .+10  ; 0x360 <digitalWrite+0x3c>
 356:   21 30           cpi  r18, 0x01   ; 1
 358:   a1 f0           breq .+40  ; 0x382 <digitalWrite+0x5e>
 35a:   22 30           cpi  r18, 0x02   ; 2
 35c:   11 f5           brne .+68  ; 0x3a2 <digitalWrite+0x7e>
 35e:   14 c0           rjmp .+40  ; 0x388 <digitalWrite+0x64>
 360:   26 30           cpi  r18, 0x06   ; 6
 362:   b1 f0           breq .+44  ; 0x390 <digitalWrite+0x6c>
 364:   27 30           cpi  r18, 0x07   ; 7
 366:   c1 f0           breq .+48  ; 0x398 <digitalWrite+0x74>
 368:   24 30           cpi  r18, 0x04   ; 4
 36a:   d9 f4           brne .+54  ; 0x3a2 <digitalWrite+0x7e>
 36c:   04 c0           rjmp .+8   ; 0x376 <digitalWrite+0x52>
        case TIMER1A:   cbi(TCCR1A, COM1A1);    break;
 36e:   80 91 80 00     lds  r24, 0x0080
 372:   8f 77           andi r24, 0x7F   ; 127
 374:   03 c0           rjmp .+6   ; 0x37c <digitalWrite+0x58>
        case TIMER1B:   cbi(TCCR1A, COM1B1);    break;
 376:   80 91 80 00     lds  r24, 0x0080
 37a:   8f 7d           andi r24, 0xDF   ; 223
 37c:   80 93 80 00     sts  000080, r24
 380:   10 c0           rjmp .+32  ; 0x3a2 <digitalWrite+0x7e>
        case  TIMER0A:  cbi(TCCR0A, COM0A1);    break;
 382:   84 b5           in   r24, 0x24   ; 36
 384:   8f 77           andi r24, 0x7F   ; 127
```

```
386:   02 c0           rjmp .+4   ; 0x38c <digitalWrite+0x68>
       case  TIMER0B:  cbi(TCCR0A, COM0B1);    break;
388:   84 b5           in   r24, 0x24   ; 36
38a:   8f 7d           andi r24, 0xDF   ; 223
38c:   84 bd           out  0x24, r24   ; 36
38e:   09 c0           rjmp .+18  ; 0x3a2 <digitalWrite+0x7e>
       case  TIMER2A:  cbi(TCCR2A, COM2A1);    break;
390:   80 91 b0 00     lds  r24, 0x00B0
394:   8f 77           andi r24, 0x7F   ; 127
396:   03 c0           rjmp .+6   ; 0x39e <digitalWrite+0x7a>
       case  TIMER2B:  cbi(TCCR2A, COM2B1);    break;
398:   80 91 b0 00     lds  r24, 0x00B0
39c:   8f 7d           andi r24, 0xDF   ; 223
39e:   80 93 b0 00     sts  0x00B0, r24
   if (timer != NOT_ON_TIMER) turnOffPWM(timer);
   out = portOutputRegister(port);
3a2:   e9 2f           mov  r30, r25
3a4:   f0 e0           ldi  r31, 0x00   ; 0
3a6:   ee 0f           add  r30, r30
3a8:   ff 1f           adc  r31, r31
3aa:   ee 58           subi r30, 0x8E   ; 142
3ac:   ff 4f           sbci r31, 0xFF   ; 255
3ae:   a5 91           lpm  r26, Z+
3b0:   b4 91           lpm  r27, Z+
   if (val == LOW) {
3b2:   66 23           and  r22, r22
3b4:   41 f4           brne .+16  ; 0x3c6 <digitalWrite+0xa2>
       uint8_t oldSREG = SREG;
3b6:   9f b7           in   r25, 0x3f   ; 63
               cli();
3b8:   f8 94           cli
       *out &= ~bit;
3ba:   8c 91           ld   r24, X
3bc:   30 95           com  r19
3be:   83 23           and  r24, r19
3c0:   8c 93           st   X, r24
       SREG = oldSREG;
3c2:   9f bf           out  0x3f, r25   ; 63
3c4:   08 95           ret
   } else {
       uint8_t oldSREG = SREG;
3c6:   9f b7           in   r25, 0x3f   ; 63
               cli();
3c8:   f8 94           cli
       *out |= bit;
3ca:   8c 91           ld  r24, X
```

```
 3cc:   83 2b           or  r24, r19
 3ce:   8c 93           st  X, r24
        SREG = oldSREG;
 3d0:   9f bf           out 0x3f, r25   ; 63
 3d2:   08 95           ret

000003d4 <main>:
    init();
 3d4:   46 df           rcall   .-372       ; 0x262 <init>
    setup();
 3d6:   a1 de           rcall   .-702       ; 0x11a <setup>
    for (;;)
        loop();
 3d8:   90 de           rcall   .-736       ; 0xfa <loop>
 3da:   fe cf           rjmp    .-4         ; 0x3d8 <main+0x4>

000003dc <_exit>:
 3dc:   f8 94           cli

000003de <__stop_program>:
 3de:   ff cf           rjmp    .-2         ; 0x3de
<__stop_program>
```

## avr-objdump Options:

```
At least one of the following switches must be given:

-a: --archive-headers Display archive header information
-f: --file-headers Display contents of the overall file header
-p: --private-headers Display object format file header contents
-h: --[section-]headers Display contents of the section headers
-x: --all-headers Display the contents of all headers
-d: --disassemble Display contents of executable sections
-D: --disassemble-all Display assembler contents of all sections
-S: --source Intermix source code with disassembly
-s: --full-contents Display full contents of sections requested
-g: --debugging Display debug information in object file
-e: --debugging-tags Display debug information using ctags style
-G: --stabs Display (in raw form) any STABS info in the file
-W: --dwarf Display DWARF info in the file
-t: --syms Display the contents of the symbol table(s)
-T: --dynamic-syms Display contents of the dynamic symbol table
-r: --reloc Display the relocation entries in the file
-R: --dynamic-reloc Display dynamic relocation entries in file
@ Read options from
```

```
-v: --version Display this program's version number
-i: --info List object formats and architectures supported
-H: --help Display this information
```

# Chapter 19

# Startup Code

## Who Put This Stuff Here?

Ever wonder what all of the code inserted at the beginning of your program that appears in a disassembly listing is about? Well it's a bunch of required housekeeping routines that are linked into your program at compile time that come from several locations. Some of the files are Arduino specific while others are AVR GCC and libc code. Here is a breakdown of the basic "Blink" program running on a generic ATmega328.

First we see the interrupt vector table (IVT), which is a table of interrupt vectors that associates an interrupt handler with an interrupt request[36].

```
    .macro  vector name
    .if (. - __vectors < _VECTORS_SIZE)
    .weak   \name
    .set    \name, __bad_interrupt
    XJMP    \name
    .endif
    .endm

__vectors:
    XJMP    __ctors_end
```

[36] See http://www.nongnu.org/avr-libc/user-manual/group__avr__interrupts.html.

```
    __vector_1
    vector  __vector_2
    vector  __vector_3
...
    vector  __vector_126
    vector  __vector_127
    .endfunc

/* Handle unexpected interrupts (enabled and no handler), which
   usually indicate a bug.  Jump to the __vector_default function
   if defined by the user, otherwise jump to the reset address.

   This must be in a different section, otherwise the assembler
   will resolve "rjmp" offsets and there will be no relocs.
*/

__bad_interrupt:
    .weak   __vector_default
    .set    __vector_default, __vectors
    XJMP    __vector_default
    .endfunc

__ctors_end:
```

The blink IVT disassembly looks something like this:

```
   0:   0c 94 61 00     jmp 0xc2    ; 0xc2 <__ctors_end>
   4:   0c 94 7e 00     jmp 0xfc    ; 0xfc <__bad_interrupt>
   8:   0c 94 7e 00     jmp 0xfc    ; 0xfc <__bad_interrupt>
...
  3c:   0c 94 7e 00     jmp 0xfc    ; 0xfc <__bad_interrupt>
  40:   0c 94 9d 00     jmp 0x13a   ; 0x13a <__vector_16>
...
```

Note the first vector, or reset vector is set to point to the code that begins at the label "__ctors_end". With the exception of vector #16, the remaining vectors all point to the "bad interrupt code[37]". Vector #16 is the *Timer0 Overflow Vector* and is used by the Arduino millis timer[38].

Next is some program memory used to store data. For example, Arduino's digitalWrite/Read functions use program memory (PROGMEM[39]) for pin/port-mapping.

[37] See https://ucexperiment.wordpress.com/2013/05/19/bad-interrupt/.
[38] See https://ucexperiment.wordpress.com/2012/03/16/examination-of-the-arduino-millis-function/.
[39] See http://www.nongnu.org/avr-libc/user-manual/group__avr__pgmspace.html.

```
00000068 <port_to_mode_PGM>:
  68:   00 00 00 00 24 00 27 00 2a 00
...

00000072 <port_to_output_PGM>:
  72:   00 00 00 00 25 00 28 00 2b 00
...

0000007c <port_to_input_PGM>:
  7c:   00 00 00 00 23 00 26 00 29 00
...

00000086 <digital_pin_to_port_PGM>:
  86:   04 04 04 04 04 04 04 04 02 02 02 02 02 02 03 03
  96:   03 03 03 03
...

0000009a <digital_pin_to_bit_mask_PGM>:
  9a:   01 02 04 08 10 20 40 80 01 02 04 08 10 20 01 02
  aa:   04 08 10 20
...

000000ae <digital_pin_to_timer_PGM>:
  ae:   00 00 00 07 00 02 01 00 00 03 04 06 00 00 00 00
  be:   00 00 00 00
...
```

Next, the *r1* register is zeroed (it's assumed to always be zero throughout the program), and the SREG register is zeroed. Then our stack is initialized. The stack is used primarily to keep track of program execution and passing data to/from functions.

```
clr __zero_reg__
out AVR_STATUS_ADDR, __zero_reg__
ldi r28,lo8(__stack)
ldi r29,hi8(__stack)
out AVR_STACK_POINTER_HI_ADDR, r29
out AVR_STACK_POINTER_LO_ADDR, r28
```

Next is data initialization code. First, the code copies global/static initialized variables into SRAM. Then, if any uninitialized variables exist, a copy BSS section is appended to your code. Finally, if there are any C++ constructor/destructors, a bit of additional code is inserted (but not used here). In the blink disassembly, 11 bytes of memory are copied for the following data: *timer0_millis* (4 bytes), *timer0_overflow_count* (4 bytes), *timer0_fract* (1 byte), and *led* (2 bytes).

After all of this, the startup code ends by calling the “main” function.

```
__do_copy_data:
    ldi  r17, hi8(__data_end)
    ldi  r26, lo8(__data_start)
    ldi  r27, hi8(__data_start)
    ldi  r30, lo8(__data_load_start)
    ldi  r31, hi8(__data_load_start)
    rjmp __do_copy_data_start
__do_copy_data_loop:
    lpm  r0, Z+
    st   X+, r0
__do_copy_data_start:
    cpi  r26, lo8(__data_end)
    cpc  r27, r17
    brne __do_copy_data_loop

/*
__do_clear_bss is only necessary if anything in .bss section.
*/
__do_clear_bss:
    ldi  r17, hi8(__bss_end)
    ldi  r26, lo8(__bss_start)
    ldi  r27, hi8(__bss_start)
    rjmp do_clear_bss_start
.do_clear_bss_loop:
    st   X+, __zero_reg__
.do_clear_bss_start:
    cpi  r26, lo8(__bss_end)
    cpc  r27, r17
    brne do_clear_bss_loop

/*
__do_global_ctors and __do_global_dtors are only necessary if
there are any constructors/destructors.
*/
__do_global_ctors:
    ldi  r17, hi8(__ctors_start)
    ldi  r28, lo8(__ctors_end)
    ldi  r29, hi8(__ctors_end)
    rjmp __do_global_ctors_start
__do_global_ctors_loop:
    sbiw r28, 2
    mov_h r31, r29
    mov_l r30, r28
    XCALL __tablejump__
```

```
__do_global_ctors_start:
    cpi  r28, lo8(__ctors_start)
    cpc  r29, r17
    brne __do_global_ctors_loop

__do_global_dtors:
    ldi  r17, hi8(__dtors_end)
    ldi  r28, lo8(__dtors_start)
    ldi  r29, hi8(__dtors_start)
    rjmp __do_global_dtors_start
__do_global_dtors_loop:
    mov_h r31, r29
    mov_l r30, r28
    XCALL __tablejump__
    adiw r28, 2
__do_global_dtors_start:
    cpi  r28, lo8(__dtors_end)
    cpc  r29, r17
    brne __do_global_dtors_loop
    call <main>
    jmp  <_exit>
```

# Chapter 20

# Cycle Counting

## The Lost Art of Cycle Counting

An excellent demonstration for counting cycles is the exercise of creating an accurate time delay. In the creation of an accurate delay, we need to consider the factors which affect the length of the delay:

1. The operating frequency, which for the typical Arduino is 16MHz.

2. The instruction cycle, which is the time it takes for 1 instruction to complete. Keep in mind, some instructions take longer than 1 cycle, and the branching instructions take a variable amount of time.

3. The number and type of instructions which makeup the delay.

Let's assume we want a 1/2 second delay inserted into our program (at least as close as we can get given crystal accuracy). We could simply rely on the built-in delay function(s), or we could roll our own. However, let's write our own delay function and examine the assembly code with the goal of determining how long it takes to execute.

Starting with a main loop of 5 machine cycles, we can easily determine it would need to execute in a loop of 1,600,000 times to give us a 1/2 second period. Yikes!

Since 1,600,000 is far too large a value to fit into one 8-bit register for loop counting, we'll nest several loops:

```
32 x 200 x 250 = 1,600,000
```

So here's our very basic delay code with the instruction cycles annotated in the comments:

```
asm volatile (
  "ldi r20, 32  \n"  //1 machine cycle
                     //outer loop
"1:             \n"
  "ldi r21, 200 \n"  //1
                     //middle loop
"2:             \n"
  "ldi r22, 250 \n"  //1
                     //inner loop
"3:             \n"
  "nop          \n"  //1
  "nop          \n"  //1
  "dec r22      \n"  //1
  "brne 3b      \n"  //2 on branch
                     //end inner loop
  "dec r21      \n"  //1
  "brne 2b      \n"  //2 on branch
                     //end middle loop
  "dec r20      \n"  //1
  "brne 1b      \n"  //2 on branch
                     //end outer loop
  : : : "r20", "r21", "r22"
);
```

The comments signify the number of machine cycles for each instruction. This data can be found in the Atmel 8-bit AVR Instruction Set document. Our cursory look at the above code shows it should take about 800,000 cycles to execute:

```
5 x 250 x 200 x 32 = 8,000,000 cycles
```

On a 16MHz clock, 8,000,000 cycles is precisely ½ second. However, upon closer examination we discover this is not the case. This delay function takes approximately 0.501 seconds to execute, which is 0.001 seconds longer than expected. Realizing that 0.001 seconds is about 16,000 instruction cycles, where did they come from?

We neglected to account for the cycles in the middle and outer loops!

```
5 x 250 x 200 x 32 = 8,000,000 (inner loop)
    4 x 200 x 32 = 25,600 (middle loop)
         4 x 32 = 128 (outer loop)
8,000,000 + 25,600 + 128 = 8,025,728 cycles
```

That accounts for some missing cycles, but it's still not an exact accounting. It neglects the fact that the BRNE instruction uses a variable amount of cycles depending upon the result of the comparison (2 cycles per branch vs. 1 cycle to fall through).

So here is the nitty-gritty evaluation of our delay:

1. Load the registers (3 cycles)

2. Inner loop:

```
5 x 250 = 1,250 – 1 (fall-through) = 1,249 x 200 x 32 = 7,993,600
```

3. Middle loop:

```
4 x 200 = 800 – 1 = 799 x 32 = 25,568
```

4. Outer loop:

```
4 x 32 = 128 – 1 = 127
```

Total:

```
7,993,600 + 25,568 + 127 + 3 = 8,019,298 total cycles
```

For a 16MHz AVR (like the typical Arduino), a machine cycle is 1/16,000,000 of a second in duration, or 62.5ns, which results in our delay lasting 0.5012 seconds. How does an actual Arduino compare with all this cycle counting? The follow screen capture from a Saleae Logic Analyzer shows our delay lasts 0.5042 seconds. I guess my clock crystal is slightly slow.


We will leave it up to you as an exercise to construct a more accurate ½ second delay function.

# Appendix A

# AVR 8-bit Instruction Set

## Instruction Set Nomenclature

**Status Register (SREG)**

SREG: Status Register
C: Carry Flag
Z: Zero Flag
N: Negative Flag
V: Two’s complement overflow indicator
S: N ⊕ V, for signed tests
H: Half Carry Flag
T: Transfer bit used by BLD and BST instructions
I: Global Interrupt Enable/Disable Flag

**Registers and Operands**

Rd: Destination (and source) register(s)
Rr: Source register
R: Result after instruction is executed
K: Constant data
k: Constant address
b: Bit in the Register File or I/O Register (3-bit)
s: Bit in the Status Register (3-bit)
X, Y, Z: Indirect Address Register
(X=r27:r26, Y=r29:r28 and Z=r31:r30)
A: I/O location address
q: Displacement for direct addressing (6-bit)

## ARITHMETIC AND LOGIC INSTRUCTIONS

| Instr | Oprnds | Description | Operation | Flags | Cycles |
|---|---|---|---|---|---|
| ADD | Rd, Rr | Add without Carry | Rd <- Rd + Rr | Z,C,N,V,H | 1 |
| ADC | Rd, Rr | Add with Carry | Rd <- Rd + Rr + C | Z,C,N,V,H | 1 |
| ADIW | Rd, K | Add Immediate to Word | Rd+1:Rd <- Rd+1:Rd + K | Z,C,N,V | 2 |
| SUB | Rd, Rr | Subtract without Carry | Rd <- Rd - Rr | Z,C,N,V,H | 1 |
| SUBI | Rd, K | Subtract Immediate | Rd <- Rd - K | Z,C,N,V,H | 1 |
| SBC | Rd, Rr | Subtract with Carry | Rd <- Rd - Rr - C | Z,C,N,V,H | 1 |
| SBCI | Rd, K | Subtract Immediate with Carry | Rd <- Rd - K - C | Z,C,N,V,H | 1 |
| SBIW | Rd, K | Subtract Immediate from Word | Rd+1:Rd <- Rd+1:Rd - K | Z,C,N,V | 2 |
| AND | Rd, Rr | Logical AND | Rd <- Rd · Rr | Z,N,V | 1 |
| ANDI | Rd, K | Logical AND with Immediate | Rd <- Rd · K | Z,N,V | 1 |
| OR | Rd, Rr | Logical OR | Rd <- Rd v Rr | Z,N,V | 1 |
| ORI | Rd, K | Logical OR with Immediate | Rd Rd v K | Z,N,V | 1 |
| EOR | Rd, Rr | Exclusive OR | Rd <- Rd (EOR) Rr | Z,N,V | 1 |
| COM | Rd | One's Complement | Rd <- $FF – Rd | Z,C,N,V | 1 |
| NEG | Rd | Two's Complement | Rd <- $00 – Rd | Z,C,N,V,H | 1 |
| SBR | Rd,K | Set Bit(s) in Register | Rd <- Rd v K | Z,N,V | 1 |
| CBR | Rd,K | Clear Bit(s) in Register | Rd <- Rd · ($FFh - K) | Z,N,V | 1 |
| INC | Rd | Increment | Rd <- Rd + 1 | Z,N,V | 1 |
| DEC | Rd | Decrement | Rd <- Rd – 1 | Z,N,V | 1 |
| TST | Rd | Test for Zero or Minus | Rd <- Rd · Rd | Z,N,V | 1 |
| CLR | Rd | Clear Register | Rd <- Rd (EOR) Rd | Z,N,V | 1 |
| SER | Rd | Set Register | Rd <- $FF | None | 1 |
| CP | Rd,Rr | Compare | Rd – Rr | Z,C,N,V,H | 1 |
| CPC | Rd,Rr | Compare with Carry | Rd - Rr – C | Z,C,N,V,H | 1 |
| CPI | Rd,K | Compare with Immediate | Rd – K | Z,C,N,V,H | 1 |

## BRANCH INSTRUCTIONS

| Instr | Oprnds | Description | Operation | Flags | Cycles |
|---|---|---|---|---|---|
| RJMP | K | Relative Jump | PC <- PC + k + 1 | None | 2 |
| IJMP | | Indirect Jump to (Z) | PC <- Z | None | 2 |
| JMP | K | Jump | PC <- k | None | 3 |
| RCALL | K | Relative Call Subroutine | PC <- PC + k + 1 | None | 3 |
| ICALL | | Indirect Call to (Z) | PC <- Z | None | 3 |
| CALL | K | Call Subroutine | PC <- k | None | 4 |
| RET | | Subroutine Return | PC <- STACK | None | 4 |
| RETI | | Interrupt Return | PC <- STACK | I | 4 |
| CPSE | Rd, Rr | Compare, Skip if Equal | if (Rd = Rr) PC <- PC + 2 or 3 | None | 1 / 2 / 3 |
| SBRC | Rr, b | Skip if Bit in Register Cleared | if (Rr(b)=0) PC <- PC + 2 or 3 | None | 1 / 2 / 3 |
| SBRS | Rr, b | Skip if Bit in Register Set | if (Rr(b)=1) PC <- PC + 2 or 3 | None | 1 / 2 / 3 |
| SBIC | P, b | Skip if Bit in I/O Reg. Cleared | if(I/O(P,b)=0) PC <- PC + 2 or 3 | None | 1 / 2 / 3 |
| SBIS | P, b | Skip if Bit in I/O Register Set | If(I/O(P,b)=1) PC <- PC + 2 or 3 | None | 1 / 2 / 3 |
| BRBS | s, k | Branch if Status Flag Set | if (SREG(s) = 1) then PC <- PC+k + 1 | None | 1 / 2 |
| BRBC | s, k | Branch if Status Flag Cleared | if (SREG(s) = 0) then PC <- PC+k + 1 | None | 1 / 2 |
| BREQ | K | Branch if Equal | if (Z = 1) then PC <- PC + k + 1 | None | 1 / 2 |
| BRNE | K | Branch if Not Equal | if (Z = 0) then PC <- PC + k + 1 | None | 1 / 2 |
| BRCS | K | Branch if Carry Set | if (C = 1) then PC <- PC + k + 1 | None | 1 / 2 |
| BRCC | K | Branch if Carry Cleared | if (C = 0) then PC <- PC + k + 1 | None | 1 / 2 |
| BRSH | K | Branch if Same or Higher | if (C = 0) then PC PC + k + 1 | None | 1 / 2 |
| BRLO | K | Branch if Lower | if (C = 1) then PC <- PC + k + 1 | None | 1 / 2 |

| | | | | | |
|---|---|---|---|---|---|
| BRMI | K | Branch if Minus | if (N = 1) then PC <- PC + k + 1 | None | 1 / 2 |
| BRPL | K | Branch if Plus | if (N = 0) then PC <- PC + k + 1 | None | 1 / 2 |
| BRGE | K | Branch if >=, Signed | if (N (EOR) V = 0) then PC <- PC+ k + 1 | None | 1 / 2 |
| BRLT | K | Branch if <=, Signed | if (N (EOR) V = 1) then PC <- PC + k + 1 | None | 1 / 2 |
| BRHS | K | Branch if Half Carry Flag Set | if (H = 1) then PC <- PC + k + 1 | None | 1 / 2 |
| BRHC | K | Branch if Half Carry Flag Clear | if (H = 0) then PC <- PC + k + 1 | None | 1 / 2 |
| BRTS | K | Branch if T Flag Set | if (T = 1) then PC<- PC + k + 1 | None | 1 / 2 |
| BRTC | K | Branch if T Flag Cleared | if (T = 0) then PC<- PC + k + 1 | None | 1 / 2 |
| BRVS | K | Branch if Overflow Flag is Set | if (V = 1) then PC <- PC + k + 1 | None | 1 / 2 |
| BRVC | K | Branch if Overflow Flag Clear | if (V = 0) then PC <- PC + k + 1 | None | 1 / 2 |
| BRIE | k | Branch if Interrupt Enabled | if ( I = 1) then PC <- PC + k + 1 | None | 1 / 2 |
| BRID | k | Branch if Interrupt Disabled | if ( I = 0) then PC <- PC + k + 1 | None | 1 / 2 |

## DATA TRANSFER INSTRUCTIONS

| Instr | Operands | Description | Operation | Flags | Cycles |
|---|---|---|---|---|---|
| MOV | Rd, Rr | Copy Register | Rd <- Rr | None | 1 |
| LDI | Rd, K | Load Immediate | Rd <- K | None | 1 |
| LDS | Rd, k | Load Direct from SRAM | Rd <- (k) | None | 3 |
| LD | Rd, X | Load Indirect | Rd <- (X) | None | 2 |
| LD | Rd, X+ | Load Indirect and Post-Increment | Rd <- (X), X <- X + 1 | None | 2 |
| LD | Rd, -X | Load Indirect and Pre-Decrement | X <- X - 1, Rd (X) | None | 2 |
| LD | Rd, Y | Load Indirect | Rd <- (Y) | None | 2 |
| LD | Rd, Y+ | Load Indirect and Post-Increment | Rd <- (Y), Y <- Y + 1 | None | 2 |
| LD | Rd, -Y | Load Indirect and Pre-Decrement | Y <- Y - 1, Rd <- (Y) | None | 2 |
| LDD | Rd, Y+q | Load Indirect with Displacement | Rd <- (Y + q) | None | 2 |
| LD | Rd, Z | Load Indirect | Rd <- (Z) | None | 2 |
| LD | Rd, Z+ | Load Indirect and Post-Increment | Rd <- (Z), Z <- Z+1 | None | 2 |
| LD | Rd, -Z | Load Indirect and Pre-Decrement | Z <- Z – 1, Rd <- (Z) | None | 2 |
| LDD | Rd, Z+q | Load Indirect with Displacement | Rd <- (Z + q) | None | 2 |
| STS | k, Rr | Store Direct to SRAM | Rd <- (k) | None | 3 |
| ST | X, Rr | Store Indirect | (X) <- Rr | None | 2 |
| ST | X+, Rr | Store Indirect and Post-Increment | (X) <- Rr, X <- X + 1 | None | 2 |
| ST | -X, Rr | Store Indirect and Pre-Decrement | X <- X - 1, (X) Rr | None | 2 |
| ST | Y, Rr | Store Indirect | (Y) <- Rr | None | 2 |
| ST | Y+, Rr | Store Indirect and Post-Increment | (Y) <- Rr, Y <- Y + 1 | None | 2 |
| ST | -Y, Rr | Store Indirect and Pre-Decrement | Y <- Y - 1, (Y) <- Rr | None | 2 |
| STD | Y+q, Rr | Store Indirect with Displacement | (Y + q) <- Rr | None | 2 |
| ST | Z, Rr | Store Indirect | (Z) <- Rr | None | 2 |
| ST | Z+, Rr | Store Indirect and Post-Increment | (Z) <- Rr, Z <- Z + 1 | None | 2 |
| ST | -Z, Rr | Store Indirect and Pre-Decrement | Z <- Z – 1, (Z) <- Rr | None | 2 |
| STD | Z+q, Rr | Store Indirect with Displacement | (Z + q) <- Rr | None | 2 |
| LPM | | Load Program Memory | R0 <- (Z) | None | 3 |
| IN | Rd, P | In Port | Rd <- P | None | 1 |
| OUT | P, Rr | Out Port | P <- r | None | 1 |
| PUSH | Rr | Push Register on Stack | STACK <- Rr | None | 2 |
| POP | Rd | Pop Register from Stack | Rd <- STACK | None | 2 |

## BIT AND BIT-TEST INSTRUCTIONS

| Instr | Operands | Description | Operation | Flags | Clock |
|---|---|---|---|---|---|
| LSL | Rd | Logical Shift Left | Rd(n+1) <- Rd(n), Rd(0) <- 0, | Z,C,N,V,H | 1 |

| | | | | | |
|---|---|---|---|---|---|
| | | | C <- Rd(7) | | |
| LSR | Rd | Logical Shift Right | Rd(n) <- Rd(n+1), Rd(7) <- 0, C <- Rd(0) | Z,C,N,V | 1 |
| ROL | Rd | Rotate Left Through Carry | Rd(0) <- C, Rd(n+1) <- Rd(n), C <- Rd(7) | Z,C,N,V,H | 1 |
| ROR | Rd | Rotate Right Through Carry | Rd(7) <- C, Rd(n) <- Rd(n+1), C <- Rd(0) | Z,C,N,V | 1 |
| ASR | Rd | Arithmetic Shift Right | Rd(n) <- Rd(n+1), n = 0..6 | Z,C,N,V | 1 |
| SWAP | Rd | Swap Nibbles | Rd(3..0) <-> Rd(7..4) | None | 1 |
| BSET | s | Flag Set | SREG(s) <- 1 | SREG(s) | 1 |
| BCLR | s | Flag Clear | SREG(s) <- 0 | SREG(s) | 1 |
| SBI | P, b | Set Bit in I/O Register | I/O(P, b) <- 1 | None | 2 |
| CBI | P, b | Clear Bit in I/O Register | I/O(P, b) <- 0 | None | 2 |
| BST | Rr, b | Bit Store from Register to T | T <- Rr(b) | T | 1 |
| BLD | Rd, b | Bit load from T to Register | Rd(b) <- T | None | 1 |
| SEC | | Set Carry | C <- 1 | C | 1 |
| CLC | | Clear Carry | C <- 0 | C | 1 |
| SEN | | Set Negative Flag | N <- 1 | N | 1 |
| CLN | | Clear Negative Flag | N <- 0 | N | 1 |
| SEZ | | Set Zero Flag | Z <- 1 | Z | 1 |
| CLZ | | Clear Zero Flag | Z <- 0 | Z | 1 |
| SEI | | Global Interrupt Enable | I <- 1 | I | 1 |
| CLI | | Global Interrupt Disable | I <- 0 | I | 1 |
| SES | | Set Signed Test Flag | S <- 1 | S | 1 |
| CLS | | Clear Signed Test Flag | S <- 0 | S | 1 |
| SEV | | Set Two's Complement Overflow | V <- 1 | V | 1 |
| CLV | | Clear Two's Complement Overflow | V <- 0 | V | 1 |
| SET | | Set T in SREG | T <- 1 | T | 1 |
| CLT | | Clear T in SREG | T <- 0 | T | 1 |
| SEH | | Set Half Carry Flag in SREG | H <- 1 | H | 1 |
| CLH | | Clear Half Carry Flag in SREG | H <- 0 | H | 1 |
| NOP | | No Operation | None | 1 | |
| SLEEP | | Sleep | (see Sleep specifics) | None | 1 |
| WDR | | Watchdog | Reset (see WDR specifics) | None | 1 |

# Appendix B

# Modifiers and Constraints

## Modifier Characters

‘=’
Means that this operand is written to by this instruction: the previous value is discarded and replaced by new data.

‘+’
Means that this operand is both read and written by the instruction.
When the compiler fixes up the operands to satisfy the constraints, it needs to know which operands are read by the instruction and which are written by it. ‘=’ identifies an operand which is only written; ‘+’ identifies an operand that is both read and written; all other operands are assumed to only be read.

If you specify ‘=’ or ‘+’ in a constraint, you put it in the first character of the constraint string.

‘&’
Means (in a particular alternative) that this operand is an earlyclobber operand, which is written before the instruction is finished using the input operands. Therefore, this operand may not lie in a register that is read by the instruction or as part of any memory address.

‘&’ applies only to the alternative in which it is written. In constraints with multiple alternatives, sometimes one alternative requires ‘&’ while others do not. A operand which is read by the instruction can be tied to an earlyclobber operand if its only use as an input occurs before the early result is written. Adding

alternatives of this form often allows GCC to produce better code when only some of the read operands can be affected by the earlyclobber.

Furthermore, if the earlyclobber operand is also a read/write operand, then that operand is written only after it's used.

'&' does not obviate the need to write '=' or '+'. As earlyclobber operands are always written, a read-only earlyclobber operand is ill-formed and will be rejected by the compiler.

'%'
Declares the instruction to be commutative for this operand and the following operand. This means that the compiler may interchange the two operands if that is the cheapest way to make all operands fit the constraints. '%' applies to all alternatives and must appear as the first character in the constraint. Only read-only operands can use '%'.

GCC can only handle one commutative pair in an asm; if you use more, the compiler may fail. Note that you need not use the modifier if the two alternatives are strictly identical; this would only waste time in the reload pass.

## AVR family Specific Constraints

| Constraint | Use | Range |
|---|---|---|
| a | Simple upper registers | r16 to r23 |
| b | Base pointer registers pairs | y, z |
| d | Upper register | r16 to r31 |
| e | Pointer register pairs | x, y, z |
| l | Lower registers | r0 to r15 |
| n | Immediate integer operand with known numeric value | |
| q | Stack pointer register | SPH:SPL |
| r | Any register | r0 to r31 |
| s | Immediate integer whose value is not an explicit integer | |
| t | Temporary register | r0 |
| w | Special upper register pairs | r24 to r30 |
| x | Pointer register pair X | x (r27:r26) |
| y | Pointer register pair Y | y (r29:r28) |
| z | Pointer register pair Z | Z (r31:r30) |
| F | Immediate floating point number | |
| G | Floating point constant | 0.0 |
| I | 6-bit positive integer constant | 0 to 63 |
| J | 6-bit negative integer constant | -63 to 0 |
| K | Integer constant | 2 |
| L | Integer constant | 0 |
| M | 8-bit integer constant | 0 to 255 |
| N | Integer constant | -1 |
| O | Integer constant | 8, 16, 24 |
| P | Integer constant | 1 |
| Q | Memory address based on Y or Z pointer w/displacement | |
| R | Integer constant | -6 to 5 |
| X | Any operand whatsoever | |

The x register is r27:r26, the y register is r29:r28, and the z register is r31:r30

# Appendix C

# References

**ATmega 168/328 Datasheet**
http://www.atmel.com/images/atmel-8271-8-bit-avr-microcontroller-atmega48a-48pa-88a-88pa-168a-168pa-328-328p_datasheet_complete.pdf

**AVR 8-bit Instruction Set**
http://www.atmel.com/images/atmel-0856-avr-instruction-set-manual.pdf

**AVR-GCC Inline Assembler Cookbook**
http://www.nongnu.org/avr-libc/user-manual/inline_asm.html

**Basic Asm — Assembler Instructions Without Operands**
https://gcc.gnu.org/onlinedocs/gcc/Basic-Asm.html#Basic-Asm

**Extended Asm – Assembler Instructions with C Expression Operands**
https://gcc.gnu.org/onlinedocs/gcc/Extended-Asm.html#Extended-Asm

**Simple Constraints**
https://gcc.gnu.org/onlinedocs/gcc/Simple-Constraints.html#Simple-Constraints

**Machine Specific Constraints**
https://gcc.gnu.org/onlinedocs/gcc/Machine-Constraints.html#Machine-Constraints

**Constraint Modifiers**
https://gcc.gnu.org/onlinedocs/gcc/Modifiers.html#Modifiers

**MACROs and the GCC Preprocessor**
https://gcc.gnu.org/onlinedocs/cpp/Macros.html

**AVRLibc String Functions**
http://www.nongnu.org/avr-libc/user-manual/group__avr__string.html

**Mixing C and Assembly Language**
http://people.ece.cornell.edu/land/courses/ece4760/FinalProjects/s2012/xg46_jy363/xg46_jy363/Reference/Mixing%20C%20and%20assembly%20language%20programs.pdf

**Arduino Interrupts**
http://playground.arduino.cc/Code/Interrupts

**Newbie's Guide to AVR Interrupts**
http://www.github.com/abcminiuser/avr-tutorials/blob/master/Interrupts/Output/Interrupts.pdf?raw=true

**PJRC Guide to Interrupts**
http://www.pjrc.com/teensy/interrupts.html

**AVR Libc Information on Interrupts**
http://www.nongnu.org/avr-libc/user-manual/group__avr__interrupts.html

**University of Maryland, BC, C Programming and Embedded Systems Course, Interrupt Information**
http://www.csee.umbc.edu/~tinoosh/cmpe311/notes/Interrupts.pdf

**ATMEL Application Note 200: Multiply and Divide Routines**

http://www.atmel.com/Images/doc0936.pdf

**ATMEL Application Note 108: Setup and Use of the LPM Instruction**
http://www.atmel.com/images/doc1233.pdf

**Arduino ATmega168/328 Pin Mapping**
https://www.arduino.cc/en/Hacking/PinMapping168

# Appendix D

# Port and Pin Compendium

The following is a compendium of inline assembly functions dealing with ports and pins. Use these at your own risk. These functions have been trimmed of most bounds checking, so they can easily be abused.

## analogWrite

This inline code writes an analog value (in the form of a PWM wave) to a particular pin. After executing, the pin will generate a steady square wave of the specified duty cycle until the next call (or call to digitalRead() or digitalWrite() on the same pin). The frequency of the PWM signal on most pins is approximately 490 Hz. On the Uno and similar boards, pins 5 and 6 have a frequency of approximately 980 Hz. On Arduino boards with the ATmega168/328, this function works on pins 3, 5, 6, 9, 10, and 11. The analogWrite function has nothing to do with the analog pins or the analogRead function.

A pinMode() call is included inside this function, so there is no need to set the pin as an output before executing this code.

This version of AnalogWrite, with no frills saves ~542 bytes over the built-in function:

```
//analogWrite requires a PWM pin
//PWM pin/timer table:
//3:  (TIMER2B) PD3/TCCR2A/COM2B1/OCR2B
//5:  (TIMER0B) PD5/TCCR0A/COM0B1/OCR0B
```

```
//6:  (TIMER0A) PD6/TCCR0A/COM0A1/OCR0A
//9:  (TIMER1A) PB1/TCCR1A/COM1A1/OCR1A
//10: (TIMER1B) PB2/TCCR1A/COM1B1/OCR1B
//11: (TIMER2A) PB3/TCCR2A/COM2A1/OCR2A
//set below 6 defines per above table
#define ANALOG_PORT         PORTB
#define ANALOG_PIN          PORTB3
#define ANALOG_DDR          DDRB
#define TIMER_REG           TCCR2A
#define COMPARE_OUTPUT_MODE COM2A1
#define COMPARE_OUTPUT_REG  OCR2A

volatile uint8_t val = 128; //0-255

  asm (
    "sbi  %0, %1   \n" //DDR set to output (pinMode)

    "cpi  %6, 0    \n" //if full low (0)
    "breq _SetLow  \n"
    "cpi  %6, 0xff \n" //if full high (0xff)
    "brne _SetPWM  \n"

    "sbi  %2, %1   \n" //set high
    "rjmp _SkipPWM \n"

  "_SetLow:        \n"
    "cbi  %2, %1   \n" //set low
    "rjmp _SkipPWM \n"

  "_SetPWM:        \n"
    "ld   r24, X   \n"
    "ori  r24, %3  \n"
    "st   X, r24   \n" //connect pwm pin timer# & channel
    "st   Z, %6    \n" //set pwm duty cycle (val)

  "_SkipPWM:       \n"

    : : "I" (_SFR_IO_ADDR(ANALOG_DDR)), "I" (ANALOG_PIN),
    "I" (_SFR_IO_ADDR(ANALOG_PORT)),
    "M" (_BV(COMPARE_OUTPUT_MODE)),
    "x" (_SFR_MEM_ADDR(TIMER_REG)),
    "z" (_SFR_MEM_ADDR(COMPARE_OUTPUT_REG)), "r" (val)
    : "r24"
  );
```

## analogRead

The Arduino board contains a 6 channel, 10-bit analog to digital converter which is the brains beneath the analogRead function. It maps input voltages between 0 and 5 into integer values between 0 and 1023, thus yielding a resolution between readings of: 5/1024 units or, 0.0049 volts (4.9 mV) per unit. The input range and resolution can be changed through the *ANALOG_V_REF* define. This code reads the value from the specified analog channel (0-7), which correspond to the analog pins (note, do NOT use A0-A7 for the channel number in this code).

While this version of analogRead (*aRead*) saves a few bytes (~50), it also gives the option of changing the speed via the ADC prescaler. However, don't arbitrarily change the prescale without understanding the consequences. ATMEL advises the slowest prescale should be used (*PS128*). A higher speed (smaller prescale) reduces the accuracy of the AD conversion. The Arduino sets the prescale to 128 during initiation, just as the code below does.

```
//Define various ADC prescales
#define PS2   (1<<ADPS0)                                    //8000kHz
ADC clock freq
#define PS4   (1<<ADPS1)                                    //4000kHz
#define PS8   ((1<<ADPS0) | (1<<ADPS1))                     //2000kHz
#define PS16  (1<<ADPS2)                                    //1000kHz
#define PS32  ((1<<ADPS2) | (1<<ADPS0))                     //500kHz
#define PS64  ((1<<ADPS2) | (1<<ADPS1))                     //250kHz
#define PS128 ((1<<ADPS2) | (1<<ADPS1) | (1<<ADPS0)) //125kHz
#define ANALOG_V_REF     DEFAULT //INTERNAL, EXTERNAL, or DEFAULT
#define ADC_PRESCALE     PS128   //PS16, PS32, PS64 or
P128(default)

uint16_t aRead(uint8_t channel) {
  uint16_t result;

  asm (
    "andi %1, 0x07    \n" //force pin==0 thru 7
    "ori  %1, (%6<<6) \n" //(pin | ADC Vref)
    "sts  %2, %1      \n" //set ADMUX

    "lds  r18, %3             \n" //get ADCSRA
    "andi r18, 0xf8           \n" //clear prescale bits
    "ori  r18, ((1<<%5) | %7) \n" //(new prescale | ADSC)
```

```
    "sts  %3, r18             \n" //set ADCSRA

    "_loop:       \n" //loop until ADSC cleared
    "lds  r18, %3 \n"
    "sbrc r18, %5 \n"
    "rjmp _loop   \n"

    "lds  %A0, %4   \n" //result = ADCL
    "lds  %B0, %4+1 \n" //ADCH

    : "=r" (result) : "r" (channel), "M" (_SFR_MEM_ADDR(ADMUX)),
    "M" (_SFR_MEM_ADDR(ADCSRA)), "M" (_SFR_MEM_ADDR(ADCL)),
    "I" (ADSC), "I" (ANALOG_V_REF), "M" (ADC_PRESCALE)
    : "r18"
  );

  return result;
}
```

## pinMode(OUTPUT)

The Arduino pinMode function configures pin behavior. The code presented from here on, has been previously explained.

```
asm (
  "sbi %0, %1 \n" //1=OUTPUT

  : : "I" (_SFR_IO_ADDR(DDRB)), "I" (DDB5)
);
```

## pinMode (INPUT PULLUP)

```
asm (
  "cbi %0, %2 \n"
  "sbi %1, %2 \n"

  : : "I" (_SFR_IO_ADDR(DDRB)), "I" (_SFR_IO_ADDR(PORTB)),
  "I" (DDB5)
);
```

## pinMode (INPUT)

```
asm (
  "cbi %0, %2 \n"
  "cbi %1, %2 \n"

  : : "I" (_SFR_IO_ADDR(DDRB)), "I" (_SFR_IO_ADDR(PORTB)),
  "I" (DDB5)
);
```

## pinMode with Multiple Pins

```
#define PIN_DIRECTION 0b00101000 //PIN 3 & 5 OUTPUT
//#define PIN_DIRECTION (1<<DDB3) | (1<<DDB5)
asm (
  "out %0, %1 \n"

  : : "I" (_SFR_IO_ADDR(DDRB)), "r" (PIN_DIRECTION)
);
```

## digitalWrite HIGH

If a pin has been configured as an OUTPUT, its voltage will be set to the corresponding value: 5V (or 3.3V on 3.3V boards) for HIGH, 0V (ground) for LOW. However, if the pin is configured as an INPUT, digitalWrite enables (HIGH) or disables (LOW) the internal pullup on the input pin.

```
asm (
  "sbi %0, %1 \n"

  : : "I" (_SFR_IO_ADDR(PORTB)),"I" (PORTB5)
);
```

## digitalWrite LOW

```
asm (
  "cbi %0, %1 \n"

  : : "I" (_SFR_IO_ADDR(PORTB)), "I" (PORTB5)
);
```

## digitalWrite(output)

```
volatile uint8_t output = HIGH; //LOW or HIGH
asm (
  "cpi %2, 0     \n"
  "breq 1f       \n"
  "sbi %0, %1    \n"
  "rjmp 2f       \n"
  "1: cbi %0, %1 \n"
  "2:            \n"

  : : "I" (_SFR_IO_ADDR(PORTB)), "I" (PORTB5), "r" (output)
);
```

## digitalRead

digitalRead simply reads the value from a specified digital pin, either HIGH or LOW.

```
volatile uint8_t status;

asm (
  "in __tmp_reg__, __SREG__  \n"
  "cli                       \n"
  "ldi %0, 1                 \n" //high
  "sbis %1, %2               \n" //skip next if pin high
  "clr %0                    \n" //low
  "out __SREG__, __tmp_reg__ \n"

  : "=r" (status) : "I" (_SFR_IO_ADDR(PINB)), "I" (PINB5)
);
```

## digitalRead Alternative

The Arduino digitalRead function is a nice bit of code. However, it takes more than a cursory glance to determine exactly how it performs[40]. It also compiles into approximately 222 bytes of code, and is slow in comparison to a simplified inline routine:

```
void setup() {
  digitalRead(13);
```

[40] See Yak Shaving, https://ucexperiment.wordpress.com/2013/03/25/arduino-digitalwritepin-value/.

```
}

void loop() { }
```

Versus:

```
volatile uint8_t status;

void setup() {
  asm (
    "in __tmp_reg__, __SREG__ \n"
    "cli         \n"
    "ldi %0, 1   \n"
    "sbis %1, %2 \n" //skip next if pin high
    "clr %0      \n"
    "out __SREG__, __tmp_reg__ \n"

    : "=r" (status) : "I" (_SFR_IO_ADDR(PINB)), "I" (PINB5)
  );
}

void loop() { }
```

The simplified function occupies only 16 bytes. However, since this is inline code vs. a function, every time a digital read is required in your program, it will consume another 16 bytes. Yet, it would take about 13 inline routines to consume about the same amount of code the standard Arduino function uses.
I doubt you'll write a program that uses over 13 read operations. However, you might.

Another minor factor with the inline routine, is that the port and pin must be known at compile time. The port and pin are "hard coded" into assembly instruction, like so:

```
000002A1 1d.9b  SBIS 0x03, 5
```

In the above disassembly, 0x03 is the port (PINB), and 5 is the pin bit (PINB5). Not really a big issue, unless you want to address the pin and port location programmatically. In that regard, you can't use the simplified routine, or any of the C language MACROs floating around the Internet similar to:

```
#define bit_get(p,m) ((p) & (m))
```

But here is a generic alternative, which occupies approximately 34 bytes. Note it must be called using a pointer to the PIN (&PINB), otherwise the compiler will emit incorrect code:

```
//call like so:
//uint8_t status = dRead(&PINB, PINB5);

__attribute__ ((noinline)) uint8_t dRead(volatile uint8_t *port,
uint8_t pin) {
  uint8_t result, mask=1;

  asm (
    "movw  r30, %1 \n" //port reg addr in Z
  "1:              \n"
    "cpi  %2, 0    \n" //loop until pin==0
    "breq 2f       \n" //leave loop
    "lsl  %3       \n" //shift (mask) left 1 position
    "dec  %2       \n" //decrement loop counter
    "rjmp 1b       \n" //repeat
  "2:              \n"
    "in   __tmp_reg__, __SREG__ \n" //preserve sreg
    "cli           \n" //disable interrupts
    "ld   r18, Z   \n" //fetch port data
    "and  r18, %3  \n" //compare pin with mask
    "ldi  %0, 1    \n" //set return high
    "brne 3f       \n"
    "clr  %0       \n" //set return low
  "3:              \n"
    "out  __SREG__, __tmp_reg__ \n"

    : "=&r" (result)
    : "r" (port), "a" (pin), "r" (mask)
    : "r18", "r30", "r31"
  );

  return result;
}
```

## Example of turning off PWM for Arduino digital pin #11

```
//digital PWM pin registers:
//3:  (TIMER2B) PD3/TCCR2A/COM2B1/OCR2B
//5:  (TIMER0B) PD5/TCCR0A/COM0B1/OCR0B
//6:  (TIMER0A) PD6/TCCR0A/COM0A1/OCR0A
//9:  (TIMER1A) PB1/TCCR1A/COM1A1/OCR1A
//10: (TIMER1B) PB2/TCCR1A/COM1B1/OCR1B
//11: (TIMER2A) PB3/TCCR2A/COM2A1/OCR2A
```

```
asm (
  "ld  r16, Z \n"
  "ldi r17, 0xff \n"
  "eor r17, %1 \n"
  "and r16, r17 \n"
  "st  Z, r16 \n"

  : : "z" (_SFR_MEM_ADDR(TCCR2A)), "d" (COM2A1) : "r16", "r17"
);
```